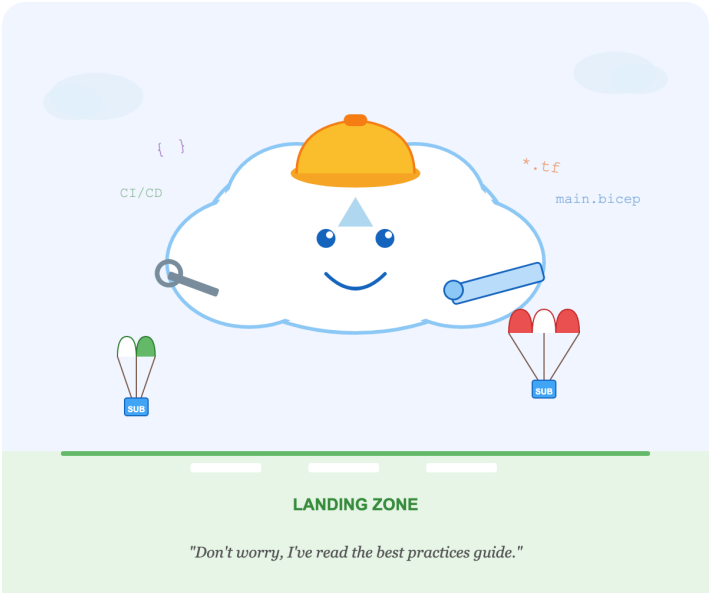


Azure Landing Zones

Infrastructure-as-Code Repository Best Practices

Repository: erjosito/alz-repo-best-practices

Version: d54d25b · 28 April 2026

A field guide for DevOps architects and platform engineers

Azure Landing Zones - Infrastructure-as-Code Repository Best Practices

> *A field guide for **DevOps architects** and **platform engineers** designing the Git repositories, pipelines, and operating model that will deliver an Azure Landing Zone (ALZ) implementation at enterprise scale.*

This guide is **opinionated but balanced**: every chapter focuses on the *design decisions* you have to make, the *tradeoffs* of each option, and a *recommendation* for the typical enterprise scenario. Anti-patterns are called out explicitly.

Why this guide exists

When the first Azure Landing Zone accelerators appeared in 2020-2021, most enterprise teams focused on the *content* of the landing zone: which policies to apply, how to structure management groups, which connectivity model to adopt. The *repository* that would host and maintain that content was treated as an afterthought -- a place to put ARM templates or Terraform files before the real work began. Years later, those same teams found the infrastructure code was solid but the operational model around it had accumulated technical debt that was genuinely hard to pay down: long-lived pipeline secrets that could not be rotated without downtime, state files covering entire environments with no blast-radius control, naming conventions that existed only in a Confluence page nobody had updated since 2022, and modules copy-pasted so many times that fixing a bug meant finding every copy first.

This guide exists to address the repository and operating model -- not the policy library or the network topology, but the decisions that determine whether your ALZ implementation remains manageable at year three as much as at day one. It draws on official Microsoft guidance, community patterns from the IaC ecosystem, and the lessons of platform engineering teams that have operated Azure estates at enterprise scale. The "How we got here" narratives at the start of each chapter trace how the industry's tooling and thinking evolved to produce the current best practices -- so you understand not just *what* to do, but *why* the field landed there.

It is written for platform engineers and DevOps architects who are either building a new ALZ implementation or inheriting one that needs to be brought up to a higher standard. If you are starting fresh, read the chapters in order. If you are inheriting an existing estate, start with the summary below and Chapter 14 -- the anti-pattern checklist is the fastest way to identify where the most pressing technical debt lives.

How to read this guide

The chapters are loosely ordered from "high-level strategy" to "day-2 operations". You can read them in order, or jump straight to the decision you are wrestling with - every page links to its neighbours.

#	Topic	Decision in one sentence
01	Repository topology - monorepo vs multi-repo	How many Git repositories, and what goes in each?
02	IaC tooling - Bicep, Terraform, ARM, Pulumi	Which language/engine for which layer of the platform?
03	Module & registry strategy (AVM, ALZ modules)	Where do reusable modules live and how are they versioned?
04	Branching, environments & promotion	How does a change flow from a developer laptop to prod?
05	Authentication & identity for pipelines	OIDC, federated credentials, MIs, service principals.
06	Secrets & supply-chain security	Secrets handling, signed commits, SBOMs, dependency hygiene.
07	State management (Terraform & deployment stacks)	Backends, locking, blast-radius, drift.
08	CI/CD pipeline patterns	GitHub Actions vs Azure DevOps; PR vs deploy workflows.
09	Testing, validation & policy-as-code	what-if, plan, PSRule, Conftest, OPA, Checkov.
10	Code quality, linting & developer experience	Pre-commit hooks, formatters, devcontainers.
11	Manageability & day-2 operations	Drift, blast radius, ownership (CODEOWNERS), runbooks.
12	Naming, tagging & metadata conventions	The boring stuff that breaks everything if you skip it.
13	Documentation & onboarding	ADRs, READMEs, diagrams-as-code.
14	Anti-patterns & common pitfalls	A checklist of things you will regret.

A consolidated list of external references is in `docs/references.md`.

Scope and assumptions

- **Target audience:** platform engineering teams shipping an ALZ implementation to one or more Azure tenants for an enterprise IT estate.
- **Definition of "ALZ":** the Microsoft Azure Landing Zones conceptual architecture (management groups, policies, platform subscriptions, application landing zones). See the Cloud Adoption Framework - Landing Zone docs.

- **Out of scope:** application-level CI/CD, AKS cluster patterns, Azure Policy authoring (referenced but not deep-dived), and the underlying networking topology (covered only where it affects repository design).

A quick summary for the impatient

If you take nothing else from this guide:

1. **Separate "platform" from "application landing zones"** at the repository level. Mixing them is the single biggest source of long-term pain.
2. **Use OIDC federated credentials** for all pipeline auth. Never store an Azure secret in a CI/CD variable in 2026.
3. **Pin every module by version**, never by `main`. Use semantic versioning and a private registry (or git tags) as the source of truth.
4. **Treat policy as the API contract** between the platform team and application teams - not as a guard-rail bolted on later.
5. **Keep the blast radius of any single `apply` small.** One state file per environment per workload, not one giant state file for the whole tenant.

The rest of this repo explains *why*.

📄 PDF version

A print-ready PDF of the full guide is available at `dist/ALZ-IaC-Best-Practices.pdf` -- optimised for e-readers (reMarkable, Kindle) with rendered diagrams, tables, and a cover page showing the version it was built from.

To regenerate after editing docs:

```
node .github/skills/generate-pdf/scripts/build-pdf.mjs \
  && npx md-to-pdf dist/alz-combined.md --config-file .github/skills/generate-pdf/scripts/pdf-config.json \
  && mv dist/alz-combined.pdf dist/ALZ-IaC-Best-Practices.pdf
```

Requires Node.js 20+. Dependencies (`md-to-pdf`, `@mermaid-js/mermaid-cli`) are fetched automatically via `npx`.

---

01 · Repository topology -- monorepo vs multi-repo

In this chapter:

> ***Decision:*** *how many Git repositories will host your ALZ IaC, and what belongs in each?*

← Back to index · Next → 02 IaC tooling

Every team starting a Landing Zone eventually faces the same paralysing question: one repository for everything, or one per team? The answer shapes your permissions model, your pipeline architecture, your module versioning strategy, and who gets paged when the foundation breaks at midnight. Get it wrong early and the refactoring tax compounds quietly until one day you are coordinating 60 pull requests to bump a VPN module. This chapter maps out the three realistic topologies, names their failure modes honestly, and gives you a decision framework you can use in your next design review.

How we got here

In the early 2010s, when "infrastructure as code" still mostly meant Chef cookbooks and Puppet manifests, the natural home was a single SVN or Git repo per project. Then the **microservices wave** of 2014-2017 cargo-culted the same fragmentation onto infrastructure: every service got its own repo, its own Terraform state, its own pipeline. Within a couple of years, "how do I bump the VPC module across 80 repos?" became a recurring LinkedIn post. By the late 2010s the pendulum swung the other way as Google, Facebook, and Microsoft published essays on the productivity benefits of monorepos backed by purpose-built tooling (Bazel, Buck, Source Depot). For IaC specifically, the Terraform community discovered that **Git submodules don't fix the problem**

and that **module registries do** (Terraform Registry, then private registries, then OCI). Today most

> 📘 ***Key terms***
>
> ***ALZ (Azure Landing Zone)*** *-- a pre-configured Azure environment (management groups, subscriptions, policies, networking) that follows Microsoft's Cloud Adoption Framework and is ready to host workloads securely at scale.*
>
> ***IaC (Infrastructure as Code)*** *-- the practice of defining cloud infrastructure in declarative or imperative code files (Terraform, Bicep, Pulumi) rather than manual portal clicks, enabling version control, review, and automation.*
>
> ***SVN (Subversion)*** *-- a centralised version-control system that predated Git's dominance.*
>
> ***Cargo-culting*** *-- blindly copying a practice from another context without understanding why it worked there.*
>
> ***Git submodules*** *-- a Git feature that embeds one repository inside another; notoriously difficult to manage and a common source of checkout confusion.*
>
> ***Module registries*** *-- hosted catalogues (e.g. Terraform Registry, Azure Container Registry) that distribute versioned IaC modules, replacing error-prone Git-source references.*
>
> ***OCI (Open Container Initiative)*** *-- a standard for container image formats and registries, increasingly used to distribute non-container artefacts such as IaC modules. mature ALZ implementations live in a small number of carefully chosen repos -- neither a single behemoth nor an unmanageable swarm -- and that's the option this chapter recommends.*

Why this is the first decision you make

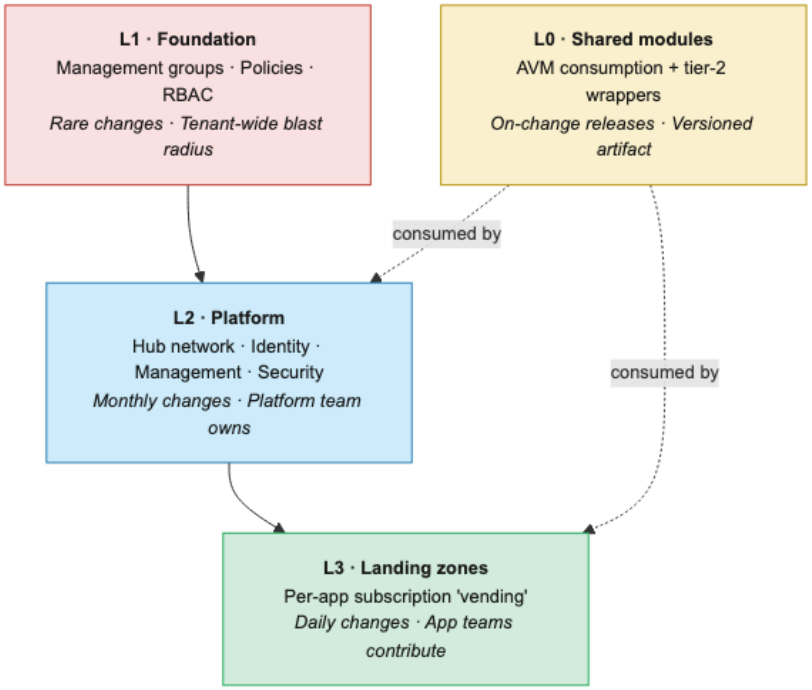
Repository topology drives almost every other choice in this guide: permissions, pipelines, branching, module versioning, and even who is on call for what. Get it wrong and you will spend the next two years either drowning in cross-repo PRs or watching a single broken pipeline freeze the entire estate.

Two extreme positions exist, with a sensible middle ground:

Option	One-liner
Pure monorepo	One repo holds management groups, policies, platform, and every application landing zone.
Pure multi-repo	Every workload, module, and platform component lives in its own repo.
Layered "few-repo" (recommended)	A small, fixed number of repos aligned to ownership boundaries.

The three layers of an ALZ estate

Almost every ALZ implementation has these natural layers. Whether each is a folder or a repo is the question:



L0 sits to the side: it's not a deployment layer at all but a *library* that L2 and L3 consume by version.

With those layers in mind, we can evaluate what each topology looks like in practice -- and more importantly, where each one quietly breaks down.

Option A -- Pure monorepo

Layout sketch

```
alz/
├── foundation/        # mgmt groups, policy assignments
├── platform/
│   ├── connectivity/  # hub VNets, ER, firewall
│   ├── identity/
│   └── management/    # log analytics, automation
├── landingzones/
│   ├── corp-app01/
│   └── online-app42/
├── modules/           # local module library
└── .github/workflows/
```

✅ Pros

- **Atomic cross-layer changes.** "Add a new region" can be one PR that touches policy, hub, and a workload simultaneously.
- **Single source of truth** for naming conventions, tags, module versions.
- **Easy refactoring** with codebase-wide search/replace and IDE rename.
- **One pipeline framework** to maintain.

❌ Cons

> 📘 ***CODEOWNERS*** *-- a file (* `.github/CODEOWNERS` *) that maps file-path patterns to GitHub teams or users who must approve PRs touching those paths. Only enforced when branch protection requires CODEOWNERS review.*

- **Permissions are coarse.** GitHub repo roles apply to the whole repo; fine-grained access requires CODEOWNERS + branch protection gymnastics.

  > 📘 ***Blast radius*** *-- the scope of damage a single failure can cause. In IaC, this is typically defined by what resources a single* `terraform apply` *or deployment stack can touch. Smaller blast radius means a mistake affects fewer resources.*

- **Pipeline scaling.** A naive workflow that runs `terraform plan` on the

  > 📘 ***Path filters & matrix builds*** *-- path filters trigger a pipeline only when files in specific directories change, avoiding unnecessary runs. Matrix builds run multiple jobs in parallel (e.g. one per environment or workload), allowing a single workflow to plan/apply many targets concurrently.*

  whole repo on every PR becomes unusable past ~50 workloads. You **must** invest in path filters and matrix builds early.
- **Blast radius psychology.** Engineers see the foundation code next to their app code and either get scared to commit or, worse, don't.
- **Single CODEOWNERS becomes a 500-line file** that nobody reviews.

When it works

Small estates (<10 application landing zones), one platform team, no strict separation of duties between platform and application teams.

For anything larger -- or any team that has received a polite request to "please add just 20 more workloads this quarter" -- the monorepo cracks start to show. The other extreme makes the ownership tradeoffs explicit.

Option B -- Pure multi-repo

Every workload, every module, every policy set in its own repo.

✅ Pros

- **Crystal-clear ownership** -- repo = team = on-call rotation.
- **Independent release cadence.** Workload A deploys 10× a day; the foundation deploys quarterly. Each gets the cadence it needs.
- **Smallest possible blast radius** per pipeline run.
- **Simple permissions** -- GitHub team → repo mapping is 1:1.

❌ Cons

- **Cross-cutting changes are nightmarish.** Bumping a module version across 60 repos = 60 PRs, 60 reviews, 60 pipeline runs (Renovate/Dependabot helps, doesn't eliminate).
- **Drift between repos.** Conventions, pipeline templates, even tool versions diverge. You end up needing a "templates" repo + scaffolding tool (cookiecutter, Backstage, `gh repo template`) just to keep them aligned.
- **Discoverability suffers.** New engineers ask "where is X?" constantly.

When it works

Very large estates (100+ landing zones), strong platform engineering investment in a self-service portal/scaffolding (Backstage, internal CLI), strict regulatory separation between teams.

Most organisations don't fit neatly into either extreme, which is precisely why the third option exists -- it trades the cleanness of the two poles for something more pragmatic and, in practice, more durable.

Option C -- Layered "few-repo" (recommended default)

The pragmatic compromise. **Three to five repos**, aligned to the natural ownership boundaries:

Repo	Owner	Cadence	Typical blast radius
`alz-foundation`	Cloud governance	Quarterly	Tenant
`alz-platform`	Platform engineering	Weekly	Platform subs
`alz-modules`	Platform engineering	On change (semver)	None (artifact only)
`alz-landingzones` *or* one repo per business unit	App teams + platform reviewers	Daily	One subscription
`alz-policies` (optional)	Security/governance	Weekly	Tenant policy

✅ Pros

- Ownership boundaries match real-world team boundaries.
- Each repo has a *coherent* CODEOWNERS, branch protection, and approval policy.
- Module repo can publish to a registry (ACR / Terraform Cloud / private registry) and consumers pin versions -- see 03 modules & registries.
- Foundation repo can require *much* stricter controls (mandatory pair review, manual approval, change-advisory ticket reference) without slowing daily workload deployments.

❌ Cons

- Cross-repo refactors still require coordination -- but they are rare by construction (foundation rarely changes).
- Need a shared **pipeline template repo** (`alz-pipeline-templates`) or reusable workflows so the four repos don't drift in their CI definitions.

When it works

This is the right answer for ~80 % of enterprise ALZ implementations.

> ⚖️ ***The debate -- monorepo vs few-repo for IaC***
>
> *The recommendation above ("layered few-repo for ~80 % of enterprises") reflects the mainstream ALZ guidance, but informed practitioners disagree.*
>
> ***The monorepo case:*** *Major tech companies (Google, Meta) operate massive monorepos successfully. Their argument: a single source of truth eliminates version-skew between layers, makes cross-cutting refactors atomic, and improves discoverability ("everything is `grep`-able in one place"). With CODEOWNERS, path-scoped branch rules, and mature CI (path filters + matrix builds), monorepos can enforce ownership boundaries nearly as well as separate repos -- without the coordination overhead of 60 PRs for a module bump. Terragrunt and Nx/Turborepo make monorepos practical even at scale.*
>
> ***The few-repo case:*** *In regulated enterprises, repo-level access control is simpler to audit than path-level CODEOWNERS. Independent release cadences (foundation = quarterly, workloads = daily) are naturally expressed by separate repos with separate pipelines. The psychological effect matters too: an app engineer who can see and accidentally edit foundation code is a risk no CODEOWNERS file fully mitigates.*
>
> ***Where the industry stands (2026):*** *Most Azure-focused guidance (CAF, ALZ accelerators) defaults to few-repo. Most Terraform-community guidance defaults to either monorepo + Terragrunt or multi-repo + registry. Neither is wrong -- the choice hinges more on your team's Git maturity and regulatory posture than on any inherent technical superiority.*

If you're still uncertain which model best fits your situation, the questions below will cut through the ambiguity in short order.

Decision framework

Ask these questions in order:

1. **How many teams will commit to IaC?**
   - 1 team → monorepo is fine.
   - 2-10 teams → layered few-repo.
   - 10+ teams → layered few-repo with one landing-zone repo per business unit, or full multi-repo if you have the platform investment.
2. **What is your regulatory posture?**
   - If auditors require separation of duties between *who can change policy* and *who can deploy workloads* → at minimum split foundation/policy from workloads.
3. **Do you have a self-service developer portal?**
   - Yes (Backstage, internal CLI) → multi-repo becomes viable.
   - No → stay layered; the discovery cost of multi-repo is too high.
4. **What is the deployment frequency of each layer?**
   - Wildly different cadences → split repos. Putting a quarterly-changing foundation in the same pipeline trigger as daily workloads is friction.

When ownership boundaries blur -- cross-team resources

Option C assumes clean ownership: the platform team owns the platform repo, app teams own their landing zones. Reality is messier. Some Azure resources sit at the boundary between platform and application, and no repo split eliminates the tension entirely.

Two recurring examples illustrate the pattern:

NSGs -- app-owned resource, platform-mandated rules

Network Security Groups live in the application landing zone subscription. The app team knows which ports their workload needs. But the central security team mandates baseline rules: no `*`-to-`*` inbound, required deny-all at the bottom, specific allowed sources for management traffic.

Recommended pattern: app ownership + policy guardrails.

- The **app team owns the NSG code** in their landing zone folder and submits PRs for rule changes.
- **Azure Policy** (owned by the security team in the policies repo) denies non-compliant rules at deploy time -- e.g. a `deny` policy that rejects NSG rules with `sourceAddressPrefix: *` and `access: Allow`.

- The NSG *module* (in `alz-modules`) bakes in mandatory baseline rules by default and exposes only constrained inputs for app-specific additions.

This keeps the blast radius narrow (app team can only affect their own subscription) while the security team governs the guardrails, not the individual rules.

> 📘 *For legitimate exceptions (a legacy app that genuinely needs a broad rule temporarily), use an **Azure Policy exemption** with an expiry date, linked to a reviewed and approved security ticket -- never an ad-hoc NSG edit.*

Azure Firewall rules -- platform-owned resource, app-sourced knowledge

The Azure Firewall sits in the connectivity subscription. The platform team owns the firewall configuration. But the *knowledge* of which TCP/UDP flows each application needs lives with the app teams -- and platform engineers can't write rules they don't understand.

Recommended pattern: PR-based request flow with optional data-driven automation.

Level 1 -- PR-based requests (works for any team size):

- App teams submit a PR to the **platform repo** adding their required firewall rules (a new `.tfvars` block, a YAML manifest, or a Terraform `locals` entry).
- `CODEOWNERS` requires platform team review before merge.
- The platform pipeline applies the change to the firewall.
- The app team describes *intent* ("app01 needs HTTPS egress to `api.contoso.com`"); the platform team translates to firewall syntax and validates against policy.

Level 2 -- data-driven automation (scales to many teams):

- Each app team maintains a **firewall request manifest** (YAML/JSON) in their *own* repo, with a versioned schema:

```yaml
# alz-landingzones/corp/app01/firewall-requests.yaml
schema_version: "1"
rules:
  - name: app01-to-api
    direction: outbound
    protocol: TCP
    destination: api.contoso.com
    port: 443
    justification: "REST API dependency -- see ADR-012"
```

- A **platform pipeline** reads manifests from all app repos, validates against policy (no overly broad rules, no blocked destinations), generates firewall rule collections, and applies them.
- The app team owns the intent; the platform team owns the implementation and remains the final enforcement point.

Decision heuristic

Scenario	Who owns the code	Who governs
NSGs / UDRs in app subscriptions	App team	Policy (security team)
Shared modules with safe defaults	Platform team (module)	Module API contract
Central firewall / WAF rules	Platform team (via PR or automation)	CODEOWNERS review
DNS records in central DNS zone	Platform team (via PR) or app team (with DNS sharding)	CODEOWNERS review / zone-level RBAC

The common thread: **separate ownership of intent from ownership of implementation**, and enforce the boundary with policy, CODEOWNERS, or both.

DNS sharding -- reducing blast radius for central DNS

Private DNS zones in a hub subscription are a classic ownership bottleneck: every app team needs records, but a single zone like `privatelink.blob.core.windows.net` is a shared, high-blast-radius resource. One bad PR can break name resolution for every workload in the estate.

DNS sharding splits the problem by delegating ownership at the zone or sub-zone level:

- **Zone-per-workload for private endpoints.** Instead of one monolithic `privatelink.*.core.windows.net` zone, create per-workload zones (or use Azure policy to auto-register into the correct zone). Each zone has its own RBAC, its own state/stack, and its own blast radius.
- **Sub-zone delegation for app-owned records.** Delegate `app01.internal.contoso.com` to the app team's subscription. The platform team owns `internal.contoso.com` and the NS delegation; the app team manages records in their delegated zone autonomously.
- **Separate state/stack per DNS zone.** Even without delegation, putting each zone in its own Terraform state or Deployment Stack means a bad `apply` affects only one zone, not all of DNS.

The tradeoff is more zones to manage and more VNet links to maintain. For estates with fewer than ~10 workloads, a single zone with PR-based governance is simpler. Beyond that, sharding pays for itself in reduced coordination cost and smaller blast radius.

Private link DNS zones -- centralised is usually right

For **private link** DNS zones specifically (`privatelink.*.core.windows.net` , `privatelink.database.windows.net` , etc.), the community consensus is increasingly that a **single centralised set** is the right default:

- **Decentralised private link zones** (one per subscription or region) lead to **NXDOMAIN failures** in hybrid scenarios -- on-premises DNS forwarders can only point to one zone, and cross-subscription zone linking creates operational complexity that few teams sustain.

- **Centralised zones** with platform-team ownership and automated record creation (via Azure Policy `DeployIfNotExists` or the private endpoint's built-in DNS integration) are simpler to operate and debug.
- For **disaster recovery**, rather than replicating zones cross-region, consider a DR runbook that redeploys the private link zones in the secondary region. Some organisations accept public endpoint fallback during DR as a pragmatic compromise.

DNS sharding remains the right approach for **non-private-link zones** (custom domains, internal app records) where team-level ownership is valuable.

> 🐿️ ***From the ALZ Weekly Questions*** *-- Private DNS Zones: Centralized vs Decentralized*
> *Most customers who start with decentralised private link DNS zones eventually migrate back to centralised. The NXDOMAIN debugging alone costs more than the perceived flexibility is worth.*

See also 11 manageability for the CODEOWNERS and governance mechanics.

Subscription vending -- repo structure at scale

When your ALZ "vends" (provisions) application landing zone subscriptions, the repo structure for subscription vending deserves its own attention:

One state file per vended subscription, not one giant state. A single Terraform state managing 200 subscriptions means every `plan` touches all 200, every apply locks all 200, and a corruption loses all 200. Instead:

- **Three-layer state architecture:**
  1. **Platform state** -- management groups, policies, hub networking (in the `alz-platform` repo).
  2. **Vending state** -- one state file per vended subscription, each in its own storage account or container. Contains the subscription scaffold: resource groups, RBAC, networking peering, baseline tags.
  3. **Workload state** -- owned by the application team, in their own repo and storage account.
- **Config file per subscription:** Each vended subscription gets a `.tfvars` or `.bicepparam` file in the repo. The pipeline uses `git diff` to build a **matrix** of changed subscriptions and deploys only those in parallel -- not a sequential mega-apply.
- **Protecting vended resources** from workload teams:
  - **Deny assignments via Deployment Stacks** -- the platform team's stack owns baseline resources with `denyDelete` settings.
  - **RBAC at the resource group scope, not subscription scope** -- give app teams Contributor on *their* resource groups, not the subscription.
  - **Deny-action policies with tag-based exclusions** -- prevent deletion of platform-managed resources while allowing app teams to manage their own.

> 📽️ ***From the ALZ Weekly Questions*** *-- Subscription Vending: Repo Structure, Security & Multi-Tenant The git-diff matrix pattern lets you scale to hundreds of subscriptions without pipeline timeouts. Each subscription deploys independently and in parallel.*

Subscription democratisation -- why more subscriptions is usually right

A common early mistake is cramming multiple workloads into a single subscription "to save money." Subscriptions are **free** -- the resources inside them cost money, not the subscription itself. Using fewer subscriptions than needed creates:

- **Larger blast radius** -- a misconfigured RBAC or policy affects all co-hosted workloads.
- **Noisy-neighbour effects** -- one workload's resource locks, API throttling, or quota consumption impacts others.
- **Harder cleanup** -- decommissioning a workload means surgically removing resources rather than deleting a subscription.

The recommended default: **one subscription per workload per environment** (e.g. `app01-prod`, `app01-nonprod`). This gives each workload its own blast radius, quota, and RBAC boundary.

> 📽️ ***From the ALZ Weekly Questions*** *-- Community ALZ Projects + Right-Sized ALZ Moving resources between subscriptions after the fact is painful and sometimes impossible. Start with the right subscription structure from Day 1.*

Multi-region expansion

When expanding an ALZ estate to a second (or third) region:

- **Start with a minimum of two regions** for your platform landing zone even if workloads are initially single-region. This establishes the pattern early.
- **Multi-region expansion is primarily a networking exercise** -- management groups and policies need minimal changes (possibly a region policy adjustment to allow the new location).
- **Lightweight expansion option:** for non-critical workloads, use **global VNet peering** back to the existing hub rather than deploying a full hub-and-spoke in the new region. This avoids the cost of a second firewall, VPN gateway, and ExpressRoute circuit.
- **Deploy a full hub** only when justified by reliability/resiliency requirements and cost -- not as the default.

> 📽️ ***From the ALZ Weekly Questions*** *-- Multi-Region ALZ Expansion The cheapest multi-region expansion is global VNet peering to the existing hub. Save the full hub deployment for workloads that genuinely need local egress.*

Anti-patterns

- ❌ **One mega-repo with one giant Terraform state.** A single `terraform apply` that touches every subscription. The blast radius of a bad commit is the entire estate. See 07 state management.
- ❌ **One repo per resource group.** Granularity gone mad -- you'll spend more time on repo plumbing than on infrastructure.
- ❌ **"Modules" folder duplicated across 40 repos.** Promote to a shared module repo + registry the moment you have *any* duplication.
- ❌ **Mixing application code (Helm charts, app source) in the IaC repo.** Different release cadence, different reviewers, different security model.

Reference layouts

Recommended starter layout (`alz-platform`)

```
alz-platform/
├── README.md
├── CODEOWNERS
├── docs/                    # ADRs, diagrams
├── envs/
│   ├── prod/
│   │   ├── connectivity/
│   │   ├── identity/
│   │   └── management/
│   └── nonprod/
│       └── ...
├── modules/                 # thin wrappers around AVM
├── policies/                # custom policy definitions (if any)
├── scripts/
└── .github/
    ├── workflows/
    └── CODEOWNERS
```

Recommended starter layout (`alz-landingzones`)

```
alz-landingzones/
├── README.md
├── CODEOWNERS               # per-folder ownership
├── _shared/                 # naming, tagging modules
├── corp/
│   ├── app01-prod/
│   ├── app01-nonprod/
│   └── app02-prod/
├── online/
│   └── ...
└── .github/workflows/
```

`CODEOWNERS` uses path globs so each landing zone has its own approvers:

```
/corp/app01-* @org/team-app01
/corp/app02-* @org/team-app02
/_shared/     @org/platform-engineering
```

With topology decided and the reference layouts in hand, you have the structural skeleton of your ALZ estate. The remaining chapters layer in the details -- what language to write in those repos, how to version the modules, how to promote code between environments, and how to keep it all secure. The very next question, tackled in Chapter 02, is which IaC engine those repos will use: a deceptively straightforward choice that carries long-term operational consequences worth thinking through carefully.

References

- Microsoft, *CAF -- Landing zones implementation options*: https://learn.microsoft.com/azure/cloud-adoption-framework/ready/landing-zone/implementation-options
- Microsoft, *ALZ Bicep -- recommended repo structure*: https://github.com/Azure/ALZ-Bicep/wiki
- Hashicorp, *Recommended Terraform code structure*: https://developer.hashicorp.com/terraform/cloud-docs/recommended-practices
- GitHub, *Monorepo vs polyrepo*: https://github.blog/engineering/architecture-optimization/monorepo-vs-polyrepo/
- 📹 ALZ Weekly -- *Subscription Vending: Repo Structure, Security & Multi-Tenant*: https://www.youtube.com/watch?v=11PmT0t6TUI
- 📹 ALZ Weekly -- *Community ALZ Projects + Right-Sized ALZ*: https://www.youtube.com/watch?v=MznCQbT-EZw
- 📹 ALZ Weekly -- *Multi-Region ALZ Expansion*: https://www.youtube.com/watch?v=nii_LF65zyE
- 📹 ALZ Weekly -- *Private DNS Zones: Centralized vs Decentralized*: https://www.youtube.com/watch?v=WnrKlflIiwo

---

← Back to index · Next → 02 IaC tooling

02 · IaC tooling -- Bicep, Terraform, ARM, Pulumi

In this chapter:

> ***Decision:*** *which IaC engine for which layer of the platform? A monolingual estate is simpler; a bilingual estate is sometimes correct.*

← 01 Repository topology · Index · 03 Modules & registries →

Choosing between Bicep and Terraform has launched more heated team debates than almost any other IaC decision. Behind the enthusiasm, the real question is operational: which tool will your team still be maintaining confidently -- under pressure, at an unglamorous 2 a.m. -- in three years? This chapter walks through each contender honestly, including where each one quietly fails, and gives you a decision framework for estates ranging from single-cloud greenfields to messy multi-cloud realities.

How we got here

The first wave of "Azure as code" was bash scripts wrapping `azure-cli` (then `az`), often committed alongside README sentences like *"run these in order, don't forget to set the subscription"*. Microsoft launched **ARM JSON templates** in 2014 -- a genuine declarative model, but the syntax made grown engineers cry. The same year, HashiCorp shipped Terraform; the **AzureRM provider** (2016) gave multi-cloud teams a sane authoring experience and a real `plan` diff, and quickly became the default in enterprises. Microsoft watched the migration and responded with **Bicep** (public preview 2020, GA 2021) -- essentially "ARM JSON, but lovable" -- and then with **Deployment Stacks** (GA 2024) to close the state-management gap that pushed many teams to Terraform in the first place. **Pulumi** (2018) bet on real programming languages and found a loyal niche but never displaced Terraform for ops-led teams. Today the honest answer to "Bicep or Terraform?" is "whichever your team will still be operating well at 2 a.m." -- both are first-class on Azure in 2026.

Summary recommendation

The short version, before we go deeper into each tool's strengths and failure modes:

Layer	Recommended	Acceptable alternative
Foundation (mgmt groups, policy assignments)	**Bicep + Deployment Stacks** *or* **Terraform AzureRM/AzAPI**	ARM templates (legacy)
Platform (hub, identity, mgmt)	**Bicep** *or* **Terraform** -- pick one and stick to it	--
Landing zones (workloads)	Same engine as platform	Mixed only with strong justification
Multi-cloud workloads	**Terraform** or **Pulumi**	--

The biggest mistake is choosing on aesthetics rather than on *who maintains what tomorrow*. The right answer is whichever your engineers can operate confidently at 2 a.m.

> ⚖️ ***The debate -- is "team skill" really the deciding factor?***
>
> *The "2 a.m." framing is popular advice, but critics argue it understates real structural differences between the tools.*
>
> ***The Bicep case beyond skills:*** *Bicep + Deployment Stacks eliminates an entire class of operational problems -- state corruption, state surgery, backend lock contention,* `terraform_remote_state` *coupling -- that affect even skilled Terraform operators. A mediocre team on Bicep may produce safer infrastructure than a good team on Terraform, simply because fewer things can go catastrophically wrong at 2 a.m.*
>
> ***The Terraform case beyond skills:*** `terraform plan` *output is genuinely more reliable for code review than* `az deployment what-if` *(which still has known gaps). Terraform's ecosystem of third-party providers means you manage GitHub repos, Datadog monitors, PagerDuty schedules, and Azure resources in one workflow -- Bicep can't. And for teams already running Terraform, migration cost is real and rarely justified by Bicep's simplicity alone.*
>
> ***Where the industry stands (2026):*** *Azure-only greenfield projects increasingly default to Bicep; brownfield and multi-cloud estates stay on Terraform. The "skills" heuristic is a useful tiebreaker, but pretending the tools are interchangeable undersells the architectural implications of each choice.*

The contenders

Bicep

Microsoft's first-party DSL (domain-specific language) that transpiles (compiles from one high-level language to another) to ARM JSON.

Strengths

- Native Azure -- every resource and API version is available *day one*.
- No state file to manage. Azure Resource Manager is the source of truth.
- **Deployment Stacks** (GA) give a Terraform-like "what is managed by this deployment" model with `denySettings` for drift protection.
- Tight integration with `az deployment` and AVM (Azure Verified Modules).
- Excellent VS Code experience: type checking, intellisense, decompile from ARM.

Weaknesses

- Azure-only. Useless for AWS/GCP, third-party SaaS, GitHub, Datadog, etc.
- Looping/conditionals are less expressive than HCL (HashiCorp Configuration Language) or general-purpose languages.
- Smaller community ecosystem of modules (though AVM closes the gap).
- `what-if` is powerful but has rough edges with complex resources (RBAC role assignments, child resources). Validate empirically.

Terraform (with AzureRM and/or AzAPI providers)

The de-facto multi-cloud standard.

Strengths

- Huge ecosystem -- providers for everything (Azure, AWS, GCP, GitHub, Azure DevOps, AAD/Entra, Datadog, Cloudflare...).
- `plan` produces a clear, reviewable diff -- gold standard for PR workflows.
- HCL is mature and expressive (functions, dynamic blocks, for_each).
- Strong testing story (terratest, native `terraform test`).
- **AzAPI provider** closes the "new Azure feature" gap by calling the ARM REST API directly when AzureRM lags.

Weaknesses

- **State management is your problem.** Backends, locking, secrets in state, state surgery -- all real operational concerns. See 07 state management.
- AzureRM provider sometimes lags new Azure features (mitigated by AzAPI).
- OpenTofu fork (post-BSL licence change) is now production-viable; choosing between Terraform and OpenTofu is a fresh decision in 2026.

> 📘 ***BSL (Business Source Licence)*** *-- in August 2023 HashiCorp relicensed Terraform from the permissive MPL 2.0 to the BSL 1.1, which restricts competitive commercial use.* ***OpenTofu*** *is the community fork that continues under the original open-source licence.*

ARM (JSON)

The original.

Strengths

- Universally supported. Always works, no tooling required.
- Useful as a *transport format* -- Bicep compiles to it; you can decompile existing deployments.

Weaknesses

- Authoring ARM JSON by hand in 2026 is an anti-pattern. Use Bicep.

Pulumi

IaC in real programming languages (TypeScript, Python, Go, C#).

Strengths

- Real language → unit tests with normal frameworks, complex logic, IDE.
- Cross-cloud with one mental model.

Weaknesses

- Smaller community than Terraform.
- Conflates infrastructure and program logic -- easy to write code that's hard to review as a "diff".
- Best suited to teams with a strong software-engineering culture; less ideal for Ops-background platform teams.

ALZ accelerators

These aren't an engine choice -- they're a *starting point* that bootstraps a production-grade CI/CD pipeline, state storage, identities, and an opinionated folder structure so you don't start from a blank repo. Understanding what each accelerator does -- and what it *doesn't* do -- is critical, because the accelerator's output becomes the codebase you will maintain for years.

> 📘 ***Key terms***
>
> ***ALZ accelerator*** *-- an automation package that provisions the scaffolding (repos, pipelines, identities, state storage) needed to deploy an Azure Landing Zone via IaC. The accelerator is a bootstrap; the AVM modules it references are the implementation.*
>
> ***Bootstrap phase*** *-- the one-time setup that creates the version control repo, CI/CD pipelines, managed identities (with federated credentials for OIDC), and Terraform state storage accounts. All accelerators use a PowerShell module (* `ALZ-PowerShell-Module` *) for this step, regardless of whether the resulting IaC is Bicep or Terraform.*

ALZ Bicep accelerator (AVM-based)

Replaces the classic `Azure/ALZ-Bicep` repo (entering extended support; archived February 2027).

Aspect	Detail
Input	Interactive questionnaire via ALZ PowerShell module: target management-group hierarchy, connectivity model (hub-spoke or vWAN), regions, policy defaults, VCS choice (GitHub / Azure DevOps).
Bootstrap	Terraform provisions: a Git repo, CI/CD pipelines (GitHub Actions or ADO Pipelines), managed identities with OIDC federation, and storage accounts for Deployment Stack parameters.
Output	A ready-to-run repo containing AVM Bicep pattern modules (≈16 resource + 3 pattern modules), YAML-driven configuration for management groups, policies, and connectivity, plus pipeline definitions that deploy via Deployment Stacks.
Day-2 workflow	Edit YAML config or Bicep parameters → PR → pipeline runs `what-if` → merge → pipeline deploys via `az stack sub create`.

ALZ Terraform accelerator (AVM-based)

Replaces the classic `Azure/terraform-azurerm-caf-enterprise-scale` module (entering extended support; archived August 2026).

Aspect	Detail
Input	Same PowerShell questionnaire: hierarchy, connectivity model, regions, VCS. Additionally: backend choice (azurerm storage), Terraform/OpenTofu version preference.
Bootstrap	Terraform provisions: a Git repo, CI/CD pipelines, managed identities with OIDC, a storage account for state, and a starter Terraform root module consuming AVM modules.
Output	A repo with a root module referencing AVM Terraform resource and pattern modules, `.tfvars` per environment, and pipeline definitions that run `terraform plan` on PR and `terraform apply` on merge.
Day-2 workflow	Edit `.tf` / `.tfvars` → PR → pipeline runs `plan` + policy checks → merge → pipeline runs `apply`.

ALZ Portal accelerator

Aspect	Detail
Input	Browser-based wizard at aka.ms/alz/portal: click-through selections for management groups, policies, connectivity, and identity.
Bootstrap	ARM deployment directly from the portal -- no local tooling required.
Output	Deployed Azure resources (management groups, policies, hub network) in your tenant. **No repo, no pipeline, no IaC artifacts.**
Day-2 workflow	None -- portal-deployed resources are not under source control.

The portal accelerator is useful for **demos, proof-of-concept, and learning** the ALZ architecture. It is **not suitable for production GitOps** because there is no IaC to review, version, or roll back. If you start here, plan to re-deploy via Bicep or Terraform before going live.

ALZ PowerShell module

The PowerShell module (`Azure/ALZ-PowerShell-Module`) is not a standalone accelerator -- it is the **bootstrap engine** used by both the Bicep and Terraform accelerators. It collects configuration via an interactive questionnaire, then invokes Terraform to provision the VCS repo, pipelines, identities, and state storage. You can also run it non-interactively with a parameter file for repeatable bootstraps.

What all accelerators share

- **Opinionated defaults aligned to CAF** -- management-group hierarchy, default policy assignments, hub network topology.
- **AVM modules under the hood** -- the accelerators don't invent their own resource definitions; they compose AVM resource and pattern modules.
- **You own the output.** The accelerator is a one-time scaffold. Once the repo exists, you maintain it -- updating AVM module versions, customising policies, adding landing zones. The accelerator doesn't "phone home" or auto-update.
- **Interactive mode** -- Running `Deploy-Accelerator` with no parameters launches a guided, interactive questionnaire that walks you through every decision. No need to craft a parameter file upfront.

You should **fork or wrap** the accelerator output, not consume it raw -- see 03 modules & registries.

> 📽️ ***From the ALZ Weekly Questions*** *-- How to use AVM in ALZ, Bicep or Terraform? As of early 2025, AVM is the **only** recommended module set for new ALZ deployments. The Bicep AVM accelerator now uses Deployment Stacks natively, giving Bicep parity with Terraform's state-based lifecycle management.*

SMB and right-sized scenarios

Not every organisation needs the full three-subscription, multi-region ALZ layout. The accelerator now ships **SMB (small and medium business) scenarios** that drastically reduce Day-1 cost and complexity:

Setting	Full ALZ	SMB ALZ
Subscriptions	3+ (management, connectivity, identity)	**2** (management + connectivity combined, plus workload)
Firewall SKU	Standard or Premium	**Basic**
DDoS Protection	On by default	**Off** by default
Regions	Multi-region recommended	**Single region** start

The SMB scenario uses the **same modules and codebase** -- only the `.tfvars` / `.bicepparam` defaults differ. This means you can grow in-place: upgrading from Basic to Standard firewall, enabling DDoS, or adding a second region is a parameter change, not a repo migration.

> 🐿 ***From the ALZ Weekly Questions*** *-- ALZ for SMB The SMB scenario was purpose-built for organisations that need governance guardrails without enterprise-level cost. Start small, grow in-place.*

The ALZ library tool (`alzlibtool`)

The ALZ library is a **data layer** separated from business logic -- a set of JSON/YAML files that define management-group archetypes, policy sets, and role assignments. The `alzlibtool` Go module exposes three key commands:

Command	Purpose
`alzlib check`	Validates the library files against the schema -- catches errors before deployment.
`alzlib generate`	Produces the JSON artefacts consumed by the Terraform provider or Bicep modules.
`alzlib document`	Auto-generates human-readable documentation of the entire policy landscape.

The same Go module powers both the Terraform ALZ provider and the Bicep generation pipeline. The library is **composable**: you can layer ALZ base + Sovereign Landing Zone (SLZ) + your own local overrides.

> 🐿 ***From the ALZ Weekly Questions*** *-- When to use SLZ over ALZ? + alzlibtool The `alzlibtool` is how both Bicep and Terraform stay in sync with the canonical ALZ policy set. Treat the library as data, the tool as compiler, and your IaC as consumer.*

Understanding the Bicep file structure

If you open the Bicep accelerator output, you will find multiple `.bicep` / `.bicepparam` file pairs rather than a single monolithic template. This is not accidental -- ARM has a **4 MB deployment payload limit**, and a full ALZ deployment exceeds it. The accelerator splits the deployment into separate files for management groups, policies, connectivity, and so on.

A future Bicep feature -- **extendable parameters** -- will allow a single parameter file to feed multiple Bicep files, reducing duplication. Until then, expect some parameter repetition across

files.

> *From the ALZ Weekly Questions -- Understanding ALZ Bicep File Structure The Bicep modules can also be consumed **standalone**, without the accelerator. If you only need the connectivity module, reference it directly from the AVM registry.*

Migrating from CAF-Enterprise-Scale

The classic `Azure/terraform-azurerm-caf-enterprise-scale` module enters its archive date on **1 August 2026**. A purpose-built **Golang state migration tool** helps you move:

1. **Phase 1 -- Connectivity and management resources** (VNets, firewalls, Log Analytics). The tool reads your existing state, maps resources from CAF-ES module addresses to AVM module addresses, and generates Terraform `import` blocks.
2. **Phase 2 -- Management groups and policies.** These are trickier because the policy structure changed between CAF-ES and AVM. The tool produces an issues CSV listing resources that need manual attention.

The migration tool works from **any CAF-ES version** -- you don't need to be on the latest before migrating. However, older versions produce more entries in the issues CSV.

Practical advice from the ALZ team: Only import resources you cannot easily delete and recreate (ExpressRoute circuits, firewalls with BGP sessions, DNS zones with live records). For management groups and policies, consider a clean re-deploy and let Deployment Stacks handle the cutover.

> *From the ALZ Weekly Questions -- Migrating from CAF-Enterprise-Scale to AVM The migration tool is also useful outside ALZ -- it can recover or restructure any Terraform state file by mapping old module addresses to new ones.*

Azure Migrate agent for platform landing zones

A preview **Azure Migrate agent** builds on top of the accelerator (it is not a replacement). Available in the Azure portal and VS Code, it lets you describe your desired landing zone in natural language and generates the accelerator configuration:

- Grounded on CAF and ALZ documentation -- recommendations are not hallucinated.
- Uses an **MCP server** for customisation -- you can extend it with your own policies or naming conventions.
- Includes **cost estimation** so you can see the monthly impact before deploying.
- Currently **Terraform-only**; Bicep support is planned.

> *From the ALZ Weekly Questions -- Azure Migrate Agent Preview and Azure Migrate Agent Deep Dive The Migrate agent is ideal for initial exploration and configuration generation. For production deployments, always review the generated output before applying.*

With all the options on the table, the practical question is which combination actually fits your team and organisation.

Decision criteria

1. Skills already on the team

The single best predictor of long-term success. A team fluent in Terraform will ship a better Terraform ALZ than a Bicep one even if Bicep is "theoretically" simpler for Azure-only.

2. Single cloud or multi-cloud?

- Pure Azure, no SaaS configuration → **Bicep** is hard to argue against.
- Any non-Azure resources (GitHub, Entra B2B, Datadog, AWS DR site) → **Terraform**.

3. State tolerance

If your operating model cannot accept a separate state store (security review, operational burden, blob/Cosmos availability concerns), Bicep + Deployment Stacks removes that burden entirely.

4. Speed of access to new Azure features

- Bicep: same day as ARM REST API.
- Terraform AzureRM: weeks-months lag.
- Terraform AzAPI: same day, at the cost of writing ARM-shaped HCL.

5. Policy / "what-if" reliability

`terraform plan` is more accurate than `az deployment what-if` today, although the gap has narrowed considerably with Bicep's `what-if` improvements.

If the criteria above still leave you genuinely split -- often because different platform layers are owned by teams with different skill sets -- a bilingual estate is sometimes the honest answer.

Mixing engines -- when, and how to survive it

Bilingual estates exist, usually because:

- Foundation/policy is in **Bicep** (Microsoft accelerator), workloads are in **Terraform** because app teams already know it.
- Platform is in **Terraform** (multi-cloud DR), but a specific workload uses **Bicep** because it needs a brand-new Azure preview feature.

If you go bilingual, enforce these rules:

1. **One engine per resource.** Never let two engines manage the same resource -- drift wars guaranteed.
2. **Clear boundary at the resource group or subscription level.** Make it impossible for the boundary to be ambiguous.
3. **Shared naming/tagging convention.** Both engines must produce identical outputs.
4. **Document it loudly** in the top-level README -- every new joiner asks "why?".

Example -- same resource in both engines

For reference, here is the same simple deployment (a VNet with two subnets) in each. Notice the difference in *what you have to think about*.

Bicep

```
// modules/network/main.bicep
param location string = resourceGroup().location
param vnetName string
param addressSpace string
param subnets array

resource vnet 'Microsoft.Network/virtualNetworks@2024-05-01' = {
  name: vnetName
  location: location
  properties: {
    addressSpace: { addressPrefixes: [ addressSpace ] }
    subnets: [for s in subnets: {
      name: s.name
      properties: { addressPrefix: s.prefix }
    }]
  }
}

output vnetId string = vnet.id
```

Deploy with:

```
az stack sub create \
  --name plat-net-prod \
  --location swedencentral \
  --template-file main.bicep \
  --deny-settings-mode denyDelete \
  --action-on-unmanage deleteAll
```

Terraform

```
# modules/network/main.tf
variable "location"     { type = string }
variable "vnet_name"    { type = string }
variable "address_space" { type = string }
variable "subnets"      { type = list(object({ name = string, prefix = string })) }

resource "azurerm_virtual_network" "this" {
  name                = var.vnet_name
  location            = var.location
  resource_group_name = var.rg_name
  address_space       = [var.address_space]
}

resource "azurerm_subnet" "this" {
  for_each             = { for s in var.subnets : s.name => s }
  name                 = each.value.name
  resource_group_name  = var.rg_name
  virtual_network_name = azurerm_virtual_network.this.name
  address_prefixes     = [each.value.prefix]
}

output "vnet_id" { value = azurerm_virtual_network.this.id }
```

Deploy with:

```
terraform init -backend-config=...
terraform plan -out=tfplan
terraform apply tfplan
```

The Bicep version is shorter; the Terraform version gives you a richer plan diff and works against AWS tomorrow. Neither is "better" -- they answer different questions.

Anti-patterns

Most IaC tool mistakes fall into two camps: picking a tool for the wrong reasons, or letting two engines drift into each other's territory. The recurring offenders:

- ❌ **Authoring ARM JSON by hand.** Use Bicep and `az bicep decompile` to migrate any inherited templates.
- ❌ **Two engines managing overlapping resources.** Drift wars; pick a side.
- ❌ **Choosing a tool for a single feature.** "I need Terraform because of one Datadog dashboard" -- use the Datadog provider in a *separate* repo rather than rewriting your platform.
- ❌ **Pulumi without a programming culture.** It will rot into spaghetti.
- ❌ **Adopting community ALZ alternatives without due diligence.** Several "vibe-coded" open-source ALZ implementations look polished but lack testing, issue triage, and long-term support. Before adopting any community project, check: contributor count, issue history

(triaged regularly?), end-to-end test coverage, and whether the project has survived more than one Azure API breaking change.

> 📹 ***From the ALZ Weekly Questions*** *-- Community ALZ Projects + Right-Sized ALZ The official ALZ repository has handled ~3 000 issues, triages twice weekly, and runs end-to-end tests. Evaluate any alternative against that baseline.*

With the toolchain chosen, the estate has a shape (Chapter 01) and a language (this chapter). The missing link is reuse: how do you avoid writing the same VNet module for each team that needs one, and how do you update it across 40 consumers without a week of coordinated PRs? Chapter 03 covers the module and registry architecture that turns "we have IaC" into "we have a maintainable IaC estate". The take-home from this chapter is simple: pick the engine your team owns confidently, keep it consistent within a layer, and put any deviation in writing.

References

- Microsoft, *Bicep documentation*: https://learn.microsoft.com/azure/azure-resource-manager/bicep/
- Microsoft, *Deployment stacks*: https://learn.microsoft.com/azure/azure-resource-manager/bicep/deployment-stacks
- Hashicorp, *AzureRM provider*: https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs
- Hashicorp, *AzAPI provider*: https://registry.terraform.io/providers/Azure/azapi/latest/docs
- Azure, *Verified Modules*: https://aka.ms/avm
- Azure, *ALZ-Bicep (Classic -- entering extended support)*: https://github.com/Azure/ALZ-Bicep
- Azure, *ALZ Terraform (Classic -- entering extended support)*: https://github.com/Azure/terraform-azurerm-caf-enterprise-scale
- Azure, *ALZ accelerator (AVM-based, current)*: https://azure.github.io/Azure-Landing-Zones/accelerator/
- Azure, *ALZ Terraform migration guide*: https://aka.ms/alz/tf/migrate
- OpenTofu: https://opentofu.org/
- 📹 ALZ Weekly -- *How to use AVM in ALZ, Bicep or Terraform?*: https://www.youtube.com/watch?v=ry39tWr_SXc
- 📹 ALZ Weekly -- *When to use SLZ over ALZ? + alzlibtool*: https://www.youtube.com/watch?v=r8h7F6IJIqw
- 📹 ALZ Weekly -- *Understanding ALZ Bicep File Structure*: https://www.youtube.com/watch?v=sPA3YWkQ-4s
- 📹 ALZ Weekly -- *Migrating from CAF-Enterprise-Scale to AVM*: https://www.youtube.com/watch?v=DSBWjQlVpSs

- 📹 ALZ Weekly -- *Community ALZ Projects + Right-Sized ALZ*: https://www.youtube.com/watch?v=MznCQbT-EZw
- 📹 ALZ Weekly -- *ALZ for SMB*: https://www.youtube.com/watch?v=cyLhLJEYIkU
- 📹 ALZ Weekly -- *Azure Migrate Agent Preview*: https://www.youtube.com/watch?v=kFS9lNPuXxM
- 📹 ALZ Weekly -- *Azure Migrate Agent Deep Dive*: https://www.youtube.com/watch?v=ODaFlsja308

← 01 Repository topology · Index · 03 Modules & registries →

03 · Modules & registries -- composition, versioning, distribution

In this chapter:

> ***Decision:*** *where do reusable modules live, how are they versioned, and how do consumers pin to a known-good revision?*

← 02 IaC tooling · Index · 04 Branching & environments →

Sooner or later every IaC team writes the same storage account module twice. The second time, someone notices. By the fifth time, the variations have accumulated enough subtle differences that a "simple consolidation" becomes a multi-sprint migration. Modules and registries exist to break that cycle -- but only if the versioning and distribution story is solved upfront. This chapter explains the three-tier model, how to choose where your modules live, and how to version and promote them without grinding platform delivery to a halt.

How we got here

The first Terraform projects had no concept of modules at all -- engineers copy-pasted `.tf` files between repos and patched the differences by hand. When that became unbearable, the community tried **Git submodules** (universally hated for confusing checkouts), then `git::` sources pinned to commits (better, but no semver), then the **public Terraform Registry** (2017) for open-source modules. Private registries followed soon after, either via Terraform Cloud or self-hosted OCI/HTTP backends. Bicep launched without modules at all in 2020, added them in 2021, then added the `br:` **registry protocol** so you could `bicep publish` to ACR. The breakthrough came in 2023 with **Azure Verified Modules (AVM)** -- a single, Microsoft-curated catalogue of Bicep *and* Terraform modules with consistent inputs, baked-in WAF defaults, and a real maintenance commitment. Before AVM, every consultancy shipped its own subtly-broken "vnet module"; after AVM, that's a smell. The three-tier model below builds on this history.

The three-tier module model

A mature ALZ implementation has **three tiers** of modules. Conflating them is the most common mistake.

- **Tier 1** is *consumed*, never modified. If AVM is missing something, contribute upstream.
- **Tier 2** is the *opinionation layer*. This is where your enterprise conventions (tags, log analytics workspace, diagnostic settings, NSG defaults) get baked in *once*.
- **Tier 3** is just glue -- picking pattern modules and parameterising them.

If your landing-zone code is calling AVM resource modules directly, you have no place to enforce your enterprise conventions, and every team will reinvent them.

The good news for tier 1 is that the work has largely been done for you.

Azure Verified Modules (AVM)

AVM is Microsoft's official, supported library of Bicep and Terraform modules. As of 2026 it covers the vast majority of common Azure resources.

Use AVM resource modules as your tier-1 source. Benefits:

- Maintained by Microsoft + community; security & WAF aligned by default.
- Predictable inputs/outputs across resources.
- Diagnostic settings, customer-managed keys, RBAC, private endpoints -- all parameterised consistently.
- Versioned in MCR (Microsoft Container Registry, for Bicep) / Terraform Registry.

> 📘 ***Key terms***
>
> ***SemVer (Semantic Versioning)*** *-- a versioning scheme (`MAJOR.MINOR.PATCH`) where each component signals the type of change: major = breaking, minor = new features, patch = bug fixes.*
>
> ***Conventional Commits*** *-- a commit-message convention (e.g. `feat:`, `fix:`, `chore:`) that enables tools like `release-please` to automate changelog generation and version bumping.*
>
> ***MCR (Microsoft Container Registry)*** *-- Microsoft's public OCI registry (`mcr.microsoft.com`) used to distribute container images and Bicep modules.*
>
> ***OCI (Open Container Initiative)*** *-- an industry standard for container image formats and distribution; registries that speak OCI can also host IaC modules.*
>
> ***DRY (Don't Repeat Yourself)*** *-- a software principle: every piece of knowledge should have a single, authoritative representation in the codebase.*
>
> ***WAF (Well-Architected Framework)*** *-- Microsoft's set of design principles for building reliable, secure, cost-efficient and performant Azure workloads.*

Don't use AVM *pattern* modules in production without wrapping them. They are reference implementations -- useful as scaffolding, not as your enterprise standard.

> ⚖️ ***The debate -- must every estate adopt AVM?***
>
> *AVM is positioned as the default tier-1 source above, but not everyone agrees.*
>
> ***The case for custom tier-1 modules:*** *Organisations with mature, battle-tested module libraries (some pre-dating AVM by years) argue that migrating to AVM for standardisation's sake is cost without benefit. AVM modules are deliberately generic -- they expose every parameter to stay broadly useful, which means your team still writes opinionated wrappers on top. If you already have those opinionated modules, AVM adds a layer without removing one. Additionally, AVM coverage, while broad, is not complete: niche resources or preview features may lag, forcing you to maintain custom modules alongside AVM anyway.*
>
> ***The case for AVM:*** *Starting from AVM means you inherit ongoing security patches, WAF alignment updates, and community contributions without maintaining them yourself. For greenfield estates or teams without a mature module library, building custom tier-1 modules from scratch is reinventing work Microsoft already maintains. AVM's predictable interface (*`diagnostic_settings`*,* `private_endpoints`*,* `role_assignments`*) also makes onboarding faster across teams.*
>
> ***Pragmatic middle ground:*** *Treat AVM as default, not mandatory. Use it where it covers your needs and wrap it where it doesn't. Don't force-migrate stable custom modules that are actively maintained and well-tested -- that's disruption for compliance's sake, not engineering's.*

Bicep AVM example

```
module storage 'br/public:avm/res/storage/storage-account:0.14.3' = {
  name: 'sa-${uniqueString(resourceGroup().id)}'
  params: {
    name: 'stplatlogs${uniqueString(resourceGroup().id)}'
    location: location
    skuName: 'Standard_GRS'
    publicNetworkAccess: 'Disabled'
    tags: tags
  }
}
```

Note `0.14.3` -- **always pin**.

Terraform AVM example

```
module "storage" {
  source  = "Azure/avm-res-storage-storageaccount/azurerm"
  version = "0.6.4"

  name                = "stplatlogs${random_string.s.result}"
  resource_group_name = var.rg_name
  location            = var.location
  account_replication_type = "GRS"
  public_network_access_enabled = false
  tags                = var.tags
}
```

Where do *your* (tier-2) modules live?

AVM handles tier 1. The tier-2 pattern modules -- the ones that bake in your enterprise conventions -- are yours to build and publish. Three viable patterns for where they live:

A) Dedicated `alz-modules` repo + Git tag versioning

- Repo: `github.com/<org>/alz-modules`
- Versioning: SemVer git tags (`v1.4.2`).
- Consumers reference by tag:
  - Terraform: `source = "git::https://github.com/<org>/alz-modules.git//network/hub?ref=v1.4.2"`
  - Bicep: `module x 'git::https://...'` is *not* supported -- use a registry.

Pros: simple, no extra infrastructure. **Cons:** Git-source modules need network access from runners; harder for Bicep.

B) Private OCI registry (Azure Container Registry)

- Publish Bicep modules to ACR via `bicep publish`.
- Publish Terraform modules to a private registry (Terraform Cloud, Azure DevOps Artifacts, JFrog, or any OCI registry with the `tfe` provider).
- Consumers reference by version:

```
module hub 'br:contoso.azurecr.io/bicep/modules/network-hub:1.4.2' = { ... }
```

Pros: identical UX to AVM consumption (`br:`); strong access control; audit trail per pull. **Cons:** ACR + auth setup; modest operational cost. **Recommended** for any organisation past a few teams.

C) Public registry fork

For Terraform, you can publish to the public Terraform Registry if your modules are open source. Rare for enterprises.

Whichever distribution mechanism you choose, it is only as trustworthy as the versioning discipline behind it.

Versioning policy

Adopt **Semantic Versioning** (`MAJOR.MINOR.PATCH`) and define explicitly what each bump means *for an IaC module*:

Bump	Triggered by
MAJOR	Breaking change: input renamed/removed, output renamed, resource type changed in a way that requires manual migration, default behaviour changes that would alter an existing deployment.
MINOR	Backwards-compatible feature: new optional input, new optional resource, new output.
PATCH	Bug fix, doc change, no behavioural change for existing callers.

Enforce with:

- **Conventional Commits** + `release-please` / `semantic-release` to automate tag creation.
- A "compat" test in CI that re-applies the module against a saved fixture and fails on unexpected diffs.

Pinning strategy for consumers

- **Foundation/platform repos:** pin to **exact version** (`= 1.4.2`).
- **Workload repos:** pin to **minor** (`~> 1.4`) so they pick up patches automatically. Major bumps require an explicit PR.
- **Never** float to `main` / `latest` in any environment, including dev. Reproducibility is non-negotiable.

Rollout choreography

Bumping a tier-2 module that 40 landing zones consume needs a process:

1. Cut module release `v1.5.0`.
2. Open auto-PRs to all consumers via Renovate / Dependabot.
3. CI in each consumer runs `plan` / `what-if`. Reviewers see exactly what would change in their workload.
4. Merge happens at the consumer's pace, within an SLA (e.g. 30 days for minor, 90 days for major).
5. Deprecation: the module repo's CHANGELOG flags removal in `v2.0.0`; an automated check in the platform pipeline reports stragglers.

A versioning process only works reliably if the modules themselves are well-structured. What that actually means in practice:

Module design checklist

A pattern module should:

- ☐ Take **opinionated defaults** (tags, diagnostic settings, log analytics workspace ID, private endpoints) so callers can't forget them.
- ☐ Expose **a small input surface** (5-15 parameters, not 50). If you need 50, you have multiple modules masquerading as one.
- ☐ Output **everything a downstream module might need** (IDs, names, principal IDs of identities) -- outputs are cheap, refactoring is not.
- ☐ Include a **`README.md`** auto-generated from the schema (`terraform-docs` , `bicep-docs` , or AVM templates).
- ☐ Include an **`examples/`** folder with at least a `minimal` and a `complete` example. CI deploys both per release.
- ☐ Include **tests** -- `terraform test` or AVM Bicep test patterns.
- ☐ Include a **`CHANGELOG.md`** auto-generated from commits.

Naming and namespacing

For your `alz-modules` repo, group by domain:

```
alz-modules/
├── network/
│   ├── hub/
│   ├── spoke/
│   └── private-dns/
├── identity/
│   ├── managed-identity/
│   └── role-assignment/
├── observability/
│   ├── log-analytics-workspace/
│   └── diagnostic-settings/
├── security/
│   ├── key-vault/
│   └── policy-assignment/
└── compute/
    └── vm-baseline/
```

Module names should describe **the pattern**, not the resource. `hub` is better than `vnet-hub-firewall-bastion-routetable` -- but the README should be explicit about what's inside.

Naming discipline is one safeguard against entropy; the following are the patterns that entropy most reliably produces.

Anti-patterns

- ❌ **Inlined modules copy-pasted between landing zones.** The "we'll DRY it later" trap. Promote to a shared module on the second use.
- ❌ **Pinning to a branch name.** "It worked yesterday" is not a strategy.
- ❌ **A "kitchen sink" module** with 80 boolean toggles. Split it.
- ❌ **Modules that wrap a single AVM resource module with no added opinionation.** Just call AVM directly.
- ❌ **Releasing a major version without a migration guide.** Always include a `MIGRATION.md` for breaking changes.

A mature module ecosystem -- tiered correctly, versioned strictly, and published from a registry -- is what makes the branching and promotion strategies in the next chapter operationally safe. Without pinned modules, "it worked in non-prod" is a coincidence rather than a guarantee; with them, the diff between environments is visible, auditable, and reversible. Chapter 04 picks up the story at the branch level, addressing how code flows from a developer laptop all the way to a production subscription.

References

- AVM -- Azure Verified Modules: https://aka.ms/avm
- AVM Bicep specs: https://github.com/Azure/bicep-registry-modules
- AVM Terraform specs: https://github.com/Azure/terraform-azurerm-avm-template
- `bicep publish` to ACR: https://learn.microsoft.com/azure/azure-resource-manager/bicep/private-module-registry
- Terraform module sources: https://developer.hashicorp.com/terraform/language/modules/sources
- Renovate: https://docs.renovatebot.com/
- Conventional Commits: https://www.conventionalcommits.org/
- SemVer: https://semver.org/

← 02 IaC tooling · Index · 04 Branching & environments →

04 · Branching, environments & promotion

In this chapter:

> ***Decision:*** *how does a change flow from a developer laptop to production, and how does the Git branching model map to environments?*

← 03 Modules & registries · Index · 05 Authentication →

Infrastructure changes are uniquely dangerous in one regard: unlike application code, a broken deploy can leave the network in a half-configured state that affects every workload sitting on top of it. A solid branching and promotion model is the operational safety net -- it determines how much blast radius is possible from a single commit, how quickly a rollback can happen, and whether auditors can trace who approved what. This chapter recommends a specific model, acknowledges the alternative honestly, and explains how to make promotion mechanics reliable rather than ceremonial.

How we got here

When IaC adoption took off around 2016, most ops teams reached for the branching model their app-dev colleagues were using: **GitFlow**. `develop` → `release/*` → `main`, each mapped to an environment, each deploy triggered by a merge. It worked -- until the first hotfix had to be cherry-picked back to `develop`, and then to `release/1.4`, and someone missed one. The next wave of teams tried **GitHub Flow** (one long branch, short feature branches) but kept the "environment = branch" idea, which ran into the same drift problems. The **GitOps movement** (Weaveworks

coined the term in 2017; Flux and ArgoCD popularised it) reframed the question: environments are *folders containing manifests*, not branches *containing code*. Combined with **trunk-based development** practices proven at high-scale shops, "one branch, folder per environment" became the dominant pattern by the early 2020s.

> 📘 ***Key terms***
>
> ***GitFlow*** *-- a branching model with parallel long-lived branches (`main`, `develop`, `release/*`, `hotfix/*`). Powerful for versioned software releases, but error-prone for infrastructure where every branch must converge.*
>
> ***Trunk-based development*** *-- a model where all developers commit to a single long-lived branch (`main` / `trunk`) via short-lived feature branches, keeping integration continuous and merge conflicts small.*
>
> ***GitOps*** *-- an operational pattern where the desired state of infrastructure is declared in Git and a reconciler (Flux, ArgoCD) or CI/CD pipeline continuously applies it, making Git the single source of truth.*
>
> ***Cherry-picking*** *-- a Git operation that copies a single commit from one branch to another. Useful for targeted hotfixes, but dangerous at scale because forgetting to cherry-pick even once creates silent environment drift.*
>
> ***Drift*** *-- the divergence between what your IaC code declares and what actually exists in the cloud. Drift can be caused by manual portal changes, failed applies, or cherry-pick misses between branches.*
>
> ***Feature flags*** *-- conditional toggles (often a boolean variable) that let code exist in `main` without being active in all environments, allowing incomplete features to be merged safely.*
>
> ***Ephemeral environments*** *-- short-lived, disposable environments (e.g. per-PR) spun up for testing and torn down automatically when no longer needed.*

The branch-per-environment model still survives in regulated industries that map approval to branches -- usually because the auditors learned Git from a 2015 tutorial. Whichever history your team carries, the two questions below cut through the legacy.

The two questions you must answer

1. **Branching model:** trunk-based vs. environment-per-branch (GitFlow-ish)?
2. **Environment promotion:** how do you guarantee that what was tested in non-prod is what hits prod?

The answers are coupled -- choose them together. For the vast majority of ALZ deployments, they resolve to the same place.

Recommended: trunk-based + folder-per-environment

The mental model: **`main` is the source of truth for every environment**; the difference between environments lives in parameter files, not in divergent branches. A change moves through environments by being *applied* to each in sequence, gated by approvals -- not by being *merged* between branches.

The repo layout that supports this:

```
alz-platform/
├── envs/
│   ├── nonprod/
│   │   ├── connectivity/   # uses modules @ pinned versions
│   │   ├── identity/
│   │   └── tfvars / params files
│   └── prod/
│       └── ... (same shape, different parameters)
└── modules/
```

- **One long-lived branch:** `main`. All work happens on short-lived feature branches that PR into `main`.
- **Environments are folders**, not branches.
- On PR: `plan` / `what-if` runs for *all affected environments* and is posted as a PR comment.
- On merge to `main`: pipeline applies to **non-prod automatically**, then **gates on a manual approval before prod**.

Why this works

- **No merge hell.** Branch-per-environment models inevitably accumulate cherry-pick debt and "what's actually in prod?" anxiety.
- **The diff between environments lives in code,** in the `envs/<name>/` parameter files -- auditable, reviewable.
- **Linear history** simplifies rollback: revert the merge commit, apply.

Why people resist it

> *"But how do we hotfix prod without releasing the in-flight feature?"*

You don't ship in-flight features to `main` until they're production-ready. Use **feature flags** (parameter toggles, conditional resources) and short branches. If a feature really must land in code but not in prod yet, gate it on an environment parameter:

```
resource "azurerm_firewall_policy_rule_collection_group" "new_rules" {
  count = var.enable_new_rules ? 1 : 0
  ...
}
```

Set `enable_new_rules = false` in `envs/prod/terraform.tfvars` until ready.

Alternative: branch-per-environment ("GitFlow for ops")

```
main      → prod
release/* → staging
develop   → nonprod
feature/* → ephemeral
```

- Pipeline triggers on push to each branch and deploys to the matching environment.
- Promotion = merging `develop` → `release/*` → `main`.

When this works

- Long release cycles (quarterly).
- Strict change advisory boards that need a human to "release" each environment.
- Compliance regimes that map approval to *branch* rather than *deployment*.

When it doesn't

- High-frequency platform changes.
- Multiple environments per "stage" (e.g. multiple non-prod tenants).
- Any team that has ever forgotten to cherry-pick a hotfix back to `develop`.

Recommendation: use it only if your compliance team mandates it. Even then, push back hard.

> ⚖️ ***The debate -- trunk-based vs branch-per-environment***
>
> *This guide recommends trunk-based development, but the question is far from settled -- especially in regulated industries.*
>
> ***The case for branch-per-environment:*** *Some compliance regimes (financial services, government, healthcare) map approval gates to branches, not deployments. Their auditors aren't confused -- they genuinely want a branch-level audit trail where a prod release corresponds to a merge event, not a CI gate. Teams with strict change advisory boards, quarterly release windows, or multi-tenant environments that must not drift between stages find branch-per-environment easier to govern.*
>
> ***The case for trunk-based:*** *Fewer long-lived branches means fewer merge conflicts, fewer forgotten cherry-picks, and a single source of truth. Environment differences live in parameter files, not in code divergence. Most modern CI systems can provide the same audit trail via deployment approvals and environment protection rules.*
>
> ***Where the industry stands (2026):*** *Trunk-based is the dominant recommendation in the DevOps literature (DORA research, Google's engineering practices), but enterprise adoption is uneven. Many teams that switched to trunk-based report fewer incidents; others reverted after losing the "branch = frozen artifact" guarantee their auditors expected. There is no single right answer -- the best model depends on your release cadence, audit requirements, and team discipline.*

Whichever branching model you commit to, the next question is what your environment estate actually looks like -- how many environments, what each one is for, and who controls access to it.

Environment topology

Define your environments **explicitly** and document why each exists. A common pattern:

Environment	Purpose	Subscription model
`sandbox`	Engineer experimentation, throwaway	Single shared sub, auto-cleanup
`dev` (optional)	Integration of in-flight platform changes	Dedicated sub per platform
`nonprod` / `staging`	Pre-prod validation, mirrors prod topology	Dedicated subs
`prod`	Production	Dedicated subs
`dr` (optional)	Disaster recovery	Mirrors prod, in second region

Rules:

1. **Prod and non-prod are in different subscriptions** (often different management groups). Anything else dilutes the value of the environment boundary.
2. **Engineers can `apply` in sandbox.** Pipelines `apply` everywhere else.

3. **Sandbox has aggressive lifecycle management.** Auto-shutdown of VMs, nightly resource group cleanup, hard cost caps.

With the environment set defined, the question is how a change moves between them without the canonical trap: "it worked in staging".

Promotion mechanics

The promotion contract: **what you tested is what you ship.**

Promote the artifact, not the source

- Run `terraform plan` (or `bicep build`) once, store the artifact (plan file / compiled ARM JSON), and *apply that exact artifact* to each environment.
- This protects against:
  - Module version drift between environments (someone bumps a tag mid-flow).
  - Time-of-check / time-of-apply differences in upstream data sources.

In practice:

```
# pseudo-pipeline
- terraform plan -out=tfplan-nonprod -var-file=envs/nonprod.tfvars
- terraform apply tfplan-nonprod
- # Manual approval
- terraform plan -out=tfplan-prod -var-file=envs/prod.tfvars
- terraform apply tfplan-prod
```

Note: you **cannot** apply a non-prod plan to prod (different state files, different resources). The "promote the artifact" pattern in IaC means *promote the same Git SHA + the same module versions + the same pipeline template*, with environment-specific parameter files.

Lock module versions per environment

Pin module versions in `envs/<env>/versions.tf` (or a `module-versions.json` read by Bicep) so you can promote `nonprod` first, observe, then update `prod` to the same version explicitly.

```
envs/
├── nonprod/
│   └── versions.tf       # module "x" { version = "1.5.0" }
└── prod/
    └── versions.tf       # module "x" { version = "1.4.2" } ← lags
```

When non-prod is happy after a soak period, a PR bumps prod to `1.5.0`. The PR diff *is* the promotion.

Of course, that PR diff only means something if the review process attached to it has teeth.

PR requirements

Recommended branch protection on `main`:

- ✅ Require pull request before merging.
- ✅ Require **at least 1** reviewer (2 for foundation/policy repos).
- ✅ Require status checks to pass: `lint`, `validate`, `plan`, `policy-test`, `security-scan`.
- ✅ Require **CODEOWNERS** review for paths under `envs/prod/` and `modules/`.
- ✅ Require branches up to date before merge.
- ✅ Require **signed commits** (see 06 security).
- ✅ Dismiss stale reviews on new commits.
- ✅ Disallow force pushes and branch deletion.

Drift between environments

Drift is inevitable. Make it visible:

- **Scheduled `plan` (or `what-if`) runs** in each environment, weekly. Any non-empty plan posts to a Teams/Slack channel.
- Treat unexplained drift as an incident.

For Bicep with Deployment Stacks, set `denySettings: denyDelete` (or `denyWriteAndDelete`) so out-of-band changes are blocked at the ARM layer. For Terraform, drift detection runs are your only signal -- invest in them.

See also 11 manageability.

There is, however, a complementary pattern that sidesteps long-lived environment drift entirely by making environments disposable.

Ephemeral environments

For pattern modules and platform components, spin up a **PR-scoped environment** automatically:

- Workflow on PR creation: `terraform apply` to a uniquely named resource group (`pr-<number>-<sha>`).
- Run integration tests against it.
- Workflow on PR close: `terraform destroy`.

This is *not* a per-PR copy of the entire ALZ -- that's prohibitively expensive. It's a per-PR slice of the modules being changed.

Keeping environment configurations DRY -- Terragrunt and alternatives

Modules solve the DRY problem for *resource definitions*. A separate DRY problem lurks in *environment configurations*: the backend blocks, provider blocks, and `.tfvars` that differ per environment. When you have five environments × eight workloads, even a well-structured `envs/` folder accumulates significant boilerplate.

> 📘 ***Key terms***
>
> ***Terragrunt*** *-- a thin wrapper around Terraform (by Gruntwork) that generates backend configs, provider blocks, and input variables from a shared template, reducing per-environment boilerplate. It also manages cross-stack dependencies (e.g. "the spoke needs the hub's VNet ID").*
>
> ***Atmos*** *-- a framework by Cloud Posse that provides environment composition, stack configuration inheritance, and component orchestration for Terraform projects.*
>
> ***Terramate*** *-- an orchestration tool that adds code generation, change detection, and execution ordering to Terraform/OpenTofu projects without wrapping the CLI.*

What Terragrunt solves

Problem	Terragrunt approach
Duplicated backend blocks	`generate "backend"` in a root `terragrunt.hcl`
Duplicated provider blocks	`generate "provider"` with inherited variables
Per-env variable files	`inputs = { ... }` inheritance from parent directories
Cross-stack dependencies	`dependency` blocks that read outputs from other stacks
Execution order	`run-all plan` applies stacks in dependency order

Terragrunt remains **widely used** and solves these problems well. It was the de facto standard for multi-environment Terraform orchestration from roughly 2019 to 2023, and many mature estates run it successfully today.

Why we don't recommend it as the default for ALZ

For a *typical ALZ estate* -- three to five environments, a handful of workloads, one cloud -- the extra layer often costs more than it saves:

- **Additional DSL to learn.** Terragrunt's HCL-dialect (`dependency` , `generate` , `include`) is conceptually simple but adds a debugging layer between the engineer and Terraform. Error messages refer to generated files, not the source.
- **Versioning surface.** You now pin Terraform *and* Terragrunt versions, and their compatibility matrix is not always smooth.

- **CI complexity.** Most pipeline examples assume `terraform plan`; Terragrunt `run-all` requires different caching, parallelism, and artifact strategies.
- **Native alternatives have closed part of the gap:**

Terragrunt capability	Native equivalent (2026)
DRY backend config	Partial backend config (`-backend-config=`) + CI variables
DRY provider config	`envs/<env>/provider.tf` shared via symlink or CI template
Per-env variables	`envs/<env>/*.tfvars` + matrix pipeline
Cross-stack data	`terraform_remote_state` data source or explicit outputs piped via CI
Execution order	Pipeline DAG stages / explicit `needs:` / `dependsOn:`
For Bicep	Parameter files + Deployment Stacks handle all of the above natively

Where native tooling still falls short: if your estate has dozens of Terraform stacks with complex inter-dependencies and you want a single `run-all plan` command, Terragrunt (or Terramate) genuinely saves time.

Recommendation

For most ALZ teams: a well-structured `envs/` folder, `.tfvars` per environment, and a CI matrix build achieve the same DRY outcome with **less tooling surface**. Adopt Terragrunt (or Atmos, or Terramate) when the environment × workload matrix grows large enough that the native approach produces measurable duplication pain -- not as a default starting point.

> ⚖️ ***The debate -- Terragrunt and DRY orchestrators***
>
> *The recommendation above ("start native, adopt Terragrunt when the pain is real") is itself debated.*
>
> ***The counter-argument:*** *Experienced practitioners -- including Gruntwork, Cloud Posse, and many platform engineers managing 50+ landing zones -- argue that Terragrunt's overhead is modest compared to the duplication and orchestration bugs you accumulate without it. They point out that "native equivalents" (symlinks, `-backend-config=`, CI variable injection) are fragile workarounds that break in non-obvious ways, whereas Terragrunt's `dependency` blocks and `run-all` give you a tested, deterministic execution graph. From their perspective, the advice to "wait until it hurts" means you only adopt good tooling after accruing tech debt -- the equivalent of saying "don't write tests until you have bugs."*
>
> ***Our position:*** *For a small-to-mid ALZ (fewer than ~15 stacks), the native approach is simpler and has a lower bus-factor risk. Beyond that threshold the tradeoff tilts, and a DRY orchestrator becomes the pragmatic choice. Reasonable engineers disagree on where the tipping point is.*

Anti-patterns

- ❌ **`main` deploys straight to prod with no non-prod stop.** The classic "we'll add staging later".
- ❌ **One state file shared across environments.** A failed `apply` in non-prod can corrupt prod state.
- ❌ **Cherry-picking commits between long-lived branches.** You will lose one eventually; that's how outages start.
- ❌ **"It worked in non-prod" without artefact promotion.** Non-prod and prod ran against different module versions.
- ❌ **Merging your own PR.** Even for "trivial" changes. Especially in foundation repos.

With branching strategy, environment topology, and promotion mechanics in place, the structural decisions for your ALZ estate are largely complete. What remains are the operational details that determine whether the structure stays trustworthy over time: authentication (Chapter 05), security controls, state management, and drift response. Get the three foundational decisions right -- topology, tooling, and promotion model -- and the later chapters are refinement. Get any one of them wrong and no amount of clever pipeline YAML will compensate.

References

- Trunk-based development: https://trunkbaseddevelopment.com/
- GitHub branch protection: https://docs.github.com/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches
- Azure DevOps branch policies: https://learn.microsoft.com/azure/devops/repos/git/branch-policies
- Hashicorp, *Recommended Terraform workflow*: https://developer.hashicorp.com/terraform/cloud-docs/recommended-practices/part1
- Microsoft, *CAF -- environments*: https://learn.microsoft.com/azure/cloud-adoption-framework/ready/considerations/environments

---

← 03 Modules & registries · Index · 05 Authentication →

05 · Authentication & identity for pipelines

In this chapter:

> ***Decision:*** *how do CI/CD runners and engineers authenticate to Azure without ever holding a long-lived secret?*

← 04 Branching & environments · Index · 06 Security →

Every pipeline that deploys to Azure needs to prove its identity. For years that proof came in the form of a secret -- a certificate or client secret tucked into an environment variable, one careless `git push` away from a breach report. This chapter shows how to eliminate that secret entirely using workload identity federation, how to scope the resulting access to the minimum blast radius, and how to make your audit trail actually useful.

How we got here

The history of pipeline auth to Azure is a sequence of *reactions to breaches*. In the early days you minted a **service principal with an X.509 certificate**, mounted it onto your build agent, and prayed nobody copied the PFX. When that proved operationally painful, the community moved to **client secrets** -- easier to handle, easier to leak, and leak they did, repeatedly, in committed YAML files and CI logs. Microsoft shipped **Managed Identities** in 2017, which solved the problem elegantly *for workloads running in Azure*, but CI/CD runners (GitHub- hosted, Jenkins on-prem, GitLab SaaS) still needed long-lived secrets. GitHub announced **OIDC for Actions** in 2021, Entra added support for **workload federated credentials** the same year, and the entire

problem class evaporated almost overnight: the runner asks GitHub for a short-lived signed JWT, exchanges it with Entra, and gets a 1-hour access token. No secret ever exists.

> 📘 ***Key terms***
>
> ***SPN (Service Principal)*** *-- an identity object in Microsoft Entra ID that represents an application or service, used to authenticate non-human workloads to Azure.*
>
> ***MI (Managed Identity)*** *-- an Azure-managed identity attached to a resource (VM, App Service, AKS pod) that obtains tokens automatically without storing any secret. Comes in system-assigned (tied to one resource) and user-assigned (reusable across resources) variants.*
>
> ***OIDC (OpenID Connect)*** *-- an identity layer built on OAuth 2.0 that lets one system prove its identity to another via signed tokens, without exchanging long-lived secrets.*
>
> ***JWT (JSON Web Token)*** *-- a compact, signed token format used to carry identity claims (e.g. "this request comes from repo X, branch main, environment prod").*
>
> ***Entra ID*** *-- Microsoft's cloud identity platform (formerly Azure Active Directory / AAD). Manages users, groups, SPNs, and federated credentials.*
>
> ***PIM (Privileged Identity Management)*** *-- an Entra ID feature that provides just-in-time, time-limited elevation to privileged roles, requiring approval and MFA.*
>
> ***SIEM (Security Information and Event Management)*** *-- a platform (e.g. Microsoft Sentinel) that aggregates logs from across your estate for threat detection and incident response.*
>
> ***Conditional Access*** *-- Entra ID policies that enforce requirements (MFA, compliant device, location) before granting access to resources.By 2024 federated credentials had spread to* ***user-assigned managed identities*** *as well, and Azure DevOps shipped its own equivalent. There is now no defensible reason to store an Azure secret in a CI system -- and yet, the surveys keep showing that most do. This chapter exists to make sure you don't.*

The hard rule

No long-lived Azure credentials in any CI/CD system. Period.

In 2026 there is no good reason to store a service principal client secret or certificate in a GitHub Actions secret, an Azure DevOps service connection of type "manual", or a Jenkins credential. OIDC federation is broadly supported and removes the entire class of "leaked secret" incidents.

What to use, by runner

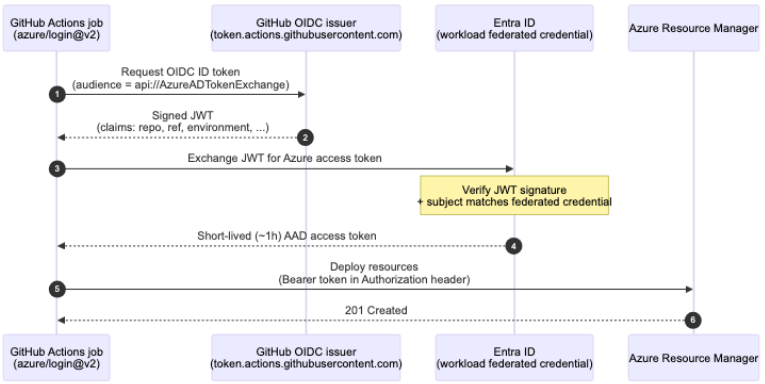
The mechanism varies by CI platform, but the principle is uniform: federate whenever possible, use managed identity for Azure-hosted runners, and reserve interactive `az login` for humans.

Runner	Recommended	How
GitHub Actions	**OIDC → Entra workload federated credential → SPN**	`azure/login@v2` with `client-id`, `tenant-id`, `subscription-id`, no secret
Azure DevOps Pipelines	**Workload identity federation service connection**	"Workload identity federation" connection type
GitLab CI	**OIDC ID tokens → federated credential → SPN**	`id_tokens:` block + `azure/login`
Self-hosted runner on Azure VM/AKS	**Managed Identity**	System- or user-assigned MI on the runner host
Local developer workstation	**`az login` (interactive) + Conditional Access**	Never share an SPN secret with developers

---

OIDC federation -- how it actually works

The whole exchange happens inside a single workflow run. No secret is created, transmitted, or stored -- only short-lived signed tokens cross the wire.



The trust is established once on the **Entra app registration** (or user-assigned managed identity) by adding a *federated credential* that restricts which workflow, environment, branch, or repo can request a token.

Setup (Bicep example)

```
resource app 'Microsoft.Graph/applications@v1.0' = {
  displayName: 'sp-alz-platform-prod'
}

resource fic 'Microsoft.Graph/applications/federatedIdentityCredentials@v1.0' = {
  parent: app
  name: 'github-prod'
  audiences: [ 'api://AzureADTokenExchange' ]
  issuer: 'https://token.actions.githubusercontent.com'
  // Restrict to the prod environment in this specific repo:
  subject: 'repo:contoso/alz-platform:environment:prod'
}
```

Setup (CLI)

```
APP_ID=$(az ad app create --display-name sp-alz-platform-prod --query appId -o tsv)
az ad sp create --id $APP_ID

az ad app federated-credential create --id $APP_ID --parameters '{
  "name": "github-prod",
  "issuer": "https://token.actions.githubusercontent.com",
  "subject": "repo:contoso/alz-platform:environment:prod",
  "audiences": ["api://AzureADTokenExchange"]
}'
```

Choosing a `subject` template

The subject is the most security-critical field. Be **as restrictive as possible**:

Pattern	When to use
`repo:org/repo:environment:prod`	**Recommended.** Ties to a GitHub Environment with reviewers.
`repo:org/repo:ref:refs/heads/main`	Only main branch can deploy.
`repo:org/repo:pull_request`	Read-only plan jobs from PRs (forks excluded).
`repo:org/repo:*`	❌ Too permissive. Anyone with write to *any* branch can assume the SPN.

You can attach **multiple** federated credentials to the same SPN (e.g. one for `prod` environment, one for the `main` branch deploy job).

GitHub Actions consumer

```yaml
permissions:
  id-token: write   # ← required for OIDC
  contents: read

jobs:
  deploy:
    environment: prod   # ← matches the subject filter
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
      - run: az account show
```

Note: those values can be in `vars` (not `secrets`) -- they aren't sensitive.

SPN vs Managed Identity vs User-Assigned MI with federation

OIDC works with more than one identity type, and the choice carries security implications beyond mere convenience.

Identity type	Used for	Notes
Service Principal (app registration)	GitHub / ADO / GitLab pipelines	Add a federated credential -- no secret.
User-assigned Managed Identity	Self-hosted runners on Azure	MIs now also support federated credentials → can be used from non-Azure runners too.
System-assigned Managed Identity	A runner VM/AKS workload that won't be replaced	Tied to the lifecycle of the host.

Microsoft now recommends **user-assigned MI with federation** even for GitHub Actions, because MIs can't have client secrets created against them at all -- defence in depth. SPNs remain valid; both are acceptable.

RBAC scope -- least privilege per environment

Create one identity per **(environment × layer)**:

```
sp-alz-foundation-prod      → Owner @ tenant root MG (rare use)
sp-alz-platform-nonprod     → Contributor @ platform-nonprod MG
sp-alz-platform-prod        → Contributor @ platform-prod MG
sp-alz-landingzone-corp-app01-prod → Contributor @ corp-app01-prod sub
```

- **Do not reuse identities across environments.** A leaked non-prod token must not give access to prod.
- Prefer **Contributor + a few explicit role assignments** (User Access Administrator, Network Contributor) over a blanket Owner.
- Foundation identities need **Owner** at tenant scope to manage RBAC and policy -- restrict their use with strict federated credential subjects and GitHub Environment approval gates.
- Use **Privileged Identity Management (PIM)** for any standing high-privilege human access; pipeline identities should not need PIM elevation since their blast radius is limited by the federated credential subject.

Use Azure Resource Manager scopes thoughtfully

Granting at the management group scope cascades to all subscriptions inside. For landing-zone identities, prefer subscription-scope grants -- they're more auditable.

That covers the Azure plane. Landing zone pipelines, however, rarely speak to ARM alone -- they also interact with GitHub APIs, container registries, secret stores, and third-party tooling.

Authenticating to other systems

Pipelines often need more than just Azure:

Target	Recommendation
GitHub (gh CLI, REST)	`${{ secrets.GITHUB_TOKEN }}` (built-in) or a GitHub App with fine-grained perms
Azure DevOps	Workload identity federation (the same SPN) + ADO PAT only as last resort
Azure Container Registry	`az acr login` after `azure/login` (uses AAD) -- never docker login with credentials
Terraform Cloud / Enterprise	Dynamic provider credentials (TFC's own OIDC to Azure)
Vault	OIDC auth method + AppRole as fallback

The principle is the same: **federate or use an issued token, never store a long-lived secret.**

Pipelines are only one half of the authentication picture. The engineers who trigger, debug, and approve them need their own distinct identity path -- one that never crosses with the deploy SPN.

Developer access

Engineers should **never** hold the same identity as a pipeline. Patterns:

- Engineers `az login` interactively. Conditional Access enforces MFA + device compliance.
- For "deploy from my laptop" cases (sandbox only), they get a *role assignment on the sandbox subscription* via PIM.
- Production access for break-glass: a dedicated emergency account, MFA, PIM-elevated, with full audit alerting. Not the deploy SPN.

Keeping humans and pipelines on separate identities is a prerequisite for meaningful audit. Here is what that audit should actually monitor.

Audit & rotation

OIDC removes secret rotation from your operational burden, but not auditing:

- Stream Entra **sign-in logs** to your SIEM. Filter on the deploy SPN app IDs.
- Alert on:
  - Sign-ins with subject claims that don't match your federation rules (the federated credential should reject these -- alert if they appear).
  - Unexpected source IPs (GitHub-hosted runners come from known ranges).
  - SPN being granted any new role assignment outside the change window.
- Review federated credentials quarterly. Remove anything that no longer maps to an active workflow.
- If you still must use SPN client secrets, rotate them ≤ 90 days, ideally via Key Vault rotation policies -- but really, just move to OIDC.

Multi-tenant scenarios

If your platform spans multiple Entra tenants (acquisitions, sovereign clouds), do **not** create one SPN that has Guest access across tenants. Create one identity *per tenant*, federated separately. The cross-tenant glue lives in the pipeline (it picks the right credential for the target subscription), not in Entra.

Anti-patterns

- ❌ **Service principal client secret stored in GitHub `secrets`** -- whoever can edit the workflow file can exfiltrate it via `echo`.
- ❌ **One SPN with Owner @ root for "convenience"** -- the full-estate blast radius is unacceptable.

- ❌ **Federated credential subject** `repo:org/repo:*` -- defeats the purpose of federation.
- ❌ **Engineers sharing an** `azureuser@contoso.onmicrosoft.com` account for "lab access". Always individual identities.
- ❌ **No alerting on the deploy SPN.** You'd never know if it was abused.
- ❌ **Same SPN used by humans and pipelines.** Audit becomes impossible.

Authentication is the gate that governs everything else in this book. Get it wrong and every other control becomes optional -- a determined attacker holding a long-lived Owner secret can undo your policy assignments, drain your state storage, and cover their tracks before morning. Get it right with OIDC federation, scoped identities, and PIM-gated human access, and the controls in the next chapter become substantially cheaper to enforce. Chapter 06 builds on this foundation to address secrets that genuinely do need to be stored, the integrity of the artefacts your pipeline produces, and the supply-chain risks that no authentication scheme alone can neutralise.

References

- GitHub, *Configuring OIDC in cloud providers -- Azure*: https://docs.github.com/actions/deployment/security-hardening-your-deployments/configuring-openid-connect-in-azure
- Microsoft, *Workload identity federation*: https://learn.microsoft.com/entra/workload-id/workload-identity-federation
- Microsoft, *Federated identity credentials on managed identities*: https://learn.microsoft.com/entra/identity/managed-identities-azure-resources/how-manage-federated-identity-credentials
- Azure DevOps, *Workload identity federation for service connections*: https://learn.microsoft.com/azure/devops/pipelines/library/connect-to-azure
- Microsoft, *Securing Azure pipelines (defender for DevOps)*: https://learn.microsoft.com/azure/defender-for-cloud/defender-for-devops-introduction

---

← 04 Branching & environments · Index · 06 Security →

06 · Secrets & supply-chain security

In this chapter:

> ***Decision:*** *how do you keep credentials out of code, prove the integrity of what's deployed, and stay ahead of supply-chain attacks?*

← 05 Authentication · Index · 07 State management →

Authentication stops the obvious attacker -- the one who shouldn't have the key at all. But a mature IaC pipeline has a wider threat surface: secrets that drift into source code, dependencies quietly poisoned upstream, and runners that, if compromised, can cause more damage than a leaked credential ever could. This chapter addresses that surface systematically, working outward from your own code to the third-party pieces your pipeline trusts.

How we got here

For a long time, "secrets management" in IaC meant *"don't commit them"* -- enforced by code review, vibes, and luck. Luck ran out repeatedly: Uber's AWS keys in a public Git repo (2016), countless `.tfvars` leaks, SAS tokens in `terraform.tfstate` files uploaded to misconfigured backends. Push-side defences emerged around 2018: `truffleHog`, `gitleaks`, eventually GitHub's **native secret scanning** and **push protection**. Then the threat shifted *upstream*: the SolarWinds attack (2020) and the Codecov bash-uploader compromise (2021) showed that a

trusted dependency could be the attack vector. The IaC world felt this directly with the `tj-actions/changed-files` Action compromise of March 2025, which exfiltrated secrets from thousands of pipelines that had pinned by *tag* rather than by *commit SHA*. The response is the defence-in-depth model this chapter describes: keep secrets out of code, sign what you ship (Sigstore, SLSA attestations), pin every dependency by digest, and assume your CI runner will eventually be compromised.

Those four imperatives map directly to the attack vectors worth defending against.

The threat model

For an IaC repo that controls an enterprise Azure estate, the realistic attack vectors are:

1. **Leaked secrets** in Git history.
2. **Compromised contributor account** pushing a malicious change.
3. **Compromised CI runner** exfiltrating tokens or planting backdoors.
4. **Malicious / typo-squatted module dependency** (Terraform Registry, npm, PyPI, GitHub Actions in `uses:`).
5. **State file leakage** exposing secrets baked into resources.
6. **Drift / out-of-band change** introducing an unreviewed configuration.

The mitigations below address each.

Secrets in code -- eliminate, don't manage

The best secret is the one you never store. Combine these:

- **OIDC for cloud auth** (see 05 authentication).
- **Reference Key Vault for application secrets**, never inline them:

```
module app 'app.bicep' = {
  params: {
    sqlPassword: keyVault.getSecret('sql-admin-password')
  }
}
```

- **Generate secrets in-pipeline** (e.g. random storage SAS) and write them straight to Key Vault -- they never appear in logs or state.
- **Mark all sensitive Terraform variables** `sensitive = true` so they don't print in plan output. (They will still appear in state -- see below.)

Pre-commit secret scanning

Run on every developer machine **and** in CI. Two layers catch what one misses.

```yaml
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.0
    hooks:
      - id: gitleaks
  - repo: https://github.com/Yelp/detect-secrets
    rev: v1.5.0
    hooks:
      - id: detect-secrets
```

In CI, run `gitleaks detect` on every PR with `--redact` and fail the build on any finding. Maintain an **allowlist file** for false positives, reviewed quarterly.

GitHub native:

- **Push protection** for secret scanning -- blocks pushes that contain known secret patterns. Enable for every repo.
- **Secret scanning alerts** -- also scans dependency files.

If a real secret is committed: **rotate the credential first**, then purge history (`git filter-repo` / GitHub support). Order matters; the secret was public the moment it was pushed.

Branch protection that matters

Keeping secrets out of the codebase is necessary but not sufficient. The second line of defence is ensuring that a compromised contributor account still cannot merge a malicious change unilaterally.

Setting	Recommended
Require PR before merge	✅
Require review approvals	≥ 1 (≥ 2 for foundation/policy)
Require review from CODEOWNERS	✅
Dismiss stale reviews on new commits	✅
Require signed commits	✅ (Sigstore/gitsign or GPG)
Require linear history	✅
Require status checks	lint, plan, security-scan, policy
Require conversation resolution	✅
Restrict who can push	only the deploy bot for `main`
Disallow force push / deletion	✅

GitHub's **rulesets** let you apply these across all repos in an org -- prefer rulesets over per-repo branch protection for consistency.

With your own contributors controlled, the remaining attack surface is the code you *don't* own: the Actions, modules, and tools your pipeline fetches from the internet.

Supply-chain hygiene

Pin everything by digest, not by tag

Tags are mutable. A malicious maintainer can re-point `v1.0.0` to a compromised commit. Pin GitHub Actions by SHA:

```
# ❌ Bad -- tag is mutable
- uses: hashicorp/setup-terraform@v3

# ✅ Good -- pinned to a SHA
- uses: hashicorp/setup-terraform@b9cd54a3c349d3f38e8881555d616ced269862dd # v3.1.2
```

Use **Dependabot** with `dependency-type: version-update:semver-major` disabled and `version-update:semver-patch` auto-merged after CI passes -- you get security updates without unsupervised major bumps.

For Terraform modules, lock with `.terraform.lock.hcl` (committed) and run `terraform init -upgrade` only via PRs.

SBOM for IaC?

Less mature than for application code, but worth doing:

- For Terraform, generate a module dependency report (`terraform providers schema -json` + custom tooling).
- For Bicep, `bicep build --stdout` then parse `metadata` blocks plus `br:` module references.
- Store SBOMs as build artifacts; ingest into your SCA tool (GHAS, Snyk, etc.).

> 📘 ***Key terms***
>
> ***SBOM (Software Bill of Materials)*** *-- a machine-readable inventory of every component (modules, providers, Actions) your pipeline depends on, used to track known vulnerabilities.*
>
> ***SCA (Software Composition Analysis)*** *-- tooling that scans your dependency tree for known CVEs and licence risks.*
>
> ***GHAS (GitHub Advanced Security)*** *-- GitHub's security suite including code scanning (CodeQL), secret scanning, Dependabot, and dependency review.*
>
> ***SLSA (Supply-chain Levels for Software Artifacts)*** *-- a framework (pronounced "salsa") that defines increasingly strict levels of build integrity, from source to artefact.*
>
> ***Sigstore*** *-- an open-source project providing keyless code-signing and verification, removing the burden of managing GPG/PGP keys.*
>
> ***GPG (GNU Privacy Guard)*** *-- a traditional public-key cryptography tool used to sign Git commits and verify author identity.*
>
> ***CMK (Customer-Managed Key)*** *-- an encryption key you own and manage in Azure Key Vault, used instead of Microsoft-managed keys for data-at-rest encryption.*
>
> `pull_request_target` *-- a GitHub Actions trigger that runs with the base branch's secrets and permissions even when a fork opens a PR. If the workflow checks out and executes fork code, it creates a privilege escalation path.*

Provenance / SLSA

Use GitHub's artifact attestations to sign your deploy artifacts (compiled Bicep, planned Terraform). The deploy job verifies the attestation before applying:

```yaml
- uses: actions/attest-build-provenance@v1
  with:
    subject-path: dist/main.json

# in the deploy job:
- run: gh attestation verify dist/main.json --owner ${{ github.repository_owner }}
```

This makes a "build a malicious artifact and apply it directly" attack detectable.

Signed commits

Require commits to be signed and verified. Two practical options:

- **Sigstore /** `gitsign` -- keyless, OIDC-backed signing. No GPG keys to manage; signatures verifiable on GitHub.
- **GPG / SSH signing** -- traditional, requires key distribution.

In CI:

```
- name: Verify signatures on commits in this PR
  run: |
    base=${{ github.event.pull_request.base.sha }}
    head=${{ github.event.pull_request.head.sha }}
    git log --pretty='%H %G?' "$base..$head" | awk '$2 != "G" { print "unsigned: "$1;
exit 1 }'
```

Securing CI runners

GitHub-hosted runners are convenient but have caveats for sensitive estates:

Concern	GitHub-hosted	Self-hosted on Azure
IP allowlist	Possible via larger runners with static IPs	Trivially yours
Network egress control	Limited	Full (NSG/Firewall)
Tooling control	Microsoft-maintained images	You patch (more work, more control)
Compromise blast radius	High (shared infra)	Limited

Recommended: GitHub-hosted **larger runners with static IPs** for most workflows; self-hosted runners on a **dedicated subscription** for foundation/prod deploys. Self-hosted runners must be:

- Ephemeral (one job per VM, then destroyed). Use the actions-runner-controller on AKS or Azure VM Scale Sets with auto-scale.
- In a network-isolated subscription with egress only to required APIs (ARM, Microsoft Graph, GitHub).
- Authenticated via a Managed Identity (no PAT for runner registration -- use the GitHub App-based runner registration).

Workflow hardening (GitHub Actions)

Securing the runner infrastructure is necessary but not sufficient -- the workflow YAML itself is an attack surface. A handful of settings make an outsized difference.

Top hits from `actionlint` + experience:

```yaml
permissions:                # ← always start with least privilege
  contents: read

concurrency:                # ← prevent duplicate parallel applies
  group: deploy-prod
  cancel-in-progress: false

jobs:
  deploy:
    environment: prod       # ← gates with reviewers
    timeout-minutes: 60     # ← bound runaway jobs

    steps:
      - uses: actions/checkout@<sha>
        with:
          persist-credentials: false   # ← no GITHUB_TOKEN left on disk
      - uses: actions/setup-node@<sha>
      # ...
```

Run `actionlint` and `zizmor` (workflow security scanner) in CI.

Even a hardened, well-scoped workflow produces an artefact that deserves its own security treatment: the Terraform state file.

State file security (Terraform)

State files **contain secrets in cleartext** by default -- DB passwords, storage keys, certs. Treat the state backend as you would a Key Vault:

- Backend storage account: **firewall-restricted**, **private endpoint**, **CMK encryption**, **soft-delete + versioning enabled**, **diagnostic logging to LAW**.
- Access only via the deploy SPN; humans access via PIM with full audit.
- Use `azurerm` backend with `use_oidc = true` so the runner authenticates the same way it authenticates ARM.
- Never download state to a developer laptop.

Details in 07 state management.

Locking down the artefacts your pipeline produces addresses one dimension of compliance. The other is preventing those artefacts from representing non-compliant configurations in the first place.

Policy / compliance gates

Treat policy as a **CI check**, not a deployment afterthought:

- Custom Azure Policy definitions live in version control alongside their assignments.

- On PR, run `Conftest` / `Checkov` / `tfsec` / **PSRule for Azure** against the planned changes -- block the merge on critical findings.
- Detail in 09 testing & policy.

Incident playbook (one paragraph)

When (not if) something happens -- leaked credential, compromised PR -- the response order is:

1. **Revoke** the credential / app role / token.
2. **Rotate** anything touched by the compromised identity.
3. **Audit** Entra sign-in logs and ARM activity logs for the blast window.
4. **Re-deploy** affected resources from a known-good Git SHA.
5. **Postmortem**, then update playbook and protections.

Document this in your repo (`docs/runbooks/credential-leak.md`). When you need it, you won't have time to invent it.

Sovereign Landing Zone (SLZ) -- when compliance demands more

The **Sovereign Landing Zone** is not a separate product -- it's a **variant layer** that sits on top of the standard ALZ. It adds sovereignty controls at three levels:

Level	Controls
L1 -- Baseline	Data residency policies, encryption requirements, audit logging to sovereign region.
L2 -- Enhanced	Confidential computing management groups, restricted service endpoints, HSM-backed key management.
L3 -- Full sovereign	Customer-managed keys everywhere, no data leaving the sovereign region, confidential VMs for management workloads.

SLZ uses the same ALZ library and modules -- the difference is in the **archetype overrides** and additional policy assignments. If you later need to add sovereign controls to a standard ALZ, you can layer SLZ archetypes onto your existing configuration without re-deploying from scratch.

> 🐿️ ***From the ALZ Weekly Questions*** *-- When to use SLZ over ALZ? Think of SLZ as a composable overlay: ALZ base + SLZ layer + your local customisations. The* `alzlibtool` *handles the composition.*

Protecting vended resources -- defence in depth

When subscription vending creates baseline resources (resource groups, networking, diagnostic settings), the platform team needs to prevent workload teams from accidentally modifying or deleting them. Three complementary mechanisms:

1. **Deny assignments via Deployment Stacks** -- when the platform team's Deployment Stack owns baseline resources with `denyWriteAndDelete`, workload teams physically cannot modify those resources even with Contributor role on the subscription.
2. **RBAC at resource group scope, not subscription scope** -- instead of granting `Contributor` on the entire subscription, grant it only on the resource groups the app team owns. The platform-managed resource groups (e.g. `rg-networking`, `rg-diagnostics`) have separate, restrictive RBAC.
3. **Deny-action policies with tag-based exclusions** -- a custom deny policy prevents deletion of resources tagged `managed-by: platform`. App teams can manage resources without that tag freely.

These mechanisms layer on top of each other -- if any one fails, the others still protect the baseline.

> 📽️ ***From the ALZ Weekly Questions*** *-- Subscription Vending: Repo Structure, Security & Multi-Tenant The combination of deny assignments + scoped RBAC + tag-based deny policies gives you defence in depth without blocking legitimate workload operations.*

Anti-patterns

- ❌ **Adding `--allow-secret` patterns to bypass scanning.** That's how real secrets slip through.
- ❌ **`pull_request_target` with checkout of PR code.** Classic privilege escalation vector -- a PR from a fork can read your secrets. Use `pull_request` for untrusted PRs.
- ❌ **Self-hosted runners on a long-lived VM with cached credentials.** A single compromised job persists into the next.
- ❌ **GitHub Personal Access Tokens for cross-repo automation.** Use a GitHub App with fine-grained, time-limited installation tokens.
- ❌ **Disabling `terraform plan` policy checks "just for this PR".** It becomes permanent.

Security is not a single gate but a set of overlapping controls, each assuming the others will occasionally fail. The practices in this chapter -- eliminating secrets at source, hardening runners and workflows, signing artefacts, and enforcing policy checks before merge -- are most valuable precisely because no single one of them is foolproof. Chapter 07 shifts focus from the pipeline itself to what the pipeline produces: the state files and deployment stacks that record the current shape of your Azure estate, and which carry their own considerable operational risk if you get the topology wrong.

References

- GitHub, *Security hardening for GitHub Actions*: https://docs.github.com/actions/security-guides/security-hardening-for-github-actions
- GitHub, *Secret scanning*: https://docs.github.com/code-security/secret-scanning
- OpenSSF, *SLSA*: https://slsa.dev/
- Sigstore, *gitsign*: https://github.com/sigstore/gitsign
- Microsoft, *PSRule for Azure*: https://azure.github.io/PSRule.Rules.Azure/
- Bridgecrew, *Checkov*: https://www.checkov.io/
- Aqua, *tfsec*: https://aquasecurity.github.io/tfsec/
- `gitleaks`: https://github.com/gitleaks/gitleaks
- `actionlint`: https://github.com/rhysd/actionlint
- `zizmor` (Actions audit): https://github.com/woodruffw/zizmor
- 📹 ALZ Weekly -- *When to use SLZ over ALZ?*: https://www.youtube.com/watch?v=r8h7F6IJIqw
- 📹 ALZ Weekly -- *Subscription Vending: Repo Structure, Security & Multi-Tenant*: https://www.youtube.com/watch?v=11PmT0t6TUI

← 05 Authentication · Index · 07 State management →

07 · State management

In this chapter:

> ***Decision:*** *for Terraform, where does state live, how is it locked, and what is the blast radius of a single* `apply`*? For Bicep, the equivalent question is how you carve up Deployment Stacks.*

← 06 Security · Index · 08 CI/CD pipeline patterns →

State is where Terraform keeps its map of the world. It is also, historically, where teams have kept their worst secrets, their most spectacular outages, and their longest Friday evenings. This chapter shows how to design a state topology that contains blast radius, harden the backend that stores it, and -- for those on Bicep -- how Deployment Stacks offer an escape from the problem entirely.

How we got here

The earliest Terraform shops kept `terraform.tfstate` on the engineer's laptop, occasionally emailing it around or -- worse -- committing it to Git. Concurrent applies were "managed" by Slack message (*"@everyone don't apply, I'm applying"*), and the inevitable state-file conflicts were resolved by hand-merging JSON. Hashicorp shipped **remote backends with locking** (S3 + DynamoDB; later `azurerm` with blob leases) around 2017, which solved the concurrency problem but introduced a new one: many teams pointed *every* environment at the same backend and the same state file, producing a single explosive `apply` whose blast radius was the whole estate. The 2018-2022 period was a slow lesson in **state granularity** -- one state per (workload × environment) -- and in **state-as-a-secret-store** hardening. Bicep deliberately avoided the question for years (ARM was the source of truth), but customers kept asking for the *features* state provided: managed-resource tracking, drift protection, "what would removing this line do?"

Microsoft's answer was **Deployment Stacks** (GA 2024), which give you Terraform-style guarantees with no state file to operate. The chapter covers both worlds.

> 📘 ***Key terms***
>
> ***Remote backend*** *-- a shared, network-accessible location (e.g. Azure Blob Storage) where Terraform stores state, enabling locking and team collaboration.*
>
> ***Blob lease*** *-- an Azure Storage locking mechanism that Terraform's `azurerm` backend uses to prevent two concurrent `apply` operations against the same state file.*
>
> ***State surgery*** *-- manually editing Terraform state (via `terraform state mv`, `rm`, or `import`) to fix drift between state and reality without recreating resources.*
>
> ***`force-unlock`*** *-- a Terraform command that manually releases a stuck state lock, typically needed after a runner crash during `apply`.*
>
> ***`denySettings`*** *-- a Deployment Stacks property that prevents out-of-band changes to managed resources (e.g. `denyDelete` blocks manual deletions).*
>
> ***`actionOnUnmanage`*** *-- a Deployment Stacks property that controls what happens to resources removed from the template: `deleteAll`, `detachAll`, or per-type settings.*
>
> ***Azure Blueprints*** *-- a now-deprecated Azure governance service (sunset July 2026) that bundled ARM templates, policy assignments, role assignments, and resource groups into a versioned, assignable package. Superseded by Deployment Stacks.*

Understanding why state management matters operationally is best approached by considering how badly it can go wrong.

Why this is not a footnote

State is the **most operationally dangerous** part of Terraform. A corrupt state file at 2 a.m. on a Friday is a career-defining moment. Get the shape right up front:

- **One state file per (environment × workload).** Not per repo, not per team -- per *unit of deployment*.
- **Backend that supports locking.** No exceptions.
- **Treat state like a secret store.** Encrypted at rest with CMK, network isolated, access audited.

For Bicep, **Deployment Stacks** play the same role: scope = blast radius.

With the principles established, the next question is which backend configuration satisfies them.

Terraform backend choice

Not all backends are created equal for enterprise use. The choice affects locking behaviour, authentication options, network isolation, and whether your OIDC story carries through end to end.

Backend	Recommendation
`azurerm` (Azure Storage)	**Default for Azure-native shops.** Supports OIDC, blob lease locking, CMK, private endpoint.
Terraform Cloud / Enterprise (HCP)	Strong if you also want run UI, Sentinel policy, and dynamic provider credentials. Costs money.
`s3` + DynamoDB	Only if you genuinely have AWS in the picture.
Local state	**Never.** Not even for dev. [1]

[1] *This is a strong stance -- see the debate box below.*

> ⚖️ ***The debate -- local state in sandbox environments***
>
> *"Never local state, not even for dev" is the safest blanket rule, but practitioners with personal sandbox subscriptions and aggressive auto-cleanup policies (e.g. Azure subscription lifecycle management, nightly resource-group purge) argue that remote state adds unnecessary ceremony for truly disposable, exploratory work. A more nuanced position:* ***remote state for anything that persists or is shared; local state is acceptable for isolated sandboxes with automatic teardown*** *-- provided the team understands this is a conscious exception, not the default.*

Recommended `azurerm` backend setup

One **dedicated subscription** (`sub-tfstate-prod`) hosts state for everything. Inside, **one storage account per environment**, **one container per workload**, and **one blob (state file) per environment-workload**.

```
sub-tfstate-prod/
├── rg-tfstate-prod
│   └── sttfstateprod001
│       ├── platform-connectivity-prod
│       │   └── terraform.tfstate
│       ├── platform-identity-prod
│       │   └── terraform.tfstate
│       ├── lz-corp-app01-prod
│       │   └── terraform.tfstate
│       └── ...
```

```
terraform {
  backend "azurerm" {
    resource_group_name  = "rg-tfstate-prod"
    storage_account_name = "sttfstateprod001"
    container_name       = "platform-connectivity-prod"
    key                  = "terraform.tfstate"
    use_oidc             = true
    use_azuread_auth     = true
  }
}
```

Hardening checklist for the storage account:

- ☐ Public network access **disabled**, private endpoint only.
- ☐ **AAD auth** for blob (`use_azuread_auth = true`); shared-key access disabled at the storage account level.
- ☐ **Customer-managed key** (CMK) in a Key Vault you control.
- ☐ **Soft delete** for blobs and containers (≥ 30 days).
- ☐ **Versioning** enabled -- `terraform.tfstate` versions are your "undo" button.
- ☐ **Diagnostic logs** to a Log Analytics workspace; alert on any access from outside the runner subnets.
- ☐ **RBAC**: only the deploy SPN has `Storage Blob Data Contributor`; humans only via PIM with elevation alerts.

Locking

Terraform's `azurerm` backend uses **blob leases** for locking. This is sufficient and battle-tested. Behaviour:

- Each `plan` / `apply` acquires a lease on the state blob.
- If a lease is held, other operations fail fast with the lock ID.
- If a runner crashes mid-apply, the lease eventually expires (default 60s acquire retry) -- but a manual `terraform force-unlock <ID>` is sometimes needed.

Pipeline contract

- **Concurrency control** at the pipeline level (`concurrency: deploy-<env>` in GitHub Actions) so you never enqueue two `apply`s for the same state.
- If a job fails mid-apply, the next job blocks on lock, alerts the team, and someone investigates *before* force-unlocking.

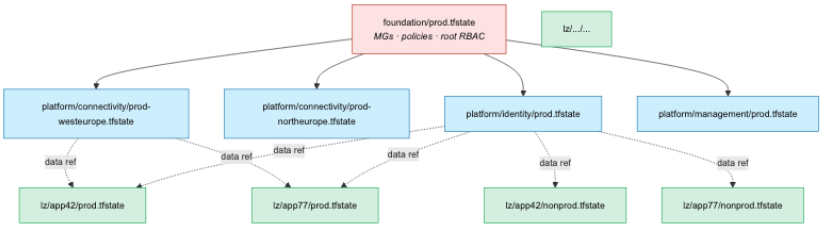
Locking prevents the concurrent-apply catastrophe. The complementary concern is the sheer amount of infrastructure a single apply can reach -- which is entirely a function of how you carve your state files.

Blast radius -- how to size state files

The smaller the state, the smaller the blast radius of a corruption or a bad apply, but the more cross-state references you need. The sweet spot:

Layer	Recommended state granularity
Foundation	One state per *environment* (prod, nonprod).
Platform -- connectivity	One state per *environment* × *region* (one for `prod-westeurope`, one for `prod-northeurope`).
Platform -- identity / mgmt	One state per *environment*.
Landing zone	One state per *(workload* × *environment)*. Never share.

Visualised as a tree, a healthy state layout is wide and shallow -- many small files at the leaves, a handful at the root:



Rule of thumb: **a single `apply` should touch resources owned by one team and deployable in under 15 minutes.** If `plan` takes 20 minutes, your state is too big.

Subscription vending -- the three-layer state model

When your estate vends (provisions) application landing zone subscriptions at scale, a **three-layer state architecture** prevents the common trap of one giant state file managing hundreds of subscriptions:

Layer	Scope	Storage	Owned by
Platform	Management groups, policies, hub networking	Dedicated storage account in the management subscription	Platform team
Vending	One state file per vended subscription -- resource groups, baseline RBAC, VNet peering, tags	Separate storage account (or container per subscription)	Platform team (automated)
Workload	Application resources inside the vended subscription	App team's own storage account	Application team

This separation means:

- A **bad vending apply** only corrupts one subscription's scaffold, not the entire estate.
- **Parallel deployments** -- each subscription's state file is independent, so your CI pipeline can deploy 50 subscriptions concurrently.
- **Clear ownership** -- the platform team owns layers 1-2; the app team owns layer 3. State permissions follow the same boundary.

For Bicep estates, the same principle applies via Deployment Stacks: one stack per vended subscription, scoped at subscription level.

> 📽️ ***From the ALZ Weekly Questions*** *-- Subscription Vending: Repo Structure, Security & Multi-Tenant The three-layer model also lets you use `git diff` in CI to build a matrix of changed subscriptions -- only deploying what changed, not re-planning the entire estate.*

Cross-state references

Workload state needs the hub VNet ID. Two patterns:

A) `terraform_remote_state` data source

```
data "terraform_remote_state" "connectivity" {
  backend = "azurerm"
  config = {
    resource_group_name  = "rg-tfstate-prod"
    storage_account_name = "sttfstateprod001"
    container_name       = "platform-connectivity-prod"
    key                  = "terraform.tfstate"
    use_oidc             = true
  }
}

resource "azurerm_virtual_network_peering" "to_hub" {
  remote_virtual_network_id =
data.terraform_remote_state.connectivity.outputs.hub_vnet_id
  ...
}
```

Pros: simple, no extra infrastructure, real-time. **Cons:** the consumer needs **read access to the producer's state file** -- which contains all of the producer's secrets in cleartext.

B) Outputs in Azure (recommended)

The producer writes its outputs to **Azure App Configuration** or **tagged on a known resource group**:

```
resource "azurerm_app_configuration_key" "hub_vnet_id" {
  configuration_store_id = data.azurerm_app_configuration.platform.id
  key                    = "platform/connectivity/hub-vnet-id/${var.env}"
  value                  = azurerm_virtual_network.hub.id
  label                  = var.env
}
```

The consumer reads via the same App Configuration:

```
data "azurerm_app_configuration_key" "hub_vnet_id" {
  configuration_store_id = data.azurerm_app_configuration.platform.id
  key                    = "platform/connectivity/hub-vnet-id/${var.env}"
  label                  = var.env
}
```

Pros: consumers never see producer state. Outputs are first-class, versioned in App Configuration, with point-in-time history. **Cons:** an extra component; subtle race if consumer reads before producer publishes.

For Bicep, the equivalent is `existing` lookups or App Configuration / RG tags -- Bicep has no remote-state concept by design.

Even the best-designed topology will eventually require manual intervention -- a resource renamed mid-flight, an import needed for something created out-of-band, a module refactor that moved resources between states. That is what state surgery is for, and it deserves a ceremony proportionate to the risk.

State surgery -- when you must

Avoid it. When unavoidable:

1. **Always back up** the current state first: `terraform state pull > backup-$(date +%s).tfstate`
2. Use `terraform state mv` / `rm` / `import` rather than hand-editing JSON.
3. Do it from a controlled runner, not your laptop, so the action is logged.
4. Open a PR with a `STATE-SURGERY.md` describing the why and the commands.
5. Run `plan` afterwards -- it must show **no changes**, otherwise stop.

For the `import` workflow, prefer **HCL `import` blocks** (Terraform 1.5+) over the legacy CLI command -- they are reviewable in PR.

```
import {
  to = azurerm_resource_group.legacy
  id = "/subscriptions/.../resourceGroups/rg-legacy"
}
```

All of the above applies to Terraform. If you are working in Bicep, you have a different but equally powerful option -- one that sidesteps the state problem altogether.

Migrating state from CAF-Enterprise-Scale to AVM

If your estate was built on the classic `Azure/terraform-azurerm-caf-enterprise-scale` module, you'll need a state migration to move to the AVM-based modules. The ALZ team provides a **Golang state migration tool** that automates much of this:

1. **Phase 1 -- Connectivity and management resources.** The tool reads your existing state, maps resource addresses from CAF-ES module paths to AVM module paths, and generates Terraform `import` blocks. Resources like VNets, firewalls, Log Analytics workspaces, and ExpressRoute circuits move cleanly.
2. **Phase 2 -- Management groups and policies.** The policy structure differs significantly between CAF-ES and AVM. The tool produces an **issues CSV** listing resources that require manual review or re-creation.

Practical guidance:

- The tool works from **any CAF-ES version** -- you don't have to be on the latest before migrating (though older versions produce more issues).
- **Only import resources you cannot easily delete and recreate** -- ExpressRoute circuits, firewalls with live BGP sessions, DNS zones with active records. For management groups and policies, a clean re-deploy with Deployment Stacks handling the cutover is often simpler.
- The accelerator generates a starting Terraform configuration with migration comments showing where to add `import` blocks.
- The migration tool is **reusable** beyond ALZ -- it can restructure any Terraform state file by mapping old module addresses to new ones.

> 📽️ ***From the ALZ Weekly Questions*** *-- Migrating from CAF-Enterprise-Scale to AVM*
> *Think of migration as resource mapping → attribute mapping → `terraform apply` with import blocks. The tool handles the first two steps; you review and apply.*

Bicep -- Deployment Stacks

Bicep doesn't have state, but **Deployment Stacks** give you the same benefits:

- A stack tracks the resources it deployed.
- `denySettings` blocks out-of-band modifications (`denyDelete` or `denyWriteAndDelete`).
- `actionOnUnmanage` controls what happens to resources removed from the template (`detach`, `delete`, or `deleteAll`).

Carve stacks the same way you'd carve Terraform state -- one stack per (workload × environment), scoped at subscription or resource group.

```
az stack sub create \
  --name lz-corp-app01-prod \
  --location swedencentral \
  --template-file main.bicep \
  --parameters @prod.bicepparam \
  --deny-settings-mode denyWriteAndDelete \
  --deny-settings-excluded-actions "Microsoft.Compute/virtualMachines/start/action" \
  --action-on-unmanage deleteAll
```

Tradeoff vs raw `az deployment`: stacks are slower to create but vastly safer for production. For ephemeral PR environments, raw deployments are fine; for prod, stacks.

A key Day-2 benefit: when the ALZ library removes a deprecated policy assignment, the Deployment Stack's `actionOnUnmanage: deleteAll` setting **automatically cleans up the orphaned assignment** -- no manual intervention required. This closes a gap that previously made Bicep ALZ harder to maintain than Terraform ALZ for policy lifecycle management.

> 🎥 ***From the ALZ Weekly Questions*** *-- How to Stay Current with ALZ Azure Policies*
> *Deployment Stacks gave Bicep ALZ automatic cleanup of deprecated policies -- matching what Terraform achieves through state tracking.*

> ⚖️ ***The debate -- are Deployment Stacks production-ready at scale?***
>
> *Deployment Stacks GA'd in late 2024 and are maturing rapidly, but the community has less collective operational experience with them than with Terraform state, which has a decade of battle scars.*
>
> ***Concerns from practitioners:*** `what-if` *for stacks is still in preview (as of early 2026) and less reliable than* `terraform plan`*. Tooling around stacks -- refactoring, automated drift detection, import of existing resources -- is thinner than Terraform's mature ecosystem (Spacelift, Env0, state-mv, etc.). Teams running early production stacks have reported edge cases with complex resource dependencies and* `denySettings` *interactions that are not yet well-documented. Migration from Blueprints is documented but not extensively battle-tested by the community.*
>
> ***The optimistic view:*** *Stacks remove an entire category of operational pain (state corruption, lock contention, backend availability). Microsoft is investing heavily, and the feature set is advancing quickly. For new Bicep estates, stacks are the obvious default; the rough edges are real but narrowing.*
>
> ***Bottom line:*** *If you're starting greenfield with Bicep, Deployment Stacks are the right bet. If you're evaluating a migration from Terraform specifically to avoid state, make sure you're gaining more than you're giving up in ecosystem maturity. Test thoroughly in non-prod before trusting* `denySettings` *with your production estate.*

Deployment Stacks vs Azure Blueprints (deprecated)

Teams migrating from **Azure Blueprints** (deprecated July 2026) will recognise the ambition: both aim to deploy a *set* of resources as a governed unit with lifecycle tracking and policy enforcement. The execution is very different.

Capability	Azure Blueprints	Deployment Stacks
Status	Deprecated (July 2026)	GA (2024)
Scope	Management group or subscription	Management group, subscription, or resource group
Template language	ARM JSON only	Bicep or ARM JSON
Versioning	Built-in (draft → published → assigned)	External -- you version the template in Git; the stack always applies the latest
Drift protection	Resource locks applied per-artefact	`denySettings` at the ARM layer -- more granular, supports excluded principals and actions
Unmanage behaviour	Resources orphaned on blueprint deletion	Configurable via `actionOnUnmanage`: delete, detach, or per-type
Policy / role bundling	Blueprints could assign policies and roles as artefacts	Stacks deploy whatever the template contains -- policies and roles are just resources
Sequencing	Built-in artefact sequencing	Template-level `dependsOn` (standard ARM/Bicep)
What-if / plan	No	`az stack ... --what-if` (preview)
State storage	Azure-managed (opaque)	Azure-managed (opaque) -- same model, better transparency

What Blueprints got right: the idea that a landing zone is a *governed package* -- not just resources but also policies, roles, and locks deployed as a single unit with lifecycle tracking. That concept survives in Stacks.

What Blueprints got wrong: coupling to ARM JSON, an opaque versioning system disconnected from Git, no real plan/what-if, and insufficient adoption to justify continued investment.

Migration path: replace each Blueprint assignment with a Deployment Stack whose Bicep template reproduces the same resources, policy assignments, and role assignments. The `denySettings` property replaces Blueprint locks, and Git versioning replaces the built-in draft/published workflow. Microsoft provides a migration guide.

Drift detection

Run a scheduled `plan` (or `what-if` for Bicep) per state/stack, weekly:

- Empty diff → all green.
- Non-empty diff → post to a "platform drift" Teams channel; create an issue; require triage within an SLA.

For Bicep stacks with `denySettings`, drift should be impossible by construction -- but human break-glass paths exist, so verify.

See 11 manageability for full drift handling.

The anti-patterns below are the architectural choices most likely to turn a controlled `apply` into an all-hands incident.

Anti-patterns

- ❌ **One state file for the whole estate.** "Apply" becomes a ritual, changes pile up, and one mistake breaks everything.
- ❌ **Local state on engineer laptops.** Even in dev. Especially in dev.
- ❌ **State backend in the same subscription as the resources it manages.** A bad `apply` could nuke the storage account holding the state. Put state in its own subscription.
- ❌ **Sharing state files across teams via `terraform_remote_state` without realising it leaks all producer secrets.** Use App Configuration outputs instead.
- ❌ **Editing state files by hand.** Even with `jq`. There is no validation; one typo and the state is unrecoverable.
- ❌ **Bicep deployments without stacks for prod.** You lose drift protection.

State -- or its Bicep equivalent -- is the operational ledger for your entire Azure estate. The decisions in this chapter determine whether that ledger is a reliable audit trail or a liability waiting to ruin someone's weekend. A granular, hardened, properly-referenced state topology is also the prerequisite for the CI/CD pipeline patterns in the next chapter: once you know exactly what each apply touches and why it is safe to run concurrently, designing the pipeline around it becomes considerably more straightforward.

References

- Hashicorp, *Backend configuration*: https://developer.hashicorp.com/terraform/language/settings/backends/azurerm
- Hashicorp, *State*: https://developer.hashicorp.com/terraform/language/state
- Microsoft, *Deployment stacks*: https://learn.microsoft.com/azure/azure-resource-manager/bicep/deployment-stacks
- Microsoft, *Migrate from Blueprints to Deployment Stacks*: https://learn.microsoft.com/azure/governance/blueprints/concepts/deployment-stacks-migration
- Microsoft, *Azure Blueprints deprecation*: https://learn.microsoft.com/azure/governance/blueprints/overview
- Microsoft, *Storage account security baseline*: https://learn.microsoft.com/security/benchmark/azure/baselines/storage-security-baseline
- Hashicorp, `import` *blocks*: https://developer.hashicorp.com/terraform/language/import

- 📹 ALZ Weekly -- *Subscription Vending: Repo Structure, Security & Multi-Tenant*: https://www.youtube.com/watch?v=11PmT0t6TUI
- 📹 ALZ Weekly -- *Migrating from CAF-Enterprise-Scale to AVM*: https://www.youtube.com/watch?v=DSBWjQlVpSs
- 📹 ALZ Weekly -- *How to Stay Current with ALZ Azure Policies*: https://www.youtube.com/watch?v=ddcVKS_MKkk

---

← 06 Security · Index · 08 CI/CD pipeline patterns →

08 · CI/CD pipeline patterns

In this chapter:

> ***Decision:*** *what does the workflow that turns a commit into a deployed Azure resource look like -- and how do you keep it consistent across many repos?*

← 07 State management · Index · 09 Testing & policy →

Every commit that touches IaC code starts a race: will the pipeline catch the problem, or will it reach production? This chapter is about structuring that pipeline so the answer is reliably the former. By the end, you will have a working mental model of the two-workflow shape, reusable pipeline templates, and the operational patterns -- retry, break-glass, caching -- that separate a mature IaC pipeline from a fragile script held together with `sleep 30`.

How we got here

Early infrastructure pipelines were **Jenkins freestyle jobs** with the Terraform commands pasted into a textbox; the pipeline definition lived in the Jenkins UI, not in Git, so reviewing a deploy meant taking a screenshot. The "pipeline-as-code" movement (Jenkinsfile in 2016, then Travis/CircleCI YAML, then Azure Pipelines YAML in 2019, then GitHub Actions in late 2019) finally put deploy logic into version control -- but every repo copy-pasted the same workflow, and within a year the copies had drifted. **Reusable workflows** (GitHub Actions, 2021) and **YAML templates with `extends:`** (Azure DevOps) gave central platform teams a way to own the pipeline once and have every consumer stay in sync. Around the same time, the GitOps community pushed the idea of **ephemeral, short-lived runners** -- and after a string of self-hosted runner compromises in 2022-2024, that became the security baseline. The patterns in this chapter assume

reusable workflows, ephemeral runners, and OIDC auth -- the consensus shape of an IaC pipeline in 2026. Before getting into those patterns, one question needs settling: which platform?

> 📘 ***Key terms***
>
> ***Pipeline-as-code*** *-- defining your CI/CD pipeline in a version-controlled file (e.g.* `.github/workflows/*.yml`, `azure-pipelines.yml`*) rather than a GUI, so it is reviewable, auditable, and versionable alongside the code it deploys.*
>
> ***Reusable workflows*** *-- GitHub Actions feature (* `workflow_call`*) that lets you define a workflow once in a central repository and call it from many consumer repos, keeping pipeline logic DRY.*
>
> ***Matrix builds*** *-- a CI/CD pattern that spawns parallel jobs for each entry in a list (e.g. one job per environment or workload), so changes to many targets are validated concurrently.*
>
> ***Ephemeral runners*** *-- CI/CD build agents that are created for a single job and destroyed afterward, eliminating the risk of credential leakage or cross-job contamination.*
>
> `workflow_dispatch` *-- a GitHub Actions trigger that allows manually starting a workflow from the UI or API, typically used for break-glass operations.*
>
> ***Idempotent*** *-- a property meaning that running the same operation multiple times produces the same result as running it once. Critical for* `apply` *retries: re-applying the same plan should not create duplicate resources.*
>
> ***DX (Developer Experience)*** *-- the overall quality of the tooling, documentation, and workflows from a developer's perspective.*

GitHub Actions vs Azure DevOps Pipelines

Both can do the job. Pick on these criteria:

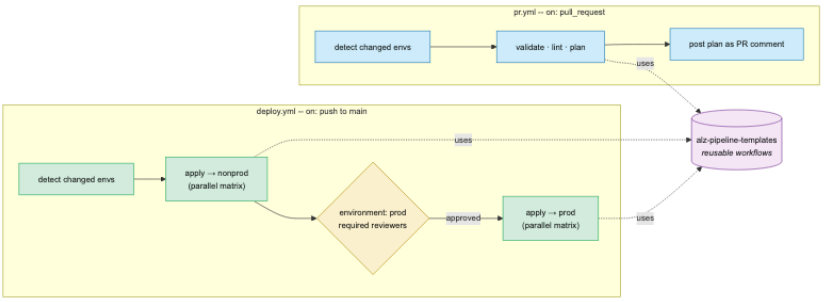
Factor	GitHub Actions	Azure DevOps Pipelines
Already there	Yes if code is on github.com	Yes if code is on dev.azure.com
OIDC to Azure	Mature, simple	Mature ("Workload identity federation")
Reusable workflow / template DX	`workflow_call`, composite actions	YAML templates, extends
Marketplace ecosystem	Larger (Actions Marketplace)	Smaller
Approval gates / environments	GitHub Environments	Environments + approvals
Cost	Generous free + per-minute	Per-user + per-pipeline minute
Secret store integration	OIDC + Key Vault works well	Variable groups + Key Vault native

Recommendation: wherever the *code* lives. Don't mix unless you have a strong reason -- the cognitive overhead of two pipeline syntaxes outweighs any feature delta.

The patterns below are illustrated with **GitHub Actions**; the same shape works in ADO. Whichever you choose, the anatomy is the same -- and it fits in two files.

The standard two-workflow shape

Every IaC repo should have exactly two workflows: one that runs on every PR to *prove* the change is safe, and one that runs after merge to *apply* it. Both delegate the actual work to a shared reusable workflow, so the per-repo files stay short and consistent.



Plus shared **reusable workflows** in a central repo (e.g. `alz-pipeline-templates`).

`pr.yml` (validation)

```yaml
name: PR validation

on:
  pull_request:
    branches: [main]

permissions:
  contents: read
  id-token: write
  pull-requests: write   # to post the plan as a PR comment

concurrency:
  group: pr-${{ github.event.number }}
  cancel-in-progress: true

jobs:
  detect:
    # Detect which envs/workloads changed → matrix output
    runs-on: ubuntu-latest
    outputs:
      changed: ${{ steps.detect.outputs.changed }}
    steps:
      - uses: actions/checkout@<sha>
        with: { fetch-depth: 0 }
      - id: detect
        run: ./scripts/detect-changed-envs.sh >> $GITHUB_OUTPUT

  validate:
    needs: detect
    if: needs.detect.outputs.changed != '[]'
    strategy:
      fail-fast: false
      matrix:
        target: ${{ fromJson(needs.detect.outputs.changed) }}
    uses: contoso/alz-pipeline-templates/.github/workflows/tf-validate.yml@v2
    with:
      working-directory: envs/${{ matrix.target }}
    secrets: inherit
```

`deploy.yml` (apply)

```yaml
name: Deploy

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  id-token: write

concurrency:
  group: deploy-${{ github.ref }}
  cancel-in-progress: false   # never cancel an in-flight apply

jobs:
  detect:
    runs-on: ubuntu-latest
    outputs:
      changed: ${{ steps.detect.outputs.changed }}
    steps:
      - uses: actions/checkout@<sha>
        with: { fetch-depth: 0 }
      - id: detect
        run: ./scripts/detect-changed-envs.sh >> $GITHUB_OUTPUT

  nonprod:
    needs: detect
    strategy:
      max-parallel: 4
      matrix:
        target: ${{ fromJson(needs.detect.outputs.changed) }}
    uses: contoso/alz-pipeline-templates/.github/workflows/tf-apply.yml@v2
    with:
      working-directory: envs/nonprod/${{ matrix.target }}
      environment: nonprod
    secrets: inherit

  prod:
    needs: nonprod
    strategy:
      max-parallel: 4
      matrix:
        target: ${{ fromJson(needs.detect.outputs.changed) }}
    uses: contoso/alz-pipeline-templates/.github/workflows/tf-apply.yml@v2
    with:
      working-directory: envs/prod/${{ matrix.target }}
      environment: prod    # ← reviewers configured here
    secrets: inherit
```

The **GitHub Environment** `prod` has:

- Required reviewers (e.g. 2 from the platform team).
- A **wait timer** (e.g. 10 minutes) so a "ship it" can be aborted.
- **Deployment branches** restricted to `main`.

- The **OIDC client-id** `vars` for the prod SPN.

Reusable workflow (the actual work)

Single source of truth -- every repo calls these.

`alz-pipeline-templates/.github/workflows/tf-apply.yml`:

```yaml
on:
  workflow_call:
    inputs:
      working-directory: { required: true, type: string }
      environment:       { required: true, type: string }

jobs:
  apply:
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}
    defaults: { run: { working-directory: ${{ inputs.working-directory }} } }
    steps:
      - uses: actions/checkout@<sha>
        with: { persist-credentials: false }

      - uses: hashicorp/setup-terraform@<sha>
        with: { terraform_version: 1.9.5 }

      - uses: azure/login@<sha>
        with:
          client-id:       ${{ vars.AZURE_CLIENT_ID }}
          tenant-id:       ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}

      - run: terraform init -input=false
      - run: terraform plan -input=false -out=tfplan -var-file=terraform.tfvars
      - run: terraform apply -input=false -auto-approve tfplan
      - if: failure()
        run: |
          gh issue create --title "Apply failed: ${{ inputs.working-directory }}" \
                          --body "${{ github.server_url }}/${{ github.repository
}}/actions/runs/${{ github.run_id }}"
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

A matching `tf-validate.yml` does `init`, `validate`, `plan`, `tflint`, `tfsec`/`checkov`, and posts the plan to the PR.

PR comments that are actually useful

Plan output dumped raw into a comment is unreadable. Format it:

- **Collapsed `<details>` block** with the full plan inside.
- A **summary table** at the top: `+ N to add · ~ M to change · - K to destroy`.

- **Highlight destructive changes** in bold.
- For Bicep, post the `what-if` output similarly -- use `--result-format` to get JSON, then render to markdown.

Use the `hashicorp/setup-terraform` Action's built-in PR comment mechanism, or roll your own with `actions/github-script`.

A readable plan comment is half the story. The other half is making sure the pipeline only plans the things that actually changed -- which is where the detect script earns its keep.

Detecting changed environments

The detect script is the bit that makes a layered repo scale:

```
#!/usr/bin/env bash
# scripts/detect-changed-envs.sh
set -euo pipefail
base="${GITHUB_BASE_REF:-main}"
git fetch origin "$base" --depth=1
changed_dirs=$(git diff --name-only "origin/$base"...HEAD \
  | grep '^envs/' \
  | awk -F/ '{print $2"/"$3}' \
  | sort -u)
json=$(printf '%s\n' "$changed_dirs" | jq -R . | jq -s -c .)
echo "changed=${json:-[]}"
```

This produces `["nonprod/connectivity","prod/identity"]` and the matrix explodes it into parallel jobs. Touched modules trigger plan runs in **every** consumer environment of that module -- implement that with a module-to-consumer map maintained in the repo.

Scaling to subscription vending -- the git-diff matrix pattern

When your repo vends application landing zone subscriptions (see 01 Repository topology -- subscription vending), the same detect-and-matrix pattern scales to hundreds of subscriptions:

1. Each vended subscription has a config file (`.tfvars` or `.bicepparam`) in a flat or shallow directory structure (e.g. `subscriptions/app01-prod/`).
2. The `detect-changed-envs.sh` script (or equivalent) uses `git diff` to identify which subscription directories changed.
3. The pipeline builds a **matrix** from the changed set and deploys each subscription independently and in parallel -- not a sequential mega-apply.

```yaml
# GitHub Actions example -- subscription vending matrix
jobs:
  detect:
    runs-on: ubuntu-latest
    outputs:
      subscriptions: ${{ steps.detect.outputs.changed }}
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - id: detect
        run: |
          changed=$(git diff --name-only origin/main...HEAD \
            | grep '^subscriptions/' \
            | awk -F/ '{print $2}' | sort -u \
            | jq -R . | jq -s -c .)
          echo "changed=${changed}" >> "$GITHUB_OUTPUT"

  deploy:
    needs: detect
    if: needs.detect.outputs.subscriptions != '[]'
    strategy:
      matrix:
        subscription: ${{ fromJson(needs.detect.outputs.subscriptions) }}
      max-parallel: 10
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying ${{ matrix.subscription }}"
      # ... terraform plan/apply scoped to this subscription
```

This ensures that adding a new subscription is a **single config file** -- the pipeline discovers it automatically.

> 📽️ ***From the ALZ Weekly Questions*** *-- Subscription Vending: Repo Structure, Security & Multi-Tenant The git-diff matrix pattern is the recommended CI/CD approach for subscription vending. It avoids pipeline timeouts and allows 10+ parallel deployments.*

Detection gives you scale; the next challenge is keeping a growing fleet of repos from drifting apart.

Pipeline-as-code, but DRY

Patterns to keep many repos consistent:

- **Reusable workflows** (`workflow_call`) -- call the same job from many repos, versioned by tag (`@v2`).
- **Composite actions** for short, repo-agnostic snippets (e.g. "post plan to PR").
- **Org-wide required workflows** -- GitHub lets you enforce that a workflow *must* run on every PR in selected repos. Use this for policy/security checks you can't allow to be removed.
- **Renovate** / Dependabot to bump the pinned `@vX.Y.Z` references in every consumer when you ship a new template version.

Keeping workflows DRY solves the consistency problem. The next challenge is reliability: Azure is eventually consistent, and some operations simply fail on first attempt.

Long-running operations & retry

Some Azure operations are flaky (Key Vault soft-delete naming, AAD propagation, role assignment lag). Don't paper over with `sleep 30`:

- In Terraform, use `time_sleep` only as a last resort; prefer `null_resource` with `local-exec` that polls the desired state.
- Implement **idempotent retry** at the pipeline step level -- `nick-fields/retry` action with `max_attempts: 3` for known-flaky steps.
- Bake bug-specific waits into modules (with comments linking to the upstream issue) rather than at the pipeline level -- the module is where the knowledge belongs.

Manual operations & the "break glass" pipeline

You will need it. Build one consciously:

- `workflow_dispatch` workflow with **inputs**: target environment, target workload, action (`plan` / `apply` / `import` / `destroy`).
- Restricted to a small group via the `environment: break-glass` (with two reviewers).
- Logs everything; posts a notification to a security channel automatically.

This is far better than engineers running Terraform locally against production state.

With the operational edge cases handled, there are some cheap wins on raw speed that cost almost nothing to add.

Caching and runtime

A few cheap wins:

- Cache `~/.terraform.d/plugin-cache` and the `.terraform` provider directory across runs.
- Use `setup-terraform` with `terraform_wrapper: false` if you parse the plan output yourself -- the wrapper truncates large outputs.
- For Bicep, cache the compiled JSON between `pr` and `deploy` workflows via `actions/cache` keyed on the file SHA.
- Larger runners pay for themselves on `init` heavy plans (more network bandwidth, more CPU for `plan`).

Anti-patterns

- ❌ **Per-repo, hand-rolled workflows.** They diverge within a quarter. Use reusable workflows.
- ❌ `apply` **without an explicit** `plan` **artifact** -- what got applied is not necessarily what was reviewed.
- ❌ `continue-on-error: true` to "make the build green" while investigating. Fix or skip; never silently swallow.
- ❌ **Pipelines that need a human to copy a value between steps.** That's not automation, that's a Slack thread.
- ❌ **Using** `cancel-in-progress: true` **on the deploy concurrency group.** An in-flight `apply` should always finish; cancellation can corrupt state.
- ❌ **One pipeline that deploys all environments serially in one run.** Use environment gates between non-prod and prod, not a long script.

The patterns in this chapter give the pipeline its shape. What that pipeline *validates* -- the static checks, policy assertions, and integration tests -- is the subject of the next chapter. A fast, well-structured pipeline running weak checks still lets bad changes through; what you run inside the workflow matters as much as how the workflow is arranged.

References

- GitHub, *Reusable workflows*: https://docs.github.com/actions/using-workflows/reusing-workflows
- GitHub, *Required workflows*: https://docs.github.com/actions/using-workflows/required-workflows
- GitHub, *Environments*: https://docs.github.com/actions/deployment/targeting-different-environments/using-environments-for-deployment
- Azure DevOps, *YAML templates*: https://learn.microsoft.com/azure/devops/pipelines/process/templates
- Hashicorp, *Run Terraform in CI*: https://developer.hashicorp.com/terraform/tutorials/automation/automate-terraform
- 📽️ ALZ Weekly -- *Subscription Vending: Repo Structure, Security & Multi-Tenant*: https://www.youtube.com/watch?v=11PmT0t6TUI

← 07 State management · Index · 09 Testing & policy →

09 · Testing, validation & policy-as-code

In this chapter:

> ***Decision:*** *what automated checks block a PR, and what's the contract between platform and application teams expressed as policy?*

← 08 CI/CD pipeline patterns · Index · 10 Code quality →

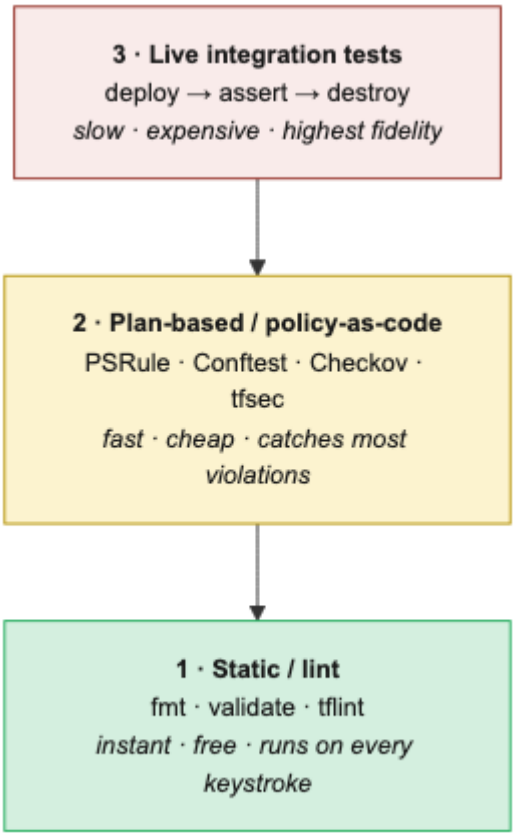
A pipeline that merely runs `terraform apply` is a deployment button, not a quality gate. What separates a mature IaC workflow from a fragile one is everything that happens *before* the apply: the linting that catches silly mistakes, the policy check that enforces enterprise rules, and the integration test that proves the module actually works. This chapter maps out that defence-in-depth pyramid and explains where each layer pays its way.

How we got here

For most of Terraform's first decade, "testing" meant *running `terraform plan` and squinting at the diff*. The brave wrote shell scripts that ran `apply`, hit a few endpoints with `curl`, then `destroy`d everything -- a category formalised in 2018 as **Terratest** (Gruntwork's Go library). It worked, but writing infrastructure tests in Go was a steep ask for ops teams, and Kitchen-Terraform never quite caught on. The **policy-as-code** movement, born from HashiCorp's Sentinel and generalised by **Open Policy Agent** (CNCF, 2018), shifted the centre of gravity: instead of *running* the deploy and checking the result, *parse the plan and reject bad ones* before they touched Azure. Microsoft's **PSRule for Azure** (2020) brought hundreds of WAF-aligned rules out of the box, while Bridgecrew's **Checkov** (2019) and Aqua's **tfsec** (2019) made multi-cloud scanning trivial. Native `terraform test` finally landed in Terraform 1.6 (October 2023), making real integration testing accessible to anyone who could write HCL. The result: the modern IaC pipeline is a **defence-in-depth pyramid** -- static linting, plan- time policy, integration tests on modules, and runtime Azure Policy as the safety net. Here is what that pyramid looks like in practice, and where each layer belongs.

> 📘 ***Key terms***
>
> ***Policy-as-code*** *-- expressing governance rules (naming, allowed SKUs, required tags) in version-controlled code so they are testable, reviewable, and enforceable automatically.*
>
> ***OPA (Open Policy Agent)*** *-- a CNCF-graduated general-purpose policy engine. Policies are written in the* ***Rego*** *language and evaluated against structured data (e.g. a Terraform plan JSON).*
>
> ***SARIF (Static Analysis Results Interchange Format)*** *-- a JSON-based standard for reporting findings from static-analysis tools. GitHub's code scanning UI can ingest SARIF files directly.*
>
> ***Terratest*** *-- a Go library by Gruntwork for writing automated integration tests that deploy real infrastructure, validate it, and tear it down.*
>
> ***CIS (Center for Internet Security)*** *-- an organisation that publishes security benchmarks (e.g. CIS Azure Foundations Benchmark) used by tools like Checkov and PSRule.*
>
> ***Dynamic blocks*** *-- a Terraform construct (`dynamic "block_name" { ... }`) that generates repeated nested blocks from a collection, used to keep module code DRY.*
>
> ***Fixture resources*** *-- pre-created, known-good resources (or resource definitions) used as stable inputs for integration tests, avoiding hard-coded names that cause collisions.*
>
> ***Smoke test*** *-- a lightweight, fast test that checks whether the most critical path works at all (e.g. "did the deployment succeed and is the resource reachable?"), without exhaustive validation.*

The testing pyramid for IaC



Bottom is cheap and fast; top is expensive and slow. Push as many checks as possible to the bottom.

Layer 1 -- Static checks (every commit)

Run on every developer machine via pre-commit and again in CI.

Terraform

Tool	What it does
`terraform fmt -check -recursive`	Formatting. Fail PR on diff.
`terraform validate`	Catches typos, missing variables, type errors.
`tflint`	Provider-aware lints (deprecated args, AzureRM-specific issues). Use the `terraform-linters/tflint-ruleset-azurerm` plugin.
`terraform-docs`	Generates module READMEs. CI verifies docs are up to date.

Bicep

Tool	What it does
`bicep build`	Catches syntax + type errors at compile time.
`bicep format --check`	Formatting.
`bicep lint` (built-in)	Common authoring issues; configured via `bicepconfig.json`.
`PSRule for Azure`	Static analysis for ARM/Bicep against Azure WAF + CIS rules.

Cross-cutting

- **EditorConfig** + Prettier for YAML / Markdown / JSON.
- `actionlint` for GitHub Actions; `zizmor` for Actions security audit.
- `markdownlint` for docs.
- Run them all from a single `pre-commit` config -- one file, one mental model.

Static checks catch what the *author* got wrong. The next layer enforces what the *organisation* requires -- and that is an entirely different problem.

Layer 2 -- Policy-as-code on the plan

This is where you enforce *enterprise rules* at PR time, before any Azure resource is created.

What to enforce

- **Naming convention** (resource names match a regex).
- **Required tags** (`CostCenter`, `Owner`, `DataClassification`).
- **Region restrictions** (no resources in `westeurope` if the workload is data-resident in `swedencentral`).
- **Forbidden SKUs** (no Basic SKU public IPs in production; no Standard_LRS storage for prod databases).
- **Network controls** (no public endpoints on storage / SQL; private endpoints required).
- **Encryption** (CMK on key vaults, soft-delete + purge protection enabled).
- **RBAC** (no `Owner` assignments outside the platform team's identity).

Tools

Tool	Best for
PSRule for Azure	Bicep + Terraform; ships hundreds of WAF rules out of the box. **Recommended default.**
Checkov	Multi-IaC (Terraform, Bicep, ARM, K8s); fast onboarding.
tfsec	Terraform-only; merged into Trivy now.
Conftest (OPA/Rego)	When you need to write *organisation-specific* rules and you're comfortable in Rego.
Sentinel (HCP)	Only if you're on Terraform Cloud/Enterprise.

Pattern: PSRule on Bicep in CI

```yaml
- name: Build Bicep to ARM
  run: bicep build envs/prod/connectivity/main.bicep --outdir build/

- name: Run PSRule
  uses: microsoft/ps-rule@<sha>
  with:
    modules: PSRule.Rules.Azure
    inputPath: build/
    baseline: Azure.Pillar.Security
    outputFormat: Sarif
    outputPath: psrule.sarif

- uses: github/codeql-action/upload-sarif@<sha>
  with:
    sarif_file: psrule.sarif
```

PSRule findings show up in the GitHub **Security** tab and on the PR.

Pattern: Checkov on Terraform plan

```yaml
- run: terraform plan -out tfplan && terraform show -json tfplan > tfplan.json
- uses: bridgecrewio/checkov-action@<sha>
  with:
    file: tfplan.json
    output_format: sarif
    soft_fail: false
```

Checking against the **plan JSON** is more accurate than against `.tf` files because it captures resolved variables, modules, and dynamic blocks.

Custom Rego example (Conftest)

Forbid public storage accounts in any non-sandbox env:

```
package main

deny[msg] {
  resource := input.resource_changes[_]
  resource.type == "azurerm_storage_account"
  resource.change.after.public_network_access_enabled == true
  not is_sandbox(input)
  msg := sprintf("storage account %q must not allow public network access",
[resource.address])
}

is_sandbox(plan) {
  plan.variables.environment.value == "sandbox"
}
```

```
conftest test --policy ./policies tfplan.json
```

Plan-time policy stops non-compliant configuration from ever touching Azure. For well-isolated modules, though, you want to go one step further and prove the thing actually deploys and behaves correctly under real conditions.

Layer 3 -- Live integration tests

Deploy a real instance, assert behaviour, destroy. Reserved for **modules**, not workloads.

Terraform

- `terraform test` (built-in since 1.6) -- assertion blocks, can run ephemeral applies. Default choice in 2026.
- **Terratest** (Go) -- older, more flexible, but heavier maintenance.

```
# tests/main.tftest.hcl
run "creates_hub_vnet" {
  variables {
    address_space = "10.0.0.0/16"
    location      = "swedencentral"
  }

  assert {
    condition     = output.vnet_id != ""
    error_message = "Hub VNet was not created"
  }
}
```

Bicep

- **PSRule unit tests** for static rules.

- `az deployment what-if` against a sacrificial resource group as a smoke test.
- For full integration: deploy via Deployment Stack to a per-PR resource group, run `az` queries to assert state, then `az stack sub delete --action-on-unmanage deleteAll`.

Where they run

- **Module repos:** every PR runs the full integration test suite in a dedicated test subscription.
- **Workload repos:** integration tests are usually unnecessary if your modules are well-tested. Plan/policy checks suffice.

> ⚖️ ***The debate -- can you skip integration tests for workloads?***
>
> *The advice above ("plan/policy checks suffice for workloads") saves time and CI cost, but it's a **risk acceptance decision**, not a universal best practice.*
>
> ***The counter-argument:** Modules are unit-tested in isolation; they validate that the module works, not that your specific composition of modules works. Unique parameter combinations, conditional feature flags, cross-module data flows, and custom policies can all produce failures that only integration tests catch. Teams that have deployed workload changes relying solely on plan-time checks have hit issues -- resource dependency ordering, eventual-consistency race conditions, private-endpoint DNS propagation -- that only manifest at apply time.*
>
> ***When skipping is reasonable:** If your tier-2 modules are heavily tested, your workload compositions are simple (thin wrappers calling modules with `.tfvars`), and you have strong runtime policy as a safety net, the residual risk may be low enough to accept.*
>
> ***When it isn't:** If workloads compose modules with complex conditionals, `for_each` over dynamic data, or cross-stack references, integration tests on the composition are the only way to catch interaction bugs before production.*

At this point you have checks at every stage of development. There is still a gap: what prevents someone creating a non-compliant resource directly through the portal? That is where Azure Policy comes in -- and it needs to stay in sync with everything above.

Policy-as-code vs Azure Policy

There are **two** layers of policy:

1. **Repo-side (PR-time) policy** -- Checkov/PSRule/Conftest. Catches issues *before* deployment. Fast feedback, free, scoped to what you can see in code.
2. **Platform-side (runtime) Azure Policy** -- applied at the management group / subscription, enforces continuously, including for resources created outside IaC.

You need **both**:

- PR-time policy gives instant developer feedback and prevents the bad PR from merging.

- Azure Policy is the *insurance* -- it catches anything the PR check missed, and resources created via portal/CLI/script.

The two should be **expressed from the same intent**. A common pattern:

- Source of truth: a YAML file describing each control.
- Generator script produces:
  - A PSRule / Checkov rule for PR time.
  - An Azure Policy `policyDefinition` and assignment for runtime.

That way, drift between "what we lint" and "what we enforce" is impossible by construction.

> ⚖️ ***The debate -- does PR-time policy violate "Azure as the single control plane"?***
>
> *One of the CAF design principles for Azure Landing Zones is to use **Azure as the single platform for operations management and policy**. Adding PSRule, Checkov, or Conftest as a second enforcement layer at PR time creates a tension with that principle: you now have policy logic in two places -- Azure Policy in the cloud and linting rules in your CI pipeline -- with no guarantee they stay in sync.*
>
> ***The case for Azure Policy only:***
>
> - *One control plane means one place to audit, one place to update, and one compliance dashboard. No rule drift, no "it passed CI but Azure Policy denied it" confusion.*
> - *Azure Policy's `what-if` and `DoNotEnforce` assignment modes can surface violations before deployment, partially closing the feedback gap.*
> - *Fewer tools to maintain, licence, and train on.*
>
> ***The case for dual-layer:***
>
> - *Azure Policy acts after deployment (or at deployment time at the earliest). PR-time checks catch issues minutes into a developer's workflow, not after a 15-minute `terraform plan` + `apply` cycle.*
> - *Not all controls map cleanly to Azure Policy. Code-level concerns -- "this module is missing a `description`", "this variable has no validation block" -- are invisible to Azure Policy because they don't exist as deployed resources.*
> - *Azure Policy cannot block a PR from merging. If your governance model requires that non-compliant code never reaches the main branch, you need a CI-side gate.*
>
> ***Where most teams land:** the dual-layer approach wins in practice, but the "single source of truth" pattern described above is non-negotiable -- without it, rule drift between the two layers erodes trust in both. Teams that skip the generation step and hand-maintain parallel rule sets in Azure Policy and PSRule/Checkov inevitably diverge, and the resulting "it passed CI but was denied at deploy" incidents undermine the entire governance model.*

Azure Policy lifecycle in this repo

- Policy *definitions* in code (Bicep/Terraform).
- Policy *assignments* in the foundation/policy repo.

- Initiative/Set definitions for grouped controls.
- On PR: `az policy assignment create --enforcement-mode DoNotEnforce` against a test management group, then run a compliance scan, then destroy.
- On merge: deploy with `Default` enforcement.

For deny vs audit:

- **Audit**: rolling out a new control. Run for ≥ 2 weeks, generate the exemption list from existing non-compliant resources, then flip to deny.
- **Deny**: steady state for any control where remediation is cheap and the violation has a real impact.

Built-in policy versioning

Azure built-in policies now carry a **`version` field** in their metadata (SemVer: `MAJOR.MINOR.PATCH`). Microsoft updates built-in definitions in place -- your existing assignments automatically pick up the latest version unless you pin to a specific one.

> 🐿️ ***From the ALZ Weekly Questions*** *-- How to Stay Current with ALZ Azure Policies*
> ***Patch versions auto-apply*** *-- you cannot pin to a patch. Minor and major version updates require you to update the reference in your library metadata. This means your compliance posture can shift at the patch level without a PR in your repo.*

This creates a **silent drift risk**: a built-in policy you assigned two years ago may have changed its logic, added parameters, or expanded its scope. Your compliance posture shifts without a PR, a review, or a test in *your* pipeline.

Mitigations:

- **Snapshot built-in definitions in your repo.** Export the JSON of every built-in you assign (via `az policy definition show` or the Azure/azure-policy repo) and store it alongside your assignments. A scheduled CI job diffs the live definition against your snapshot and opens a PR when the version changes -- giving your team a chance to review before accepting.
- **Subscribe to the Azure Policy changelog.** Microsoft publishes built-in policy change logs; feed them into your governance channel.
- **Test assignments, not just definitions.** When a built-in updates, re-run your compliance scan against your fixture resources to verify the new version doesn't flag resources that were previously compliant (false positives) or miss resources that should be flagged (false negatives).

Updating ALZ policies -- the practical workflow

Whether you use Bicep or Terraform, the ALZ library is the **source of truth** for which policies are assigned and at what version. Updating policies is a routine Day-2 operation with a well-defined workflow:

Terraform workflow:

1. Update `metadata.json` in your repo to reference a newer ALZ library version.

2. Run `terraform plan` -- the ALZ provider dynamically pulls the library and shows what policy definitions, assignments, or role definitions changed.
3. Review the plan output carefully -- don't blindly approve.
4. Merge the PR → pipeline runs `terraform apply`.

Bicep workflow:

1. Delete the local `lib/` directory that contains the generated ALZ library files.
2. Run the `alzlib` tool to regenerate from the latest library version: `alzlib generate`.
3. Run the `Update-ALZReferences.ps1` script to align your parameter files.
4. Open a PR -- the pipeline runs `what-if` to show changes.
5. Merge → pipeline deploys via Deployment Stacks (which automatically clean up deprecated policy assignments).

> 🐿️ ***From the ALZ Weekly Questions*** *-- How to Stay Current with ALZ Azure Policies*
> *Deployment Stacks give Bicep a major advantage here: when a policy is removed from the ALZ library, the stack automatically deletes the orphaned assignment. Terraform achieves the same via state tracking, but Bicep teams historically had to clean up manually.*

EPAC as an alternative policy management tool

EPAC (Enterprise Policy as Code) is a community-driven tool that provides a structured way to manage Azure Policy assignments at scale, especially across **multi-tenant** environments. It uses a declarative JSON/CSV format and supports complex policy ecosystems with hundreds of assignments.

When EPAC makes sense:

- Large multi-tenant estates where the ALZ-native policy management feels limiting.
- Organisations that want a single policy repository covering multiple ALZ instances.
- Teams that need advanced features like policy exemption management, effect overrides, and compliance reporting.

The risks:

- EPAC is **community-driven** -- it has no Microsoft product lifecycle, no SLA, and no guaranteed long-term support.
- If the maintainers step away, you own the codebase. Only adopt EPAC if your team has the skills and willingness to **fork and maintain** it independently.
- EPAC has its own sequencing requirements that can conflict with how ALZ deploys policies. Integration requires careful planning.

If you don't use EPAC, you can disable ALZ-native policy assignments and manage policies entirely through your own definitions:

- **Bicep:** set the policy module references to `null` or `no` in the configuration.
- **Terraform:** use an **empty archetype** that deploys management groups without any policy assignments.

> 📽️ ***From the ALZ Weekly Questions*** *-- Using EPAC for Azure Policy in ALZ EPAC is a power tool -- powerful in expert hands, dangerous if adopted without deep understanding. The ALZ team's position: if you don't understand EPAC well enough to fork it, you probably shouldn't depend on it.*

The custom → built-in lifecycle

A common pattern in maturing ALZ estates:

1. **You write a custom policy** because no built-in exists for a specific control (e.g. "deny storage accounts without infrastructure encryption").
2. **Months later, Microsoft ships a built-in** that covers the same control -- often with better alias coverage, edge-case handling, and ongoing maintenance.
3. **You now maintain a custom policy that duplicates a built-in** -- accruing maintenance cost and risking divergence.

The responsible lifecycle:

- **Inventory custom policies quarterly.** For each, check whether a built-in equivalent now exists. The Azure/azure-policy repo and `az policy definition list --filter "policyType eq 'BuiltIn'"` are your search tools.
- **Don't auto-swap.** A built-in may have different parameter names, a broader or narrower scope, or different default values. Test the built-in against your estate in audit mode before replacing.
- **Migration steps:**
  1. Assign the built-in in **audit mode** alongside the custom policy.
  2. Compare compliance results -- they should match. Investigate discrepancies.
  3. Once confident, remove the custom assignment and flip the built-in to **deny** (or whichever effect applies).
  4. Retire (but keep archived) the custom definition.
- **Tag custom policies with** `lifecycle: custom-pending-review` so your quarterly inventory script can flag them automatically.

> ⚖️ ***The debate -- should you always prefer built-ins?***
>
> *Built-ins are maintained by Microsoft and get free updates, but they are also* ***opaque****: you can't modify their logic, and a version bump can change behaviour without your consent. Custom policies give you full control at the cost of full maintenance. Some teams prefer to wrap built-ins in initiatives with explicit version pins and treat the built-in as an upstream dependency (similar to how you'd pin a module version). Others mandate custom-only for critical controls so that every change flows through their PR process. Neither approach is universally superior -- the right balance depends on how much governance automation your team can sustain.*

Test coverage targets

Pragmatic, not dogmatic:

Artifact	Minimum coverage
Tier-2 pattern modules	Static + plan-policy + ≥ 1 integration test per major code path
Tier-3 workload composition	Static + plan-policy. No need for integration tests if modules are tested.
Custom Azure Policy	Compliance test against a fixture resource that should pass + a fixture that should fail
Pipeline templates	Unit test the templates with `act` or by running them against a sample repo

These targets are intentionally conservative. A 45-minute test suite that engineers skip is worse than no tests at all -- so cut scope ruthlessly, parallelise what remains, and treat build time as a metric worth watching.

Anti-patterns

- ❌ **All policy lives in Azure Policy.** Developers find out at deploy time, after they've waited for a 15-min `terraform plan` to come back. Move what you can to PR time.
- ❌ **All policy lives at PR time.** Anyone clicking in the portal bypasses your controls.
- ❌ `soft_fail: true` **on Checkov to "fix later".** Later never comes.
- ❌ **Integration tests against a shared test subscription with hard-coded names.** Two PRs run simultaneously → name collision → both fail.
- ❌ **Tests that take 45 minutes.** Engineers will avoid them. Parallelise or trim coverage.

The discipline of layered validation ultimately comes down to shortening the feedback loop: moving pain from the 4 AM incident to the 30-second pre-commit hook. Once that loop is tight, the question shifts to the *experience* of working inside the repo day-to-day -- which is the subject of the next chapter.

References

- PSRule for Azure: https://azure.github.io/PSRule.Rules.Azure/
- Checkov: https://www.checkov.io/
- Conftest: https://www.conftest.dev/
- Hashicorp, `terraform test`: https://developer.hashicorp.com/terraform/language/tests
- Terratest: https://terratest.gruntwork.io/
- Microsoft, *Azure Policy as code*: https://learn.microsoft.com/azure/governance/policy/concepts/policy-as-code

- OPA / Rego: https://www.openpolicyagent.org/docs/latest/policy-language/
- 📹 ALZ Weekly -- *How to Stay Current with ALZ Azure Policies*: https://www.youtube.com/watch?v=ddcVKS_MKkk
- 📹 ALZ Weekly -- *Using EPAC for Azure Policy in ALZ*: https://www.youtube.com/watch?v=x1I_XhC6GtA

← 08 CI/CD pipeline patterns · Index · 10 Code quality →

10 · Code quality, linting & developer experience

In this chapter:

> ***Decision:*** *what does it look like to clone the repo and submit a high-quality PR on day one? DX is a force multiplier -- invest in it.*

← 09 Testing & policy · Index · 11 Manageability →

Good infrastructure code doesn't just work -- it's readable, self-documenting, and easy to contribute to. The best IaC teams treat developer experience as a first-class engineering concern, not an afterthought bolted on when onboarding gets painful. This chapter covers the tools and conventions that collapse the time from `git clone` to productive contribution, and the automation that keeps quality high without relying on humans to remember.

How we got here

The original "developer experience" of an IaC repo was a one-line README: *"install Terraform and run `terraform apply`."* Three months later, half the team was on Terraform 0.11 and half on 0.12, and a fresh clone took an afternoon to bootstrap. Tooling caught up in waves: `tflint` (2018) added Azure-aware lints, `terraform-docs` (2019) generated module READMEs from the schema, **pre-commit-terraform** (2019) bundled it all into a single `pre-commit install`. The biggest leap was the **devcontainer** specification (Microsoft, open-sourced 2022) and **GitHub Codespaces** -- for the first time you could pin every tool, every extension, and every shell helper in one JSON file and have CI use the *identical* image. Tools like **`mise`** and **`asdf`** brought the same discipline to engineers who don't want a container around their editor. A mature ALZ repo in 2026 should let a new joiner go from `git clone` to a green `plan` in under five minutes -- and that's not aspirational, it's table stakes. The question is what it actually takes to get there.

> 📘 ***Key terms***
>
> ***Devcontainer*** *-- a specification (`.devcontainer/devcontainer.json`) that defines a reproducible development environment inside a container, including all tools, extensions, and settings. GitHub Codespaces and VS Code use it natively.*
>
> ***`mise`*** */* ***`asdf`*** *-- version-manager tools that pin CLI tool versions (Terraform, az, tflint) per project via a dotfile (`.mise.toml` / `.tool-versions`), ensuring every developer uses the same versions without containers.*
>
> ***Taskfile*** *-- a modern alternative to `Makefile` (`Taskfile.yml`) for defining project-level commands (e.g. `task plan`, `task lint`), with YAML syntax and dependency support.*
>
> ***Pre-commit hooks*** *-- Git hooks that run checks (formatting, linting, secret scanning) automatically before each `git commit`, catching issues at the earliest possible stage.*
>
> ***Conventional Commits*** *-- a commit-message convention (e.g. `feat:`, `fix:`, `chore:`) that enables automated changelog generation and semantic version bumping.*
>
> ***EditorConfig*** *-- a simple dotfile (`.editorconfig`) that standardises indent style, charset, and line endings across editors and IDEs.*
>
> ***Codespaces*** *-- GitHub's cloud-hosted development environment that launches a devcontainer in the browser, eliminating local setup entirely.*

The "five-minute onboarding" target

A new engineer should be able to:

```
gh repo clone contoso/alz-platform
cd alz-platform
make bootstrap     # installs everything
make plan ENV=nonprod WORKLOAD=connectivity
```

...and see a meaningful plan within **five minutes** of the first `git clone`. Anything more is friction tax you pay forever.

> ⚖️ ***The debate -- is five minutes realistic?***
>
> *Five minutes is aspirational and achievable for small-to-mid repos, but practitioners managing large estates (100+ modules, heavy provider sets) report that `terraform init` alone can take 2-3 minutes, and devcontainer image pulls add another 2-5 on first use. Hitting the five-minute target at scale requires pre-built devcontainer images (ongoing maintenance cost), local provider caching, and carefully curated module sets -- effort that not every team can justify.*
>
> *A more realistic goal for large estates: **under fifteen minutes** to first meaningful plan. Still fast, less likely to set an expectation that fails on day one. The five-minute target remains the aspiration; document what your actual number is and actively work to reduce it.*

The two ingredients: **devcontainers** and **a `Makefile` (or `Taskfile`)**.

Devcontainer / mise / asdf

Pin tool versions in the repo. Three viable approaches:

Devcontainer (recommended for VS Code / Codespaces shops)

```
// .devcontainer/devcontainer.json
{
  "name": "alz-platform",
  "image": "mcr.microsoft.com/devcontainers/base:ubuntu-24.04",
  "features": {
    "ghcr.io/devcontainers/features/azure-cli:1": {},
    "ghcr.io/devcontainers/features/terraform:1": { "version": "1.9.5" },
    "ghcr.io/devcontainers/features/github-cli:1": {},
    "ghcr.io/devcontainers/features/powershell:1": {}
  },
  "postCreateCommand": "make bootstrap",
  "customizations": {
    "vscode": {
      "extensions": [
        "hashicorp.terraform",
        "ms-azuretools.vscode-bicep",
        "github.vscode-github-actions",
        "redhat.vscode-yaml"
      ]
    }
  }
}
```

Same container runs locally in VS Code, in GitHub Codespaces, and (with minor tweaks) in CI -- no "works on my machine" left.

`mise` / `asdf` (lighter)

```
# .mise.toml
[tools]
terraform = "1.9.5"
bicep     = "0.30.3"
azure-cli = "2.65.0"
node      = "20.18.0"
```

Engineers run `mise install` once and have the same versions as CI.

Pinning the tools solves the "which version?" problem. The next step is making those tools easy to invoke without memorising long incantations.

`Makefile` as a UX layer

Hide the long incantations behind short, memorable commands.

```
ENV ?= nonprod
WORKLOAD ?= connectivity
TF_DIR := envs/$(ENV)/$(WORKLOAD)

.PHONY: bootstrap fmt lint plan apply test clean

bootstrap:
    pre-commit install
    cd $(TF_DIR) && terraform init -input=false

fmt:
    terraform fmt -recursive
    bicep format

lint:
    pre-commit run --all-files
    tflint --recursive

plan:
    cd $(TF_DIR) && terraform plan -var-file=terraform.tfvars -out=tfplan

apply:
    @if [ "$(ENV)" = "prod" ]; then \
      echo "Use the pipeline for prod. Refusing to apply locally."; exit 1; \
    fi
    cd $(TF_DIR) && terraform apply tfplan

test:
    cd modules && terraform test

clean:
    find . -type d -name '.terraform' -prune -exec rm -rf {} +
```

Note `make apply` refuses to run against `prod` -- the only way to apply prod is the pipeline.

Pre-commit hooks

`pre-commit` is the highest ROI single tool to add to an IaC repo.

```yaml
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-json
      - id: check-added-large-files

  - repo: https://github.com/antonbabenko/pre-commit-terraform
    rev: v1.92.0
    hooks:
      - id: terraform_fmt
      - id: terraform_validate
      - id: terraform_tflint
      - id: terraform_docs
        args: [--args=--config=.terraform-docs.yml]

  - repo: https://github.com/bridgecrewio/checkov
    rev: 3.2.255
    hooks:
      - id: checkov
        args: [-d, ., --quiet, --soft-fail]   # informational locally; fatal in CI

  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.4
    hooks:
      - id: gitleaks

  - repo: https://github.com/rhysd/actionlint
    rev: v1.7.1
    hooks:
      - id: actionlint
```

Run the **same hooks in CI** -- `pre-commit run --all-files` -- so what's caught locally and what's caught in CI is identical.

Hooks handle the checks. Documentation is a different discipline -- and the kind that degrades fastest when it has to be maintained by hand.

Auto-generated module documentation

Every module's `README.md` is generated from its variables and outputs. Engineers never write them by hand; CI verifies they're current.

`.terraform-docs.yml`:

```
formatter: markdown table
output:
  file: README.md
  mode: inject
  template: |-
    <!-- BEGIN_TF_DOCS -->
    {{ .Content }}
    <!-- END_TF_DOCS -->
sort:
  enabled: true
  by: required
```

CI step:

```
- run: terraform-docs --output-check markdown table modules/network/hub
```

For Bicep, use `PSDocs.Azure` or parse the `metadata` blocks with a custom script.

Conventional commits + automated releases

Force commit messages into a parseable shape:

```
feat(network/hub): add Azure Firewall premium SKU support
fix(identity): correct role assignment scope for spoke VNets
chore(ci): bump terraform to 1.9.5

BREAKING CHANGE: hub module renamed param `firewall_sku_name` → `firewall_sku`
```

Tools:

- `commitlint` + Husky to validate locally.
- `release-please` (recommended) or `semantic-release` to automate:
  - Tag + GitHub Release.
  - `CHANGELOG.md` update.
  - Migration notes in release body.

The benefit compounds: every consumer's Renovate bot can post a "v1.5.0 → v1.6.0" PR with the changelog inline.

Structured commits make the changelog practically free. The remaining pieces of the DX puzzle are smaller but worth locking down: editor settings that prevent trivial formatting noise, and templates that shape what contributions look like before they hit the pipeline.

Editor configuration

```
# .editorconfig
root = true

[*]
charset = utf-8
end_of_line = lf
insert_final_newline = true
trim_trailing_whitespace = true
indent_style = space
indent_size = 2

[*.{tf,tfvars}]
indent_size = 2

[*.bicep]
indent_size = 2

[*.md]
trim_trailing_whitespace = false  # MD line breaks
```

Combine with Prettier for YAML/MD/JSON; Bicep and Terraform have their own formatters.

Repository templates

For organisations spinning up many landing-zone repos, a **GitHub template repository** + `gh repo create --template` gives every new repo:

- The standard `Makefile`, `.pre-commit-config.yaml`, devcontainer.
- A starter `envs/` skeleton.
- Standard `.github/workflows/*.yml` calling reusable workflows.
- A pre-filled `CODEOWNERS` you customise.

For more sophisticated scaffolding (with prompts), use **Backstage templates** or `copier`.

Templates provision the repo. The next layer shapes how contributions land in it.

Issue & PR templates

Make the right thing the easy thing.

`.github/PULL_REQUEST_TEMPLATE.md`:

```
## What

## Why

## How (high-level)

## Affected environments
- [ ] nonprod
- [ ] prod

## Checklist
- [ ] `terraform plan` reviewed in CI
- [ ] PSRule findings reviewed
- [ ] Module bumps (if any) include CHANGELOG link
- [ ] If breaking change, ADR added under `docs/adr/`
```

`.github/ISSUE_TEMPLATE/`:

- `bug.yml` (structured form: env, repro, expected vs actual)
- `feature.yml`
- `change-request.yml` (for production changes that need a paper trail)

Local development against Azure

Engineers should be able to safely experiment:

- Each engineer has a personal **sandbox subscription** (or RG) with their name as the owner.
- `make plan ENV=sandbox WORKLOAD=hub` runs against their own sub via `az login`.
- **Aggressive lifecycle:** all sandbox resource groups have a `DeleteAt` tag and a scheduled function/runbook tears down anything past expiry.
- No shared "team-dev" subscription -- they always become the prod everyone forgot was prod.

Anti-patterns

- ❌ **README that says "install Terraform 1.x"** without pinning a version. Tomorrow's release breaks today's repo.
- ❌ **Linters that produce warnings nobody reads.** Either fail the build or remove the linter.
- ❌ **Module READMEs written by hand.** They drift from the variables in one PR.
- ❌ **`make apply` that works against prod.** Pipeline-only or you lose audit trail.
- ❌ **Devcontainer that nobody uses.** Make it the *only* supported path -- including for CI -- and it stays maintained.

Developer experience is the invisible multiplier: when a team can move fast and confidently, the quality of every other practice in this book improves. The Makefile runs the tests, the hooks run the

linters, the devcontainer keeps the tools consistent, and the PR template makes the right information the default -- all of it compounding quietly. The next chapter turns from how you build and ship the platform to how you keep it observable and manageable once it is running in production.

References

- `pre-commit`: https://pre-commit.com/
- `pre-commit-terraform`: https://github.com/antonbabenko/pre-commit-terraform
- `terraform-docs`: https://terraform-docs.io/
- Devcontainers: https://containers.dev/
- `mise`: https://mise.jdx.dev/
- `release-please`: https://github.com/googleapis/release-please
- `commitlint`: https://commitlint.js.org/
- Backstage software templates: https://backstage.io/docs/features/software-templates/

← 09 Testing & policy · Index · 11 Manageability →

11 · Manageability & day-2 operations

In this chapter:

> ***Decision:*** *how do you keep an ALZ healthy after the first deploy -- ownership, drift, blast radius, lifecycle, and runbooks?*

← 10 Code quality · Index · 12 Naming & tagging →

The real test of any ALZ implementation is not whether the first `apply` succeeds -- it's whether the system stays coherent under the steady pressure of operational reality: teams bypassing the pipeline "just this once", subscriptions that were meant to be temporary becoming permanent, and identities that quietly accumulate permissions like barnacles. This chapter tackles the day-2 problem head-on, covering ownership governance, drift detection, blast-radius control, lifecycle management, and the humble runbook that saves you at 2 a.m.

How we got here

The first generation of "infrastructure as code" was really *infrastructure as code, plus quite a lot of clicking when nobody was looking*. Drift wasn't a concept; it was a fact of life that a `vnet` created by Terraform on Monday would have an extra subnet by Friday because someone needed to *just quickly fix something*. Tools like **driftctl** (2021) and **Terraformer** brought visibility, and scheduled `terraform plan -detailed-exitcode` runs became a standard Day-2 practice. Microsoft's **Deployment Stacks** with `denySettings` (GA 2024) finally made out-of-band changes *blockable at the ARM layer*, not just detectable after the fact. **CODEOWNERS** (GitHub 2017) gave path-based governance that scales beyond "the platform team reviews everything", and **Privileged Identity Management** (Entra) made standing high-privilege access the exception rather than the rule. Day-2 ops in 2026 is no longer a heroic firefighting practice -- done right, it's

mostly Renovate PRs, drift dashboards, and the occasional runbook. This chapter shows the building blocks.

> 📘 ***Key terms***
>
> ***Drift detection*** *-- the practice of periodically running* `terraform plan` *(or* `az stack show`*) and alerting when the actual cloud state diverges from what the code declares.*
>
> ***driftctl*** *-- an open-source tool (now archived) that scanned AWS/Azure resources and compared them against Terraform state to find unmanaged or drifted resources.*
>
> ***KQL (Kusto Query Language)*** *-- the query language used by Azure Resource Graph, Log Analytics, and Microsoft Sentinel for querying structured data.*
>
> ***DORA metrics*** *-- four key metrics from the DevOps Research and Assessment programme: deployment frequency, lead time for changes, change failure rate, and time to restore service. Used to benchmark engineering performance.*
>
> ***Subscription vending*** *-- the automated process of creating and configuring a new Azure subscription (with budget, policies, RBAC, and networking) so it's ready for a workload team to use.*
>
> ***EA / MCA*** *-- Enterprise Agreement / Microsoft Customer Agreement -- the two main Azure billing models through which subscriptions are created programmatically.*
>
> ***Zombie subscriptions*** *-- subscriptions that are no longer actively used or maintained but remain billable and potentially insecure because no decommission process was followed.*
>
> ***Bus factor*** *-- the number of team members who would need to be unavailable before the project stalls. A bus factor of 1 means a single person's absence can halt operations.*

CODEOWNERS -- your first line of governance

`CODEOWNERS` enforces *who must approve* a PR for any given path. In a layered repo it's the difference between governance and chaos.

```
# .github/CODEOWNERS
# Everything by default
*                                   @org/platform-engineering

# Foundation needs a second pair of eyes from cloud governance
/envs/prod/foundation/              @org/platform-engineering @org/cloud-governance
/policies/                          @org/cloud-governance @org/security

# Per-landing-zone owners
/envs/prod/lz-corp-app01/           @org/team-app01 @org/platform-reviewers
/envs/prod/lz-corp-app02/           @org/team-app02 @org/platform-reviewers

# Pipeline templates: only platform team
/.github/workflows/                 @org/platform-engineering
```

Combine with **branch protection** that *requires* CODEOWNERS review. Without that requirement, CODEOWNERS is informational only.

> 💡 ***CODEOWNERS complexity scales with repo size.*** *The example above assumes a single repo containing foundation, platform, and landing zones -- the monorepo model. In practice, the recommended layered few-repo topology splits these into separate repos, which means each repo's CODEOWNERS file stays short and obvious (a handful of rules instead of hundreds). If you do run a monorepo or a large shared landing-zone repo, expect the CODEOWNERS file to grow with every onboarded team -- review it quarterly, and consider generating it from a source-of-truth YAML to avoid stale entries.*

Review the file quarterly -- leavers, reorgs, retired apps.

Ownership governance tells you *who* must approve a deliberate change -- but it says nothing about changes that never went through the PR process at all. Detecting those requires something more active.

Drift detection

Drift is changes made to Azure resources outside your IaC. It happens -- a firefighter clicks in the portal, an automation script bypasses the pipeline. Make it visible.

A commonly overlooked drift source: Modify and DINE policies

Not all drift comes from humans. **Modify** and **DeployIfNotExists (DINE)** Azure policies deliberately change or create resources outside your IaC pipeline -- and they're *supposed* to. A DINE policy that adds diagnostic settings to every new storage account is doing its job, but `terraform plan` will report those settings as drift because the state file never knew about them.

This creates a tension:

- **Suppress the diff** (e.g. `ignore_changes` in Terraform, or omit the property from Bicep) and accept that the policy is the source of truth for that attribute. Clean drift reports, but the IaC no longer describes the full resource.
- **Codify the policy's effect** in your IaC so the plan matches reality. Complete resource description, but duplicates intent already expressed by the policy -- and changes to the policy require matching IaC updates.
- **Accept the noise** and train the team to recognise known policy-induced diffs during triage. Honest, but erodes trust in drift reports over time.

There is no universally agreed best practice here. Many teams use a hybrid: `ignore_changes` for attributes managed *exclusively* by policy (e.g. diagnostic settings added by DINE), and codify attributes where the IaC and policy must agree (e.g. TLS version enforced by Modify). Document which approach you chose and why -- undocumented `ignore_changes` blocks are a maintenance hazard.

Pattern: scheduled `plan` / `what-if`

```yaml
# .github/workflows/drift.yml
on:
  schedule:
    - cron: '0 6 * * 1'  # Mondays 06:00 UTC
  workflow_dispatch:

jobs:
  drift:
    strategy:
      matrix:
        env: [nonprod, prod]
        workload: [connectivity, identity, management]
    uses: contoso/alz-pipeline-templates/.github/workflows/tf-drift.yml@v2
    with:
      working-directory: envs/${{ matrix.env }}/${{ matrix.workload }}
```

`tf-drift.yml` runs `terraform plan -detailed-exitcode`:

- Exit 0 → no changes ✅
- Exit 2 → drift detected → **post to a Teams/Slack channel + open an issue** with the diff.
- Exit 1 → error → page the on-call.

For Bicep with Deployment Stacks, use `az stack sub show` to compare the declared state with the actual `Microsoft.Resources/deployments`. The `denySettings: denyWriteAndDelete` mode prevents most drift at the source.

When drift is found

Triage:

1. **Reproduce** by running `plan` interactively.
2. Was the change legitimate?
   - **Yes** → codify it: open a PR that adopts the change and merge.
   - **No** → revert by running `apply` to restore desired state.
3. **Always investigate why** -- drift indicates a process gap. Maybe a policy is missing, maybe an engineer doesn't know the rules.

Detecting drift is valuable; limiting how much damage a single bad change can cause is equally important. The structural approaches to keeping your blast radius small are the subject of the next section.

Blast radius management

The size of "what one bad apply can break." Strategies:

1. State-file granularity

Already covered in 07 state management. One state per (environment × workload).

2. Bicep Deployment Stacks `denySettings`

```
az stack sub create \
  --deny-settings-mode denyWriteAndDelete \
  --deny-settings-excluded-actions "Microsoft.KeyVault/vaults/secrets/write" \
  --deny-settings-excluded-principals "<sp-app-runtime-id>"
```

A subsequent rogue `apply` (or click in the portal) is blocked at the ARM layer. Excluded principals are the rare identities allowed to mutate inside the stack.

3. Resource locks

Azure Resource Locks (`CanNotDelete` / `ReadOnly`) are a complementary last line of defence. Apply them via IaC (so they're code-managed) on critical resources: hub VNet, ExpressRoute circuits, central Key Vaults, log analytics workspaces.

```
resource "azurerm_management_lock" "hub_vnet" {
  name       = "lock-cannotdelete"
  scope      = azurerm_virtual_network.hub.id
  lock_level = "CanNotDelete"
  notes      = "Critical hub network -- break-glass via PIM"
}
```

Tradeoff: locks make legitimate IaC operations harder too -- `terraform apply` that needs to *replace* a locked resource fails. Reserve locks for genuinely critical, slow-moving resources.

4. Soft delete + purge protection

For Key Vault, Storage, and similar -- enable soft delete and purge protection unconditionally in your modules. Recovery beats prevention when the worst happens.

Architectural controls limit the structural blast radius. But those controls are only as strong as the identities that hold the keys to the pipeline -- which makes RBAC for the IaC system itself equally worth examining.

RBAC for the IaC system itself

Day-2 RBAC concerns are different from day-1:

Identity	Permissions
Pipeline SPN (per env)	Contributor at the env scope; explicit User Access Admin if it manages role assignments
Engineers (read, all envs)	Reader on all subscriptions -- debugging needs visibility
Engineers (write, sandbox only)	Contributor on personal sandbox subs
Engineers (write, prod break-glass)	PIM-elevated, alerting on every elevation
Platform team leads	Owner via PIM only, hours-bounded

Use **Azure Resource Graph** queries on a schedule to audit:

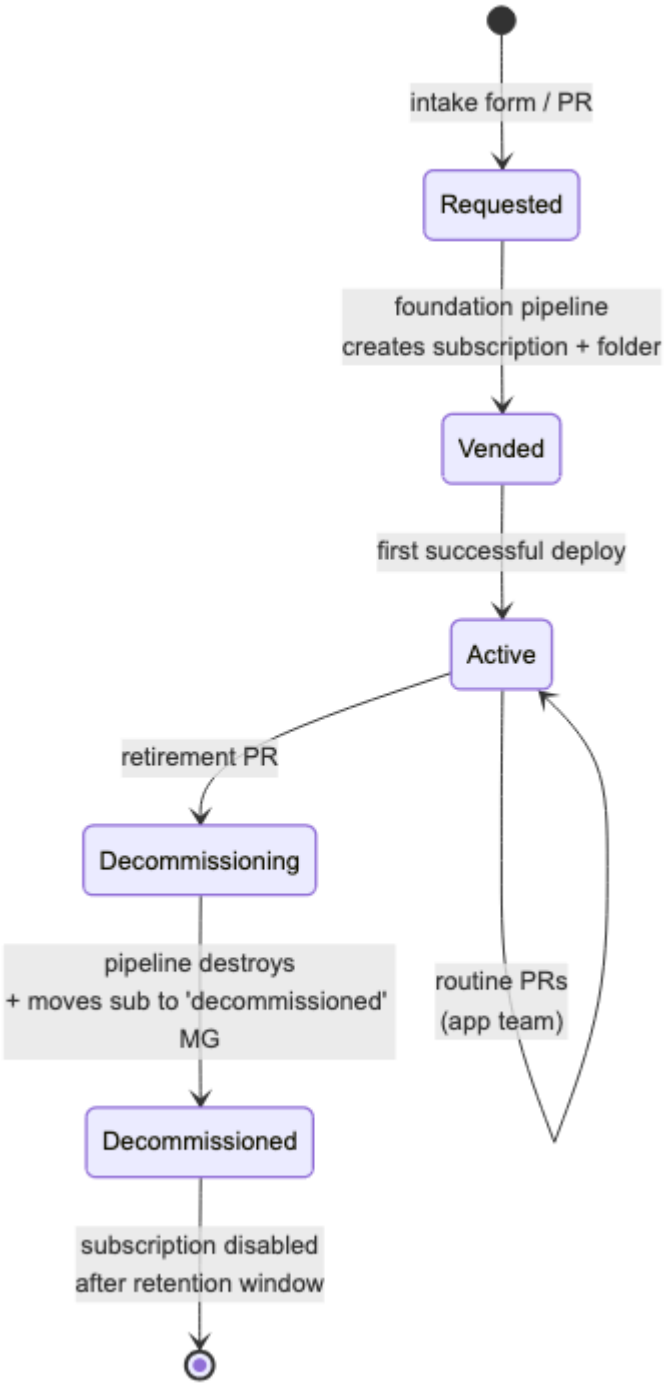
```
AuthorizationResources
| where type =~ "microsoft.authorization/roleassignments"
| where properties.roleDefinitionId endswith "/8e3af657-a8ff-443c-a75c-2fe8c4bcb635"
// Owner
| project subscription = subscriptionId, principalId = properties.principalId, scope =
properties.scope
```

Send the diff vs last week to the security team.

Role assignments are a snapshot of *who can do what right now*. The complementary concern is the lifecycle of the resources those identities manage -- landing zones are created, evolve, and should be retired cleanly when they are no longer needed.

Lifecycle of a landing zone

Define an explicit lifecycle. The repo should support each transition:



Phase	Repo action
Vended	A PR creates a folder under `envs/<env>/lz-<name>/` from a template
Active	Routine PRs from app team; pipeline deploys
Decommissioned	A PR removes the folder; pipeline destroys; the subscription is moved to a `decommissioned` MG and disabled

Decommission must be **first-class**. Without it, your estate accumulates zombie subscriptions forever.

Subscription vending

Two viable patterns:

- **Manual via repo:** A new landing zone PR includes a `subscription.yml` that the foundation pipeline consumes to call the EA / MCA API and create the subscription.
- **Automated via Azure subscription vending API + Service Catalog template:** the LZ accelerator / ALZ vending module handles it.

Either way, **the subscription belongs to the foundation/platform repo**, not the workload repo. The workload repo only consumes the subscription ID.

A well-defined lifecycle tells you *what* needs to happen at each stage. A runbook tells you *exactly how* -- step by step, copy-pasteable -- so that an operation works the first time it is needed, under pressure, by someone who may not have done it before.

Runbooks (in-repo)

Every IaC repo benefits from a `docs/runbooks/` folder with one-pagers for the operations your team actually does:

- `credential-leak.md`
- `state-corruption-recovery.md`
- `force-unlock-state.md`
- `import-existing-resource.md`
- `decommission-landing-zone.md`
- `roll-back-failed-deploy.md`
- `rotate-encryption-key.md`

Each runbook has:

1. **When to use this** (1 sentence)
2. **Prerequisites** (access, tools)
3. **Steps**, copy-pasteable
4. **Verification** of success
5. **Communication** template (who to notify, where)

These get used at 2 a.m. Make them findable, terse, and tested.

Runbooks address the sudden, high-pressure operational events. Cost management addresses the opposite: the slow, silent accumulation of spend that nobody notices until the monthly finance review lands in someone's inbox.

Cost & usage management

Cost is a manageability concern; surface it in the repo:

- Tag every resource with `CostCenter` and `Owner` -- enforced via the policy layer.
- Run Azure Cost Management exports to a storage account, ingest into Log Analytics, and surface per-landing- zone cost in a dashboard.
- For module changes that significantly affect cost (e.g. SKU bump), the PR template includes a "cost impact" field; reviewer asks for an estimate using the Azure Pricing Calculator.
- Set **subscription budgets** in IaC; alert at 50/75/90 % to the LZ owner.

Knowing what your infrastructure costs is one signal. Knowing how the system that *deploys* that infrastructure is behaving is another -- and the pipeline deserves the same observability treatment as any other production service.

Observability of the IaC pipeline itself

The pipeline is a service; treat it as one:

- Send pipeline metrics to Application Insights or LAW: run duration per env, success/failure rate, time-to-production for a typical PR.
- Track lead time and change failure rate (DORA metrics) per repo.
- Alert on:
  - Apply failures in prod (page on-call).
  - Drift detection alerts (queue for triage).
  - Unusual SPN sign-ins (security).
  - Pipelines that haven't run in > N days (dead workloads?).

Anti-patterns

- ❌ **No drift detection.** You are flying blind. A misconfiguration can exist for months before someone trips over it.
- ❌ **CODEOWNERS that lists `@org/everyone`.** Reviews become rubber stamps.
- ❌ **Resource locks that the pipeline SPN bypasses without anyone noticing.** A lock that doesn't lock the deploy identity is theatre.
- ❌ **No decommission process.** Subscriptions accumulate forever and you'll find a "test-sub-2019" still costing $4 k/month.
- ❌ **Runbooks in a wiki nobody can find at 2 a.m.** Keep them with the code.
- ❌ **Engineers fixing prod by editing in the portal "just this once".** Either it goes through code, or your code is no longer authoritative.

Manageability is where the gap between "it worked on day one" and "it still works reliably at year three" lives. The practices in this chapter -- ownership via CODEOWNERS, scheduled drift detection, Deployment Stacks to block drift at the ARM layer, PIM-bounded access, structured

landing-zone lifecycles, in-repo runbooks, and pipeline observability -- do not all need to be in place before you ship. Start with drift detection and CODEOWNERS; the rest grows naturally as the estate matures. Chapter 12 turns to the seemingly mundane but genuinely load-bearing question of what you call things -- and what metadata travels with them.

References

- GitHub, *About code owners*: https://docs.github.com/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners
- Microsoft, *Manage drift in deployment stacks*: https://learn.microsoft.com/azure/azure-resource-manager/bicep/deployment-stacks#protect-managed-resources-against-deletion
- Microsoft, *Azure resource locks*: https://learn.microsoft.com/azure/azure-resource-manager/management/lock-resources
- Microsoft, *Subscription vending*: https://learn.microsoft.com/azure/architecture/landing-zones/subscription-vending
- Microsoft, *PIM*: https://learn.microsoft.com/entra/id-governance/privileged-identity-management/pim-configure
- DORA metrics: https://dora.dev/

← 10 Code quality · Index · 12 Naming & tagging →

12 · Naming, tagging & metadata conventions

In this chapter:

> ***Decision:*** *what do you call things, and what metadata travels with them? The boring stuff that breaks everything if you skip it.*

← 11 Manageability · Index · 13 Documentation →

Every process in a well-run Azure estate -- cost reporting, security automation, decommissioning, disaster recovery -- relies on one assumption: that a resource's name and tags are trustworthy, consistent, and machine-readable. This chapter argues that naming and tagging are not housekeeping but infrastructure, and that the only way to treat them as such is to encode them in code and enforce them with policy.

How we got here

In the early days of every Azure tenant there's an artisanal naming convention written by the first cloud architect on a whiteboard -- and a second one written by their replacement six months later. By 2018 the average enterprise had three: the documented one, the one used for new resources, and the one used by the legacy migration team. Microsoft's **CAF naming guidance** (2019) and the canonical **resource abbreviation list** finally gave the industry a shared vocabulary, and reusable **naming modules** (the `Azure/naming` Terraform module, the `nianton/azure-naming` Bicep module) made the convention executable rather than aspirational.

> 📘 ***Key terms***
>
> ***CAF (Cloud Adoption Framework)*** *-- Microsoft's comprehensive guidance for cloud strategy, governance, and implementation. Its naming and tagging recommendations are the de facto standard for Azure estates.*
>
> ***Resource abbreviations*** *-- a CAF-published list of short prefixes for each Azure resource type (e.g.* `rg-` *for resource groups,* `st` *for storage accounts,* `kv-` *for key vaults).*
>
> ***Modify effect*** *-- an Azure Policy effect that automatically adds or corrects properties on resources (e.g. appending a tag inherited from the resource group) without denying the deployment.*
>
> ***Canonical values*** *-- a fixed, enumerated set of allowed values for a tag or naming segment (e.g.* `env` $\in \{$ `prod`, `nonprod`, `sandbox` $\}$*), enforced by policy to prevent free-text inconsistency.*
>
> ***Data classification*** *-- a tagging practice that labels resources by sensitivity level (e.g.* `Public`, `Internal`, `Confidential`, `Restricted`*) to drive security automation and access control.*

Tagging followed a similar arc: a brief flirtation with "free-text whatever the engineer felt like" gave way to **Azure Policy** `modify` **effects** that auto-append RG tags, and to mandatory tag sets enforced by `deny`. The boring takeaway: **what isn't enforced by code or policy doesn't exist**. Training slides don't enforce; CI does.

Why this gets a whole chapter

Naming and tagging are the **load-bearing infrastructure of every other process**: cost reporting, access control, automation, decommissioning, disaster recovery -- all rely on consistent metadata. A weak convention costs you forever.

The opposite is also true: an over-engineered convention nobody can remember gets violated immediately. Aim for **rigorous but humane**.

With the case for rigour established, here is what rigour looks like in practice -- starting with the naming convention itself.

> ⚖️ ***The debate -- do you even need a naming convention?***
>
> *A provocative but defensible position: name every resource with a random identifier (or let Terraform/Bicep generate one), and rely entirely on* ***tags*** *and* ***Azure Resource Graph queries*** *for discoverability, cost reporting, and ownership. After all, the resource's metadata -- subscription, resource group, type, tags -- already tells you everything the name encodes, and unlike names, tags are mutable when your conventions change. Location in the name? The resource has a* `location` *property. Environment? That's a tag. Why duplicate it in a string with a 24-character limit?*
>
> ***The case for meaningful names:*** *Humans still read names -- in portal URLs,* `az` *CLI output, cost exports, alert emails, and (critically) in 2 a.m. incident logs where you need to recognise the resource before you have time to query its tags. A name like* `kv-platform-prod-swc-01` *is instantly parsed by a human;* `kv-7f3a2c91` *requires a lookup. Community experience on Reddit and in DevOps forums consistently reports that GUID-named estates become unnavigable at scale, especially when tags are inconsistently applied -- which, despite policy enforcement, happens more often than anyone admits.*
>
> ***The case for random/generated names:*** *Naming conventions create a false sense of order. They embed assumptions (region, environment) that become wrong when resources move or conventions evolve. They hit length limits on storage accounts and key vaults, producing ugly truncations. And they tempt engineers into parsing names programmatically instead of using proper metadata -- a brittle pattern that breaks the moment the convention changes. Some teams (particularly those with mature automation and strict tag enforcement) report success with generated names + rich tags.*
>
> ***Where the industry stands (2026):*** *The overwhelming majority of Azure guidance (CAF, WAF, ALZ accelerators) recommends structured naming conventions. But the strongest argument for conventions is a human factors one, not a technical one: people debug faster when names mean something. If your estate is fully automated and humans rarely read raw resource names, the argument weakens. In practice, most teams adopt a convention and enforce tags -- treating names as a human-readable index and tags as the machine-readable source of truth.*

Naming convention

Anatomy

```
<resource-abbr>-<workload>-<env>-<region-abbr>-<instance>
```

Examples:

```
vnet-hub-prod-swc-01
rg-platform-connectivity-prod-swc
sa-tflogs-prod-swc-001       (no dashes, lowercase, ≤ 24 chars)
kv-platform-prod-swc-01      (≤ 24 chars, globally unique)
nsg-app01-web-prod-swc-01
```

Rules:

- Lowercase only. Azure is mostly case-insensitive, but tools, KQL, and bash scripts aren't.
- Use `-` separators except where Azure forbids them (storage, ACR, KV).
- **Region abbreviations** (3 letters): pick once and document. Examples: `swc` (Sweden Central), `weu` (West Europe), `nwe` (North Europe), `wus2` (West US 2). Don't invent new ones; consult your standards page.
- **Instance suffix** (`-01`, `-02`) -- even for resources you expect to be singletons. The day you need a second one, your name doesn't fight you.
- **Avoid embedding subscription / tenant info in names.** Tags and the scope hierarchy already convey it.

Canonical resource abbreviations

Use the Microsoft CAF resource abbreviations list as a starting point. Don't invent your own -- they aren't worth the bus-factor cost.

Length & character constraints

Some Azure resources have nasty limits -- **plan for the worst**:

Resource	Max length	Allowed
Storage account	24	a-z, 0-9
Key Vault	24	a-z, A-Z, 0-9, `-`
ACR	50	a-z, A-Z, 0-9
Function App	60	a-z, A-Z, 0-9, `-`
VM (Windows)	15	a-z, A-Z, 0-9, `-`
Resource Group	90	broad

For names that don't fit your convention, define **shortened forms** explicitly in your naming module -- don't truncate ad-hoc.

Code it, don't print it

Build a **naming module** in `alz-modules`:

```
module "naming" {
  source  = "Azure/naming/azurerm"
  version = "0.4.2"
  suffix  = ["${var.workload}", var.env, local.region_abbr]
}

resource "azurerm_storage_account" "logs" {
  name = module.naming.storage_account.name_unique
  ...
}
```

Or for Bicep, the Azure naming Bicep module (note: this community module has not been actively maintained since 2023; consider Bicep user-defined functions or the Azure Naming Tool as alternatives). Either way, **never let humans type the name into a parameter file** -- typos become permanent.

A consistent naming convention tells you what a resource *is*. A consistent tagging convention tells you who it belongs to, what it costs, and how automation should treat it. The two are complementary, and both require the same discipline.

Tagging convention

Mandatory tags

Every resource (and every resource group) must carry these. Enforce via Azure Policy `Append` (auto-apply RG tags to resources) and `Audit` / `Deny` for the rest.

Tag key	Example	Notes
`CostCenter`	`CC-12345`	Charging code; numeric ideally.
`Owner`	`team-app01@contoso.com`	Group address, not a person.
`Environment`	`prod`, `nonprod`, `sandbox`	Drives a lot of automation.
`Workload`	`app01`, `connectivity-hub`	The "what".
`BusinessUnit`	`corp`, `online`	The "for whom".
`DataClassification`	`public`, `internal`, `confidential`, `restricted`	Drives policy decisions.
`Criticality`	`1`, `2`, `3`, `4`	Tier for SLA / DR.
`ManagedBy`	`iac`, `manual`	Detect drift; expect `iac` everywhere.
`Repo`	`alz-platform`	The Git repo of authority.
`DeploymentId`	`<commit-sha>` or `<run-id>`	Trace a resource to a pipeline run.

Optional but useful

Tag key	Example	Use
`DeleteAt`	`2026-12-31`	For sandbox / time-bounded resources; cleanup automation.
`Project`	`migration-2026`	Programme tracking.
`Compliance`	`pci`, `gdpr`	Regulatory scope.

Inheritance & enforcement

Tags do **not** inherit automatically from RG to resource (despite many tutorials implying so). Implement:

- **Policy `modify` effect** to auto-append RG tags to resources at creation. Use sparingly -- too many `modify` policies make `terraform plan` noisy.
- **Module defaults** -- every module accepts a `tags` map and merges with its own additions. Do **not** rely on `default_tags` in the AzureRM provider for required tags; module-level merging is more explicit.

```
locals {
  base_tags = {
    Environment        = var.env
    Workload           = var.workload
    Owner              = var.owner
    CostCenter         = var.cost_center
    DataClassification = var.data_classification
    BusinessUnit       = var.business_unit
    Criticality        = var.criticality
    ManagedBy          = "iac"
    Repo               = "alz-platform"
    DeploymentId       = var.deployment_id
  }

  tags = merge(local.base_tags, var.additional_tags)
}
```

Tag reporting

Run a weekly Resource Graph query to find non-compliant resources:

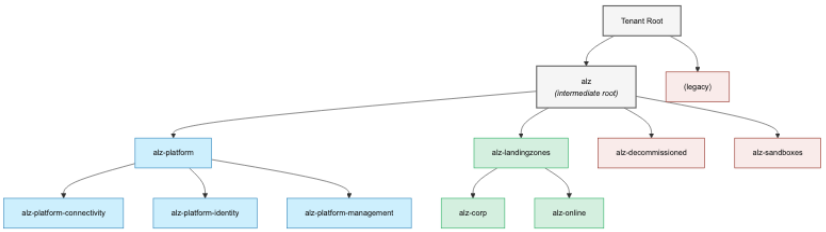
```
Resources
| where tags !has "CostCenter" or tags !has "Owner" or tags !has "Environment"
| project name, type, subscriptionId, resourceGroup, tags
```

Send the count + worst offenders to a dashboard; gate the next quarterly budget on improvement.

Resource-level naming and tagging address the workload layer. The same principles apply one level up, where management groups and subscriptions form the structural skeleton of the tenant -- and where an inconsistent naming scheme will haunt your governance queries for years.

Management group / subscription naming

These are top-level too -- bake them into the foundation:



Subscription names mirror the MG path:

```
sub-platform-connectivity-prod
sub-corp-app01-prod
sub-corp-app01-nonprod
sub-sandbox-jdoe
```

Document the tree in the foundation repo's `docs/management-groups.md`.

Management group and subscription names establish *what* things are and *whose* they are. Region strategy is a natural extension of the same discipline: capping the allowed set of deployment locations is one of the simpler, highest-leverage policy decisions you will make.

Region strategy

Pick a small number of **primary regions** (typically 2 per geo for DR pairing) and don't deploy outside them without an exception process.

- **Primary:** `swedencentral`
- **Paired:** `northeurope` (DR target for Sweden)
- **Approved exception:** `westeurope` (legacy)

Bake the allowed list into a policy:

```
// policies/restrict-regions.bicep
param allowedLocations array = [
  'swedencentral'
  'northeurope'
  'westeurope'
]
```

The policy assignment is **`deny`** in landing zones; **`audit`** in sandbox.

Anti-patterns

- ❌ **Inventing your own resource abbreviations.** Use Microsoft's CAF list; the bus factor is too high otherwise.
- ❌ **Free-text tags** (`Owner: "John (he sits next to Sara)"`). Tags are data; treat them as such.
- ❌ **Required tags enforced only by training docs.** People forget. Enforce with policy.
- ❌ **Names that include the subscription ID** ("just in case"). The scope is already there.
- ❌ **`Environment` tag values that drift** (`Prod`, `prod`, `Production`). Pick canonical values and `Deny` anything else with policy.
- ❌ **Tagging at the resource level only**, ignoring RG. RG tags are the inheritance source for `modify` policies.
- ❌ **Naming module added "later".** Names are the hardest thing to refactor -- costs days of pipeline work per resource type.

Naming and tagging may feel like administrative overhead until the day someone asks "which team owns this resource, and what does it cost them?" and the answer cannot be retrieved in under thirty seconds. Enforce the convention from day one -- it is genuinely cheaper to build right than to rename a thousand resources later. The next chapter turns to the documentation that explains *why* all these decisions were made, so the engineer who joins two years from now understands the system rather than simply inheriting it.

References

- Microsoft, *CAF -- Naming and tagging*: https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-best-practices/naming-and-tagging
- Microsoft, *Resource abbreviations*: https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-best-practices/resource-abbreviations
- Microsoft, *Resource naming rules*: https://learn.microsoft.com/azure/azure-resource-manager/management/resource-name-rules
- `Azure/naming` Terraform module: https://registry.terraform.io/modules/Azure/naming/azurerm/latest
- `nianton/azure-naming` Bicep module *(inactive since 2023)*: https://github.com/nianton/azure-naming
- Azure Naming Tool (actively maintained alternative): https://github.com/mspnp/AzureNamingTool

---

← 11 Manageability · Index · 13 Documentation →

13 · Documentation & onboarding

In this chapter:

> ***Decision:*** *what gets written, where, and who keeps it current? Documentation that lives outside the repo decays; documentation in the repo doesn't write itself.*

← 12 Naming & tagging · Index · 14 Anti-patterns →

Code tells you *what* the system does. Documentation tells you *why* it does it, *how* to change it safely, and *where* to start when everything is on fire. Most IaC repos skimp on all three -- and then wonder why onboarding takes three months and runbooks get executed incorrectly under pressure. This chapter sets out a disciplined, minimal approach to documentation that lives in the repo, decays slowly, and actually gets used.

How we got here

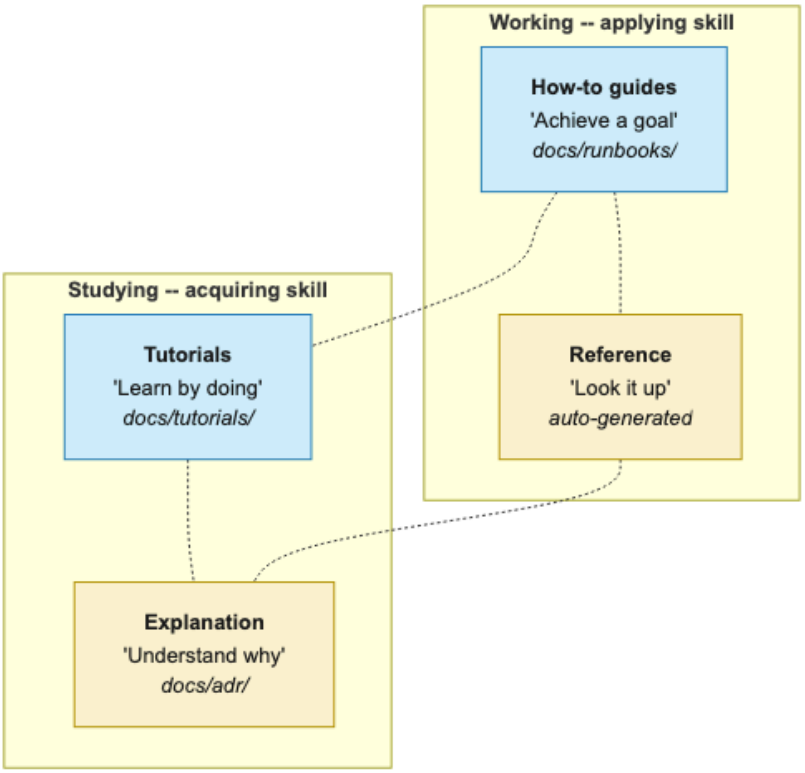
Pre-2010, infrastructure documentation lived in Word documents on a shared drive nobody had write access to. The wiki era (Confluence, SharePoint, MediaWiki) was a step forward in *editability* but a step backward in *trustworthiness* -- there was no diff, no PR, no link to the code, and the docs decayed silently. Two ideas fixed this. First, **Architecture Decision Records** -- a one-page format proposed by Michael Nygard in 2011 -- gave teams a way to capture *why* a decision was made, immutably and in version control. Second, the **Diátaxis framework** (Daniele Procida, 2017-2021) explained *why* most documentation is bad: it conflates tutorials, how-tos, reference, and explanation into one unreadable mess. Tooling caught up: **Mermaid** landed in GitHub markdown in 2022, killing the binary-diagram era; **D2** and **Excalidraw** gave teams text-based architecture diagrams; and `terraform-docs` / **PSDocs** automated the reference layer. The modern IaC repo's docs/ folder is a small, disciplined library -- not a dumping ground.

> 📘 ***Key terms***
>
> ***ADR (Architecture Decision Record)*** *-- a short, immutable document capturing a single architectural decision: the context, the decision itself, and the consequences. Stored in* `docs/adr/` *and numbered sequentially.*
>
> ***Diátaxis*** *-- a documentation framework by Daniele Procida that classifies all technical content into exactly four types (tutorials, how-to guides, reference, explanation), arguing that mixing types in one document is the most common documentation failure.*
>
> ***Mermaid*** *-- a text-based diagramming language rendered natively by GitHub in markdown code blocks (* ```` ```mermaid ```` *). Supports flowcharts, sequence diagrams, state diagrams, and more.*
>
> ***D2*** *-- a modern, text-based diagram scripting language designed for software architecture diagrams, with auto-layout and theming.*
>
> ***Excalidraw*** *-- a virtual whiteboard tool that produces hand-drawn-style diagrams, storable as JSON and embeddable in repos.*
>
> ***PSDocs*** *-- a PowerShell-based documentation generator that produces markdown from Azure resource templates, analogous to* `terraform-docs` *for Bicep/ARM.*

Not every piece of content that belongs in that library is the same kind of thing -- and treating them as interchangeable is the first step towards a docs folder nobody trusts. The Diátaxis framework provides a useful taxonomy.

The four kinds of documentation

Inspired by the Diátaxis framework -- every piece of documentation should be exactly one of:



Left column = practical steps · Right column = theoretical understanding

Type	Purpose	Where in the repo
Tutorials	Hand-hold a newcomer to first success	`docs/tutorials/`
How-to guides	Achieve a specific outcome	`docs/runbooks/` , `docs/howto/`
Reference	Look up exact details	Auto-generated from code
Explanation	Understand *why*	`docs/adr/` , this guide

Mixing types in one document is the most common documentation failure.

With the taxonomy established, it is worth working through the documents themselves -- starting with the one every reader encounters first.

Top-level `README.md`

The first thing every reader sees. Aim for **one screen**:

```
# alz-platform

Infrastructure-as-Code for Contoso's Azure Landing Zone -- platform layer.

## Quick start

git clone ... && cd alz-platform
make bootstrap
make plan ENV=nonprod WORKLOAD=connectivity

## Where things live

- envs/        Per-environment compositions
- modules/     Local pattern modules (tier 2)
- policies/    Custom Azure Policy definitions
- docs/        Architecture, ADRs, runbooks
- .github/     Pipelines, CODEOWNERS

## Documentation

- [Architecture overview](docs/architecture.md)
- [Architecture Decision Records](docs/adr/)
- [Runbooks](docs/runbooks/)
- [Contributing](CONTRIBUTING.md)

## Owners

Platform engineering · #alz-platform · platform@contoso.com
```

Anything more goes behind a link. Long READMEs go unread.

A good README is a table of contents for the rest of the documentation. The most important item it should point to is the ADR log -- the record of *why* the system is built the way it is, without which every architectural choice becomes oral tradition that disappears when the original architect leaves.

Architecture Decision Records (ADRs)

Every non-trivial decision (this guide is full of them) should be captured as a short, dated, immutable record.

`docs/adr/0001-monorepo-or-multirepo.md`:

```
# 0001 · Repository topology -- layered few-repo

- Status: Accepted
- Date: 2026-04-12
- Deciders: @jane.platform, @joe.architect
- Tags: topology, governance

## Context
We need to host IaC for a tenant containing ~30 landing zones across 4 BUs.
The platform team owns connectivity/identity/management, app teams own
their landing zones.

## Decision
Adopt three repositories: `alz-foundation`, `alz-platform`, and one
`alz-landingzones-<bu>` per BU. Modules live in a separate `alz-modules`
repo published to ACR.

## Consequences
+ Each repo has a coherent CODEOWNERS and approval policy.
+ Platform releases independent of workload releases.
- Cross-cutting changes require coordination via the modules repo.
- Need a pipeline-templates repo to keep CI consistent.

## Alternatives considered
- Monorepo: rejected for blast-radius and CODEOWNERS complexity at our scale.
- Pure multi-repo: rejected -- discoverability cost too high without a
  developer portal.

## References
- docs/01-repository-topology.md
```

ADRs are **never edited**. If the decision changes, write a new ADR that *supersedes* the old one and link them.

Tools:

- `adr-tools` -- `adr new "Topology"`.
- `log4brains` -- renders ADRs as a static site.

ADRs capture decisions in prose; some decisions -- particularly around network topology and identity flows -- are better expressed visually. The challenge is keeping those visuals version-controlled and diffable, rather than letting them become the same undiffable artefacts as the Word documents they replaced.

Diagrams as code

Binary `.drawio` / Visio files in Git are an anti-pattern: undiffable, unmergeable, immediately stale.

Use diagrams as code:

Tool	Best for
Mermaid	Sequence, flowcharts, class -- renders natively in GitHub markdown.
Excalidraw	Sketch-style architecture; export `.excalidraw` (JSON) + PNG.
D2	Architecture diagrams with great auto-layout.
PlantUML	Sequence/component diagrams; long history.
Draw.io with `.drawio.svg`	If you must -- the SVG variant is text and renders as PNG on GitHub.

Keep both source (`*.mmd`, `*.d2`, `*.excalidraw`) and rendered PNG/SVG so the README looks right without local tooling. CI re-renders on PR.

Embed in markdown:

````
```mermaid
flowchart LR
  Hub[(Hub VNet)] --- Spoke1[App01]
  Hub --- Spoke2[App02]
  Hub --- ER[ExpressRoute]
```
````

Auto-generated reference

Reference docs **cannot** be hand-written reliably. Generate them:

- **Module READMEs:** `terraform-docs` or `PSDocs.Azure` -- see 10 code quality.
- **Pipeline reference:** parse workflow YAML; produce a list of inputs, required secrets, and called workflows.
- **Policy reference:** generate a markdown table from your policy definitions JSON.
- **Naming convention:** generate a table of all resource abbreviations from your naming module.

CI runs the generators and fails the build if the generated output differs from the committed copy. This is **the only way** reference docs stay correct.

CONTRIBUTING.md

Tells new contributors how to play nicely:

- How to set up the dev environment (link to README quick start).
- PR conventions (Conventional Commits, PR template).
- Required checks and how to run them locally.
- Review process (who, when, SLA).
- Where to ask questions (channel, office hours).

Keep it under 200 lines.

CHANGELOG.md

Auto-generated from Conventional Commits via `release-please`. The repo itself doesn't need a hand-written changelog if releases are tagged consistently -- the GitHub Releases page *is* your changelog.

For module repos this is non-negotiable; consumers depend on it.

Onboarding checklist

A new engineer joining the platform team gets a checklist issue auto-created from a template:

```
## Onboarding · @newjoiner

- [ ] Access requested: Azure (Reader on all subs), GitHub team membership
- [ ] Devcontainer or local toolchain working: `make bootstrap`
- [ ] First successful sandbox plan: `make plan ENV=sandbox`
- [ ] Read: README, this guide (docs/), top 5 ADRs
- [ ] Pair on first PR (assigned: @buddy)
- [ ] Pager rotation onboarding (assigned: @oncall-lead)
- [ ] Shadow a deploy to prod (assigned: @release-eng)
```

The checklist itself is a documented artifact: improvements come from new joiners filing PRs against the template.

Architecture overview document

`docs/architecture.md` -- the **single canonical map** of the estate:

- Diagram of management group hierarchy.
- Diagram of network topology.
- Diagram of identity model (Entra tenants, MIs, federated creds).
- List of subscriptions with owner and purpose.
- List of regions and their role.

Update on every significant change. CI verifies the diagram source files exist; the human verifies they're current -- quarterly review on the team's calendar.

"Why" lives in commit messages and ADRs, not in code comments

Code comments rot. Commit messages and ADRs are immutable. Prefer:

```
# ❌ Comment that explains a workaround
resource "azurerm_role_assignment" "ra" {
  # Wait 30 seconds because role propagation lags - issue #1234
  ...
}
```

...over a comment, write the **commit message** that adds it explicitly:

```
fix(identity): wait 30s after role assignment

AzureRM #20871 -- RBAC propagation can take up to 30 seconds. Without
this delay, the next module fails with AuthorizationFailed.

Removable when the upstream issue is closed.
```

Comments belong in code only when they explain *what* the code does in a way the code cannot. *Why* belongs in commits and ADRs.

The discipline of routing *why* through commits and ADRs rather than code comments is the capstone of a documentation culture. The anti-patterns below catalogue the ways that culture most commonly collapses in practice.

Anti-patterns

- ❌ **Wiki-only documentation** (Confluence, SharePoint, Loop). Decays fast, can't be PR-reviewed, can't be searched alongside code.
- ❌ **The "TODO: document this" comment.** Open an issue or write the doc -- TODOs go unread.
- ❌ **Architecture diagrams in proprietary binary formats.** No diff, no merge, no render in PR.
- ❌ **CHANGELOG hand-written and forever stale.** Generate it.
- ❌ **One giant `docs/all.md`.** Split. Use the index.
- ❌ **Documenting *implementation* instead of *design*.** Code is the authoritative implementation; docs explain *why* and *how to use*.

Documentation is not a deliverable that ships alongside the code -- it is the medium through which the code communicates its intent to the engineers who will work with it for the next five years. The practices in this chapter -- Diátaxis-informed structure, immutable ADRs, generated reference

docs, version-controlled runbooks, and an onboarding checklist that improves with every new joiner -- require no heroic effort. They require the same discipline applied to everything else in this guide: a clear convention, enforced consistently, with the repo as the single source of truth. That discipline is the thread running through all fourteen chapters; the next and final chapter draws it together in a consolidated checklist of everything that can go wrong -- and how to check whether it already has.

References

- Diátaxis: https://diataxis.fr/
- MADR -- Markdown ADR template: https://adr.github.io/madr/
- `adr-tools`: https://github.com/npryce/adr-tools
- GitHub markdown -- Mermaid: https://github.blog/developer-skills/github/include-diagrams-markdown-files-mermaid/
- D2: https://d2lang.com/
- `terraform-docs`: https://terraform-docs.io/

← 12 Naming & tagging · Index · 14 Anti-patterns →

14 · Anti-patterns & common pitfalls

In this chapter:

> *A consolidated checklist of mistakes seen in real ALZ implementations. If you find yourself doing any of these, stop and reconsider.*

← 13 Documentation · Index · References →

Thirteen chapters of design decisions, tradeoffs, and recommendations culminate here. This chapter is the consolidation -- a catalogue of every significant mistake this guide has warned against, organised by the same categories you have been building through: repository structure, tooling, authentication, pipelines, modules, state, testing, and operations. Think of it as the final exam -- not a test of whether you remember the theory, but a structured exercise in honestly assessing how close your current implementation is to the practices this guide advocates. No real-world ALZ implementation scores perfectly on first review; the point is to identify the gaps, prioritise them by risk, and address them through the same PR process as everything else. Read each item as a question: *Is this us?*

Repository & code structure

- ❌ **One mega-repo with one giant Terraform state.** Blast radius = entire estate. Split state by (environment × workload). See 01, 07.
- ❌ **One repo per resource group.** Granularity gone mad; more pipeline glue than infrastructure.

- ❌ **Mixing application source code with IaC.** Different cadences, reviewers, and security models. Split.
- ❌ **Copy-pasted "modules" folder across many repos.** Promote to a shared module repo with a registry.
- ❌ **Branch-per-environment** (`dev`, `staging`, `main`) with cherry- picking between them. You will lose a hotfix eventually.
- ❌ **Tests, modules, and envs all jumbled together.** A consistent layout is a load-bearing readability feature.

Tooling & versioning

- ❌ **Large ARM JSON templates without tooling assistance.** Raw ARM JSON is 3-5× more verbose than Bicep or Terraform equivalents, making PR reviews harder, merge conflicts more frequent, and resource references error-prone (`[resourceId(...)]` chains). The VS Code ARM extension and ARM TTK mitigate some of this, but at scale a higher-level language (Bicep, Terraform, Pulumi) with modules, loops, and type-checked references pays for itself quickly. See 02.
- ❌ **Floating versions** (`source = "...?ref=main"`, `version = "latest"`). Reproducibility gone.
- ❌ **GitHub Actions referenced by mutable tags** (`@v3`). Pin by SHA.
- ❌ **Two IaC engines managing the same resource.** Drift wars.
- ❌ **Pulumi adopted by an ops-background team without a software engineering culture.** Becomes spaghetti.
- ❌ **Provider versions unlocked.** A provider minor bump can rewrite half your plan unexpectedly.

Authentication & secrets

- ❌ **Long-lived service principal client secret in CI.** Use OIDC federation. Always.
- ❌ **One SPN with Owner @ tenant root for "convenience".** Catastrophic blast radius.
- ❌ **Federated credential subject `repo:org/repo:*`.** Defeats the purpose of federation.
- ❌ **Same identity used by humans and pipelines.** Audit becomes impossible.
- ❌ **Secrets stored in `terraform.tfvars` "just for now".** State files also leak them -- see 07.
- ❌ **Adding `--allow-secret` to bypass scanners.** That's how real secrets get through.

Pipelines & CI/CD

- ❌ **`apply` directly without an explicit `plan` artifact.** What was reviewed is not necessarily what was applied.
- ❌ `continue-on-error: true` to "make the build green" while investigating. Permanent.

- ❌ `pull_request_target` **with checkout of PR code from forks.** Privilege escalation vector.
- ❌ `cancel-in-progress: true` **on the deploy concurrency group.** An in-flight `apply` should never be cancelled.
- ❌ **One pipeline that deploys all environments serially.** Use proper environment gates between non-prod and prod.
- ❌ **Per-repo bespoke workflows.** Drift within a quarter. Use reusable workflows.
- ❌ **No drift detection.** You're flying blind.

Modules & registries

- ❌ **Inlined modules duplicated between landing zones.** "We'll DRY it later" never comes.
- ❌ **A "kitchen sink" module** with 80 boolean toggles. Split it.
- ❌ **Calling AVM resource modules directly from landing-zone code.** No place to enforce enterprise opinions.
- ❌ **Wrapping AVM with a module that adds nothing.** Just call AVM directly.
- ❌ **Releasing a major version without a** `MIGRATION.md`**.** Consumers can't safely upgrade.
- ❌ **Pinning to a branch name** (`?ref=main`). "Worked yesterday".

State management

- ❌ **State backend in the same subscription as the resources it manages.** A bad apply could nuke its own state storage. Separate sub.
- ❌ **State on engineer laptops.** Even in dev. Especially in dev.
- ❌ `terraform_remote_state` **cross-references that leak producer secrets to consumers.** Use App Configuration outputs.
- ❌ **Editing state files by hand.** Even with `jq`.
- ❌ **Running** `force-unlock` **reflexively when a lock appears.** Investigate first -- the previous run might still be applying.
- ❌ **Bicep prod deployments without Deployment Stacks.** No drift protection, no managed-resources tracking.

Testing & policy

- ❌ **All policy lives at PR time.** Anyone clicking in the portal bypasses your controls.
- ❌ **All policy lives in Azure Policy.** Developers find out at deploy time after a 15-min plan. Move what you can earlier.
- ❌ `soft_fail: true` **on Checkov to "fix later".** Later never comes.

- ❌ **Integration tests against a shared sub with hard-coded names.** Two PRs collide → both fail.
- ❌ **45-minute test suites.** Engineers will avoid them. Trim or parallelise.
- ❌ **Custom rules in Rego when a built-in PSRule rule exists.** Reinvented wheels rust.

Code quality / DX

- ❌ **README that says "install Terraform 1.x"** without pinning.
- ❌ **Linters that produce warnings nobody reads.** Either fail the build or remove them.
- ❌ **Module READMEs written by hand.** Drift in one PR.
- ❌ `make apply` **that works against prod from a laptop.** Pipeline-only.
- ❌ **Devcontainer that nobody uses.** Make it the only path or it dies.

Manageability / day-2

- ❌ **CODEOWNERS =** `@org/everyone`**.** Reviews become rubber stamps.
- ❌ **Resource locks that the pipeline SPN bypasses without anyone noticing.** Theatre.
- ❌ **No decommission process.** Subscriptions accumulate forever.
- ❌ **Runbooks in a wiki nobody can find at 2 a.m.** Keep them with the code.
- ❌ **Engineers fixing prod by editing in the portal "just this once".** Either it goes through code or your code is no longer authoritative.
- ❌ **Drift alerts that nobody triages.** Either set an SLA or remove the alert.

Naming & tagging

- ❌ **Inventing your own resource abbreviations.** Use Microsoft's CAF list.
- ❌ **Free-text tags (**`Owner: "John (he sits next to Sara)"`**).** Tags are data.
- ❌ `Environment` **values that drift (**`Prod`**,** `prod`**,** `Production`**).** Pick canonical values; deny the rest.
- ❌ **Required tags enforced only by training docs.** Enforce with policy.
- ❌ **Naming module added "later".** Names are the hardest thing to refactor.

Documentation

- ❌ **Wiki-only documentation** (Confluence, SharePoint). Decays fast, can't be PR-reviewed.
- ❌ **Architecture diagrams in proprietary binary formats.** No diff, no merge.
- ❌ **TODO comments instead of issues.** Go unread.
- ❌ **One giant** `docs/all.md`**.** Split.
- ❌ **Documenting implementation rather than design.** Code is the implementation; docs explain *why*.

Operational

- ❌ **Pager configured but no on-call rota.** Alerts go nowhere at 2 a.m.
- ❌ **No SLO on time-to-production for a typical PR.** Improvement is invisible without a baseline.
- ❌ **No mechanism to retire old subscriptions.** Cost balloons silently.
- ❌ **No fire drills** for state recovery, credential leak, full-estate outage. The first time you do it should not be in production.

Working through this list is most useful not as a one-time audit but as a recurring exercise -- quarterly for a mature platform, monthly when the estate is growing quickly. When you find an anti-pattern you are currently living with, resist the temptation to note it privately and move on. Open an issue, assign an owner, and track its remediation alongside everything else the team ships. The self-assessment below gives you a structured framework for that conversation -- a score honest enough to be useful and specific enough to drive action.

A "is your repo healthy?" self-assessment

Score yourself out of 20:

- ☐ State files are split per (env × workload).
- ☐ No long-lived Azure secrets in any CI system.
- ☐ OIDC federation in use, with restrictive `subject` claims.
- ☐ Modules pinned by exact version, never by branch.
- ☐ Reusable pipeline templates shared across repos.
- ☐ Pre-commit hooks installed and run in CI.
- ☐ Plan posted as a PR comment for every PR.
- ☐ Policy gates run at PR time *and* at runtime.
- ☐ CODEOWNERS exist and are reviewed quarterly.
- ☐ Branch protection requires CODEOWNERS + signed commits.
- ☐ Drift detection runs weekly per state file.
- ☐ Required tags enforced via Azure Policy.
- ☐ Module READMEs auto-generated and CI-verified.
- ☐ Devcontainer or `mise` / `asdf` pinning all toolchain versions.
- ☐ At least 5 ADRs in `docs/adr/`.
- ☐ At least 3 runbooks in `docs/runbooks/`.
- ☐ Decommission process documented and exercised.
- ☐ Naming convention enforced via a naming module.
- ☐ Storage account hosting state is private endpoint + CMK only.
- ☐ State recovery has been tested in the last 6 months.

Score	Verdict
18-20	World class -- write a blog post.
14-17	Solid -- pick the next two and improve.
10-13	Functional -- significant risk in 1-2 areas.
< 10	Stop and re-plan before scaling further.

← 13 Documentation · Index · References →

References & further reading

In this chapter:

← 14 Anti-patterns · Index

A consolidated list of the external references cited across this guide.

Microsoft -- Cloud Adoption Framework

- CAF Landing Zones overview -- https://learn.microsoft.com/azure/cloud-adoption-framework/ready/landing-zone/
- CAF Landing Zone implementation options -- https://learn.microsoft.com/azure/cloud-adoption-framework/ready/landing-zone/implementation-options
- CAF Naming and tagging -- https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-best-practices/naming-and-tagging
- CAF Resource abbreviations -- https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-best-practices/resource-abbreviations
- CAF Environments -- https://learn.microsoft.com/azure/cloud-adoption-framework/ready/considerations/environments
- Subscription vending -- https://learn.microsoft.com/azure/architecture/landing-zones/subscription-vending

Microsoft -- Bicep & Deployment Stacks

- Bicep documentation -- https://learn.microsoft.com/azure/azure-resource-manager/bicep/
- Deployment stacks -- https://learn.microsoft.com/azure/azure-resource-manager/bicep/deployment-stacks
- Private module registry -- https://learn.microsoft.com/azure/azure-resource-manager/bicep/private-module-registry
- What-if -- https://learn.microsoft.com/azure/azure-resource-manager/templates/deploy-what-if

Microsoft -- Identity & Security

- Workload identity federation -- https://learn.microsoft.com/entra/workload-id/workload-identity-federation
- Federated identity credentials on managed identities -- https://learn.microsoft.com/entra/identity/managed-identities-azure-resources/how-manage-federated-identity-credentials
- Privileged Identity Management -- https://learn.microsoft.com/entra/id-governance/privileged-identity-management/pim-configure
- Defender for DevOps -- https://learn.microsoft.com/azure/defender-for-cloud/defender-for-devops-introduction
- Storage security baseline -- https://learn.microsoft.com/security/benchmark/azure/baselines/storage-security-baseline

Microsoft -- Governance & Policy

- Azure Policy as code -- https://learn.microsoft.com/azure/governance/policy/concepts/policy-as-code
- Azure Resource Locks -- https://learn.microsoft.com/azure/azure-resource-manager/management/lock-resources
- Resource naming rules -- https://learn.microsoft.com/azure/azure-resource-manager/management/resource-name-rules

Azure Verified Modules

- AVM home -- https://aka.ms/avm
- AVM Bicep specs -- https://github.com/Azure/bicep-registry-modules
- AVM Terraform template -- https://github.com/Azure/terraform-azurerm-avm-template

ALZ accelerators

- ALZ-Bicep (Classic -- entering extended support) -- https://github.com/Azure/ALZ-Bicep
- Terraform CAF/ESLZ module (Classic -- entering extended support, archived Aug 2026) -- https://github.com/Azure/terraform-azurerm-caf-enterprise-scale
- ALZ accelerator (AVM-based, current) -- https://azure.github.io/Azure-Landing-Zones/accelerator/
- ALZ Terraform migration guide -- https://aka.ms/alz/tf/migrate
- ALZ PowerShell module -- https://github.com/Azure/ALZ-PowerShell-Module
- ALZ Portal accelerator -- https://aka.ms/alz/portal

Terraform / OpenTofu

- AzureRM provider -- https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs
- AzAPI provider -- https://registry.terraform.io/providers/Azure/azapi/latest/docs
- Backend `azurerm` -- https://developer.hashicorp.com/terraform/language/settings/backends/azurerm
- Recommended practices -- https://developer.hashicorp.com/terraform/cloud-docs/recommended-practices
- `terraform test` -- https://developer.hashicorp.com/terraform/language/tests
- `import` blocks -- https://developer.hashicorp.com/terraform/language/import
- Terratest -- https://terratest.gruntwork.io/
- OpenTofu -- https://opentofu.org/

GitHub

- Reusable workflows -- https://docs.github.com/actions/using-workflows/reusing-workflows
- Required workflows -- https://docs.github.com/actions/using-workflows/required-workflows
- Environments -- https://docs.github.com/actions/deployment/targeting-different-environments/using-environments-for-deployment
- Configuring OIDC in Azure -- https://docs.github.com/actions/deployment/security-hardening-your-deployments/configuring-openid-connect-in-azure
- Security hardening for Actions -- https://docs.github.com/actions/security-guides/security-hardening-for-github-actions
- Secret scanning -- https://docs.github.com/code-security/secret-scanning
- Artifact attestations -- https://docs.github.com/actions/security-guides/using-artifact-attestations-to-establish-provenance-for-builds
- CODEOWNERS -- https://docs.github.com/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners

Azure DevOps

- Workload identity federation for service connections -- https://learn.microsoft.com/azure/devops/pipelines/library/connect-to-azure
- Branch policies -- https://learn.microsoft.com/azure/devops/repos/git/branch-policies
- YAML templates -- https://learn.microsoft.com/azure/devops/pipelines/process/templates

Policy / scanning tools

- PSRule for Azure -- https://azure.github.io/PSRule.Rules.Azure/
- Checkov -- https://www.checkov.io/
- tfsec / Trivy -- https://aquasecurity.github.io/tfsec/
- Conftest (OPA/Rego) -- https://www.conftest.dev/
- OPA / Rego -- https://www.openpolicyagent.org/docs/latest/policy-language/
- Sentinel -- https://developer.hashicorp.com/sentinel

Developer experience

- `pre-commit` -- https://pre-commit.com/
- `pre-commit-terraform` -- https://github.com/antonbabenko/pre-commit-terraform
- `terraform-docs` -- https://terraform-docs.io/
- `tflint` -- https://github.com/terraform-linters/tflint
- Devcontainers -- https://containers.dev/
- `mise` -- https://mise.jdx.dev/
- `release-please` -- https://github.com/googleapis/release-please
- Conventional Commits -- https://www.conventionalcommits.org/
- SemVer -- https://semver.org/
- `actionlint` -- https://github.com/rhysd/actionlint
- `zizmor` -- https://github.com/woodruffw/zizmor
- `gitleaks` -- https://github.com/gitleaks/gitleaks
- Sigstore `gitsign` -- https://github.com/sigstore/gitsign
- Renovate -- https://docs.renovatebot.com/

Documentation

- Diátaxis -- https://diataxis.fr/
- MADR -- Markdown ADR template -- https://adr.github.io/madr/
- `adr-tools` -- https://github.com/npryce/adr-tools

- GitHub Mermaid -- https://github.blog/developer-skills/github/include-diagrams-markdown-files-mermaid/
- D2 -- https://d2lang.com/

Methodology / metrics

- DORA metrics -- https://dora.dev/
- SLSA -- https://slsa.dev/
- Trunk-based development -- https://trunkbaseddevelopment.com/

Community video resources

- **ALZ Weekly Questions** -- YouTube playlist with best practices and Q&A from the ALZ product team: https://www.youtube.com/playlist?list=PLukmLwi08-bTb4jpVTGdgFi5adbzj54jj
  - Week 1: How to use AVM in ALZ, Bicep or Terraform? -- https://www.youtube.com/watch?v=ry39tWr_SXc
  - Week 2: When to use SLZ over ALZ? + alzlibtool -- https://www.youtube.com/watch?v=r8h7F6IJIqw
  - Week 3: How to Stay Current with ALZ Azure Policies -- https://www.youtube.com/watch?v=ddcVKS_MKkk
  - Week 4: Using EPAC for Azure Policy in ALZ -- https://www.youtube.com/watch?v=x1I_XhC6GtA
  - Week 5: Multi-Region ALZ Expansion -- https://www.youtube.com/watch?v=nii_LF65zyE
  - Week 6: Understanding ALZ Bicep File Structure -- https://www.youtube.com/watch?v=sPA3YWkQ-4s
  - Week 7: Migrating from CAF-Enterprise-Scale to AVM -- https://www.youtube.com/watch?v=DSBWjQlVpSs
  - Week 9: Community ALZ Projects + Right-Sized ALZ -- https://www.youtube.com/watch?v=MznCQbT-EZw
  - Week 10: Subscription Vending: Repo Structure, Security & Multi-Tenant -- https://www.youtube.com/watch?v=11PmT0t6TUI
  - Week 11: Azure Migrate Agent Preview -- https://www.youtube.com/watch?v=kFS9lNPuXxM
  - Week 12: Private DNS Zones: Centralized vs Decentralized -- https://www.youtube.com/watch?v=WnrKlflIiwo
  - ALZ for SMB -- https://www.youtube.com/watch?v=cyLhLJEYIkU
  - Azure Migrate Agent Deep Dive -- https://www.youtube.com/watch?v=ODaFlsja308

← 14 Anti-patterns · Index